Keras 3: From First Layer to Production

Measured on Keras 3.15.1 and TensorFlow 2.21.0, CPU only

16 chapters · 31,943 words

Every number is reproduced by code that ships with the book.

Contents

Chapters

Chapter 1 — The first model 4
Chapter 2 — Tensors and ops 12
Chapter 3 — Layers and weights 18
Chapter 4 — Three ways to build a model 28
Chapter 5 — Data pipelines 35
Chapter 6 — Losses, metrics and optimizers 43
Chapter 7 — Callbacks and training control 52
Chapter 8 — Overfitting and regularization 61
Chapter 9 — Custom losses, metrics and training steps 67
Chapter 10 — Convolutional networks 77
Chapter 11 — Sequences 84
Chapter 12 — Attention and transformers 93
Chapter 13 — Transfer learning 100
Chapter 14 — Saving, loading and export 110
Chapter 15 — Performance and reproducibility 120
Chapter 16 — A field guide 127

Appendices

Appendix A — Described, not built 132

Exercises

Exercises — Chapter 1 133
Exercises — Chapter 2 134
Exercises — Chapter 3 135
Exercises — Chapter 4 136
Exercises — Chapter 5 137
Exercises — Chapter 6 138
Exercises — Chapter 7 139
Exercises — Chapter 8 140
Exercises — Chapter 9 141
Exercises — Chapter 10 142
Exercises — Chapter 11 143
Exercises — Chapter 12 144
Exercises — Chapter 13 145
Exercises — Chapter 14 146
Exercises — Chapter 15 147
Exercises — Chapter 16 148

Worked solutions

Solutions — Chapter 1 149
Solutions — Chapter 2 151

Solutions — Chapter 3 153
Solutions — Chapter 4 155
Solutions — Chapter 5 157
Solutions — Chapter 6 159
Solutions — Chapter 7 161
Solutions — Chapter 8 163
Solutions — Chapter 9 165
Solutions — Chapter 10 168
Solutions — Chapter 11 171
Solutions — Chapter 12 174
Solutions — Chapter 13 177
Solutions — Chapter 14 180
Solutions — Chapter 15 182
Solutions — Chapter 16 184

Chapter 1 — The first model

> **Build** — a model with one weight and one bias that learns the line y = 2x + 1, and a small table that shows how the optimizer and the learning rate decide whether it learns at all.
>
> **Theory kept** — a model is a function with weights. Training changes the weights to make a number, the loss, smaller.
>
> **Source** — Keras 3.15.1 on TensorFlow 2.21.0, CPU only. The code is New.
>
> **Run this first** — `uv run python -m examples.ch01_the_first_model`, which is `examples/ch01_the_first_model.py`.

Keras is a library to build neural networks and to train them. You describe a model with layers. Keras does the rest: it creates the weights, computes the gradients and updates the weights. This book teaches Keras 3, the current major version.

The aim of this chapter is small. You will build the smallest model that can learn something, and you will read every number that Keras prints about it. Later chapters reuse each call from this chapter.

What Keras 3 is

Keras 3 is a front end. The same Keras code can run on three back ends: TensorFlow, JAX and PyTorch. The back end does the tensor arithmetic. Keras defines the layers, the models, the losses and the training loop.

This book runs on the TensorFlow back end, on a CPU. Every number in the book comes from that setup. The first block prints the back end and the default number type.

```
import keras

print(keras.backend.backend(), keras.config.floatx(), keras.config.image_data_format())
```

```
tensorflow float32 channels_last
```

The three values are the back end, the default floating-point type and the default layout of image tensors. All three are settings. The back end comes from the environment variable `KERAS_BACKEND`, and you must set it before you import Keras. Chapter 15 returns to the back ends and says what this book did not test on the other two.

The smallest model

The task is to learn a straight line. The data are 64 points on y = 2x + 1, with x between -1 and 1. A model with one weight and one bias can represent this line exactly. Training must find the values 2 and 1.

```
import keras
import numpy as np

keras.utils.set_random_seed(0)
x = np.linspace(-1, 1, 64, dtype="float32").reshape(-1, 1)
y = 2 * x + 1

model = keras.Sequential([keras.Input((1,)), keras.layers.Dense(1)])
model.compile(optimizer=keras.optimizers.Adam(0.1), loss="mse")
history = model.fit(x, y, epochs=100, batch_size=16, verbose=0)

kernel, bias = model.layers[0].get_weights()
print("first epoch loss:", round(history.history["loss"][0], 4))
print("last epoch loss: ", round(history.history["loss"][-1], 6))
print("weight:", kernel.ravel(), "bias:", bias)
print("prediction at x=3:", model.predict(np.array([[3.0]], dtype="float32"),
verbose=0).ravel())
```

```
first epoch loss: 1.8592
last epoch loss:  0.0
weight: [2.] bias: [1.]
prediction at x=3: [7.]
```

The model found the line. The weight is 2, the bias is 1, and the prediction at x = 3 is 7. This is a fit to noise-free data, so the loss can reach zero.

The five calls

Every Keras program in this book uses the same five calls. The block above contains all of them.

1. `keras.Input((1,))` declares the shape of one sample. The batch size is not part of it.
2. `keras.layers.Dense(1)` is a layer. It computes `x @ kernel + bias` with one output unit.
3. `keras.Sequential([...])` stacks the layers into a model.
4. `model.compile(optimizer=..., loss=...)` chooses how to learn. It does not change the weights.
5. `model.fit(x, y, ...)` runs the training loop and returns a `History` object.

Two more calls read the result. `model.evaluate(x, y)` returns the loss on data. `model.predict(x)` returns the output of the model. Both calls work on batches, so the input must have a batch axis as its first axis.

Read the summary

`model.summary()` prints a table of the layers and a count of the parameters. The next block builds the same model and prints the table. The argument `line_length` keeps the table narrow.

```
import keras

keras.utils.set_random_seed(0)

model = keras.Sequential([keras.Input((1,)), keras.layers.Dense(1)])
model.compile(optimizer=keras.optimizers.Adam(0.1), loss="mse")
model.summary(line_length=60)
```

```
Model: "sequential"
┏━━━
```

of the random start.

One epoch is one pass through the data. With 64 samples and a batch size of 16, one epoch has 4 steps. After 5 epochs the optimizer makes 20 updates. The count of updates, not the count of epochs, is what the learning rate acts on.

The optimizer and the learning rate decide the result

The model and the data did not change between the runs below. Only the optimizer did. Each run trains for 100 epochs from the same seed, and the block prints the final loss.

```
import keras
import numpy as np

x = np.linspace(-1, 1, 64, dtype="float32").reshape(-1, 1)
y = 2 * x + 1

def final_loss(optimizer):
    keras.utils.set_random_seed(0)
    model = keras.Sequential([keras.Input((1,)), keras.layers.Dense(1)])
    model.compile(optimizer=optimizer, loss="mse")
    history = model.fit(x, y, epochs=100, batch_size=16, verbose=0)
    return history.history["loss"][-1]

runs = {
    "SGD  lr=0.01": keras.optimizers.SGD(0.01),
    "SGD  lr=0.1": keras.optimizers.SGD(0.1),
    "SGD  lr=1.5": keras.optimizers.SGD(1.5),
    "Adam lr=0.1": keras.optimizers.Adam(0.1),
    "Adam lr=0.001": keras.optimizers.Adam(0.001),
}
for name, optimizer in runs.items():
    print(f"{name:14s} final loss {final_loss(optimizer):.3g}")
```

```
SGD  lr=0.01   final loss 0.00538
SGD  lr=0.1    final loss 1.31e-13
SGD  lr=1.5    final loss nan
Adam lr=0.1    final loss 0
Adam lr=0.001  final loss 1.27
```

Read the table from the top. SGD with a learning rate of 0.01 is slow but it works. A rate of 0.1 is much better. A rate of 1.5 is too large, and each step overshoots, so the loss grows until it becomes `nan`.

Adam with its default rate of 0.001 is too slow for this problem. After 100 epochs the loss is still 1.27. Adam with a rate of 0.1 reaches zero.

Keras did not raise an error for the `nan` run. The run finished and `fit` returned a history. A loss of `nan` is a result, not an exception. Chapter 7 shows the callback that stops a run when this happens.

Three mistakes and what Keras says

Shape errors are the most common first failure. The block below makes three of them on purpose. It prints the first line of each message.

```
import keras
import numpy as np

keras.utils.set_random_seed(0)

x = np.linspace(-1, 1, 64, dtype="float32").reshape(-1, 1)
y = 2 * x + 1
model = keras.Sequential([keras.Input((1,)), keras.layers.Dense(1)])
model.compile(optimizer="adam", loss="mse")

def attempt(label, xs, ys):
    try:
        model.fit(xs, ys, epochs=1, verbose=0)
        print(f"{label}: accepted")
    except Exception as error:
        print(f"{label}: {type(error).__name__}: {str(error).splitlines()[0][:60]}")

attempt("rank-1 x      ", x.ravel(), y.ravel())
attempt("3 features    ", np.zeros((64, 3), dtype="float32"), y)
attempt("10 targets    ", x, y[:10])
```

```
rank-1 x      : accepted
3 features    : ValueError: Exception encountered when calling Sequential.call().
10 targets    : ValueError: Data cardinality is ambiguous. Make sure all arrays contain
```

The first call is the surprise. A rank-1 array of 64 numbers is accepted for a model that expects shape `(1,)`. Keras adds the missing axis. This is convenient here and wrong in other places, so give your arrays explicit shapes.

The second and third calls fail with a `ValueError`. The second message names the layer. The third message names the cause.

How it works

`fit` is a loop. In each step it does four things: a forward pass, a loss, a gradient and an update. You can write one step with a pencil, and Keras does the same arithmetic.

For the line model, the output is `w * x + b`. The loss is the mean of `(w * x + b - y)²`. The gradient of the loss with respect to `w` is `2 * mean(error * x)`. The gradient with respect to `b` is `2 * mean(error)`. SGD subtracts the learning rate times the gradient from each weight.

The next block does one step by hand and one step with `fit`. It uses one batch that holds all 64 samples, so one epoch is one step. It compares the weights.

```
import keras
import numpy as np

x = np.linspace(-1, 1, 64, dtype="float32").reshape(-1, 1)
y = 2 * x + 1

keras.utils.set_random_seed(0)
model = keras.Sequential([keras.Input((1,)), keras.layers.Dense(1)])
model.compile(optimizer=keras.optimizers.SGD(0.1), loss="mse")
w0, b0 = (a.copy() for a in model.layers[0].get_weights())

error = w0[0, 0] * x + b0[0] - y
gradient_w, gradient_b = 2 * np.mean(error * x), 2 * np.mean(error)
by_hand = [w0[0, 0] - 0.1 * gradient_w, b0[0] - 0.1 * gradient_b]

model.fit(x, y, epochs=1, batch_size=64, shuffle=False, verbose=0)
w1, b1 = model.layers[0].get_weights()
print("before: w = %.4f  b = %.4f" % (w0[0, 0], b0[0]))
print("gradient: dw = %.4f  db = %.4f" % (gradient_w, gradient_b))
print("by hand: w = %.4f  b = %.4f" % tuple(by_hand))
print("fit:     w = %.4f  b = %.4f" % (w1[0, 0], b1[0]))
print("equal:", bool(np.allclose(by_hand, [w1[0, 0], b1[0]], atol=1e-6)))

default = keras.Sequential([keras.Input((1,)), keras.layers.Dense(1)])
default.compile(optimizer="sgd", loss="mse")
default.fit(x, y, epochs=1, verbose=0)
print("steps in one epoch with the default batch_size:",
int(default.optimizer.iterations.numpy()))
```

```
before: w = 0.0543  b = 0.0000
gradient: dw = -1.3383  db = -2.0000
by hand: w = 0.1882  b = 0.2000
fit:     w = 0.1882  b = 0.2000
equal: True
steps in one epoch with the default batch_size: 2
```

The two results agree. The invariant is simple: one step of `fit` is one gradient step on one batch. The last line shows the default batch size. With `batch_size=None`, Keras uses 32 samples, so 64 samples give 2 steps.

The mechanism explains two results of this chapter. A learning rate of 1.5 gives a step that is larger than the distance to the minimum. Each step overshoots by more than the step before, and the loss grows to `nan`. A batch size of 16 gives 4 steps in one epoch. Each of the 4 gradients is a noisier estimate than the gradient of all 64 samples.

Reference

The next block prints the signature of each function that this chapter uses. The signatures come from the installed Keras. The text under the block is what the library printed.

```
import keras
from keras import layers
from examples.signatures import show

show(("Sequential", keras.Sequential.__init__), ("Input", keras.Input), layers.Dense,
     ("Model.compile", keras.Model.compile), ("Model.fit", keras.Model.fit),
     ("Model.predict", keras.Model.predict), ("Model.evaluate", keras.Model.evaluate))
```

```
Sequential(layers=None, trainable=True, name=None)
Input(shape=None, batch_size=None, dtype=None, sparse=None, ragged=None,
    batch_shape=None, name=None, tensor=None, optional=False)
Dense(units, activation=None, use_bias=True,
    kernel_initializer='glorot_uniform', bias_initializer='zeros',
    kernel_regularizer=None, bias_regularizer=None, activity_regularizer=None,
    kernel_constraint=None, bias_constraint=None, lora_rank=None,
    lora_alpha=None, quantization_config=None, **kwargs)
Model.compile(optimizer='rmsprop', loss=None, loss_weights=None, metrics=None,
    weighted_metrics=None, run_eagerly=False, steps_per_execution=1,
    jit_compile='auto', auto_scale_loss=True)
Model.fit(x=None, y=None, batch_size=None, epochs=1, verbose='auto',
    callbacks=None, validation_split=0.0, validation_data=None, shuffle=True,
    class_weight=None, sample_weight=None, initial_epoch=0,
    steps_per_epoch=None, validation_steps=None, validation_batch_size=None,
    validation_freq=1)
Model.predict(x, batch_size=None, verbose='auto', steps=None, callbacks=None)
Model.evaluate(x=None, y=None, batch_size=None, verbose='auto',
    sample_weight=None, steps=None, callbacks=None, return_dict=False,
    **kwargs)
```

The parameters that matter most in this chapter:

parameter	default	what it does	what goes wrong
`Input(shape)`	`None`	shape of one sample, without the batch axis	a rank-1 array is accepted and Keras adds the axis (Chapter 1)
`Dense(units)`	required	number of outputs; the kernel has shape `(features, units)`	—
`Dense(activation)`	`None`	function applied to the output	`None` gives raw scores, which a loss may need to know about (Chapter 6)
`compile(optimizer)`	`'rmsprop'`	rule that turns a gradient into a step	`SGD(1.5)` gave `nan`; `Adam(0.001)` was too slow for the line (Chapter 1)
`compile(loss)`	`None`	number that training makes smaller	—
`fit(epochs)`	`1`	passes through the data	the number of steps is `epochs × ceil(n / batch_size)`
`fit(batch_size)`	`None`	samples in one step; `None` gives 32	4 steps in one epoch at 16, 2 at the default of 32
`fit(verbose)`	`'auto'`	progress output	—
`fit(shuffle)`	`True`	shuffle the arrays at each epoch	ignored for a `tf.data.Dataset` (Chapter 5)
`predict(batch_size)`	`None`	samples in one forward pass	—
`evaluate(return_dict)`	`False`	return a dictionary and not a list	a list has the loss first, then the metrics in the order of `compile`

What you have now

You built a model, trained it and read its summary, its history and its parameter count. You also saw that the learning rate can decide the result and that `nan` does not raise an error.

Chapter 2 goes below the layers. It shows the tensors that the back end creates and the `keras.ops` functions that work on every back end. Exercises for this chapter are in exercises/01.md, and the worked solutions are in exercises/solutions/01.md.

Chapter 2 — Tensors and ops

> **Build** — a layer-normalization function written only with `keras.ops`, checked against NumPy, and a set of four measured rules for number types and random seeds.
>
> **Theory kept** — a tensor is an array that a back end owns. A function that uses only `keras.ops` runs on every back end.
>
> **Source** — Keras 3.15.1 on TensorFlow 2.21.0. The code is New.
>
> **Run this first** — `uv run python -m examples.ch02_tensors_and_ops`, which is `examples/ch02_tensors_and_ops.py`.

Chapter 1 used layers and did not look at what flows between them. This chapter does. A layer takes a tensor and returns a tensor. You must know what a tensor is, which functions can work on it, and which number types and random numbers it uses.

A tensor belongs to a back end

NumPy arrays belong to NumPy. The tensors in a Keras model belong to the back end. With the TensorFlow back end they are `tf.Tensor` objects. With the JAX back end they are JAX arrays.

The module `keras.ops` is the common interface. Its functions accept NumPy arrays and Python lists. They return tensors of the active back end.

```
import keras
import numpy as np
from keras import ops

ones = ops.ones((2, 3))
print(type(ones).__name__, ones.dtype, ops.shape(ones))

array = np.arange(6, dtype="float32").reshape(2, 3)
tensor = ops.convert_to_tensor(array)
print(ops.is_tensor(array), ops.is_tensor(tensor))
print(type(ops.convert_to_numpy(tensor)).__name__)
```

```
EagerTensor <dtype: 'float32'> (2, 3)
False True
ndarray
```

The first line shows the class of the tensor on this back end. The second line shows that a NumPy array is not a tensor. The third line shows the way back: `ops.convert_to_numpy` returns a NumPy array. Use it whenever you print, plot or compare a result.

Write once with `keras.ops`

Most NumPy functions have a twin in `keras.ops`, with the same name and the same arguments. The next block writes layer normalization with `ops` only. Layer normalization subtracts the mean of each row and divides by the standard deviation of the row. A small constant `eps` keeps the division safe.

The block compares the result with the same formula in NumPy.

```
import keras
import numpy as np
from keras import ops

def layer_norm(x, eps=1e-6):
    mean = ops.mean(x, axis=-1, keepdims=True)
    variance = ops.var(x, axis=-1, keepdims=True)
    return (x - mean) / ops.sqrt(variance + eps)

x = np.random.default_rng(0).normal(size=(4, 5)).astype("float32")
reference = (x - x.mean(-1, keepdims=True)) / np.sqrt(x.var(-1, keepdims=True) + 1e-6)
result = ops.convert_to_numpy(layer_norm(ops.convert_to_tensor(x)))

print("max difference below 1e-6:", bool(np.abs(result - reference).max() < 1e-6))
print("row means near zero:", bool(np.abs(result.mean(-1)).max() < 1e-6))
print("row variances near one:", bool(np.abs(result.var(-1) - 1).max() < 1e-4))
```

```
max difference below 1e-6: True
row means near zero: True
row variances near one: True
```

The function `layer_norm` contains no back end code. This book ran it on TensorFlow only. The three functions `ops.mean`, `ops.var` and `ops.sqrt` are part of `keras.ops`, so the same code is valid on the other back ends. Chapter 15 says what that claim is worth. A function that calls `tf.reduce_mean` is valid on one back end only. This is the reason to prefer `keras.ops` inside a custom layer (Chapter 3).

Number types decide what a number is

A tensor has a dtype. The default is `float32`, and `keras.config.floatx()` returns it. Two other types appear often: `float16`, which has less precision, and the 8-bit integers, which wrap.

```
import keras
import numpy as np
from keras import ops

half = ops.cast(0.1, "float16")
stored = float(ops.convert_to_numpy(half))
print("0.1 as float16:", stored, f"(error {0.1 - stored:.2e})")

total = ops.add(ops.cast([200], "uint8"), ops.cast([100], "uint8"))
print("uint8 200 + 100:", ops.convert_to_numpy(total))

big = ops.cast([70000.0], "float16")
print("float16 70000:", ops.convert_to_numpy(big), "largest finite:", np.finfo("float16").max)
```

```
0.1 as float16: 0.0999755859375 (error 2.44e-05)
uint8 200 + 100: [44]
float16 70000: [inf] largest finite: 6.55e+04
```

The value 0.1 becomes 0.0999755859375 in 16 bits. The error is 2.44e-05. A sum of `uint8` values wraps: 200 + 100 is 300, and 300 modulo 256 is 44. No error is raised. The `float16` type has a largest finite value of 65504, so 70000 becomes infinity.

These three results are the reasons for care when you train with 16-bit numbers or quantize a model to 8 bits. Chapter 15 measures what mixed precision costs and what it saves.

Random numbers need a seed policy

Keras has its own random functions in `keras.random`. They use the seed that you give them, or a global seed that you set with `keras.utils.set_random_seed`. That call sets the seed for Python, NumPy and the back end at the same time. The block below checks this claim in its first lines.

The two seed styles behave in different ways. The block below shows the three cases that surprise people. It also confirms that `Dropout` owns a seed generator.

```
import keras
import numpy as np
from keras import ops

import random

keras.utils.set_random_seed(5)
python_a, numpy_a = random.random(), np.random.rand()
keras.utils.set_random_seed(5)
python_b, numpy_b = random.random(), np.random.rand()
print("python and numpy restart together:", python_a == python_b and numpy_a == numpy_b)
print("Dropout owns a seed generator:", hasattr(keras.layers.Dropout(0.5), "seed_generator"))

def same(a, b):
    return bool(np.array_equal(ops.convert_to_numpy(a), ops.convert_to_numpy(b)))

keras.utils.set_random_seed(1)
first = keras.random.normal((3,))
keras.utils.set_random_seed(1)
second = keras.random.normal((3,))
print("global seed set twice, same values:", same(first, second))

generator = keras.random.SeedGenerator(7)
draw_a = keras.random.normal((3,), seed=generator)
draw_b = keras.random.normal((3,), seed=generator)
print("SeedGenerator, two draws equal:", same(draw_a, draw_b))

draw_c = keras.random.normal((3,), seed=7)
draw_d = keras.random.normal((3,), seed=7)
print("integer seed, two draws equal:", same(draw_c, draw_d))
```

```
python and numpy restart together: True
Dropout owns a seed generator: True
global seed set twice, same values: True
SeedGenerator, two draws equal: False
integer seed, two draws equal: True
```

Setting the global seed again restarts the stream, so the two values are equal. A `SeedGenerator` is a stateful object. It advances on each use, so two draws differ and the whole sequence is still repeatable. An integer seed is stateless. The same integer gives the same numbers on every call.

The third result is the trap. A custom layer that calls `keras.random.normal(shape, seed=7)` inside `call` adds the same noise to every batch. Use a `SeedGenerator`, which is what the built-in `Dropout` layer does.

Four rules

The chapter reduces to four rules. Each rule has a measurement above.

1. Convert with `ops.convert_to_numpy` before you print or plot a tensor.
2. Write custom code with `keras.ops` so that it runs on every back end.
3. Name the dtype when the range or the precision matters. Do not rely on the default.
4. Use `set_random_seed` once at the start of a script. Use a `SeedGenerator` inside a layer.

How it works

A `keras.ops` function does not compute anything itself. It looks at the active back end, calls the matching function of that back end and returns a back end tensor. This is why the same code runs on three back ends, and why the type of the result is not a NumPy array.

The random functions need one more idea. A back end random function is a pure function of a seed and a shape. The same seed gives the same numbers.

For a program to draw different numbers on each call, something must change the seed between calls. In Keras, a `SeedGenerator` holds that state. Its state is a variable of two numbers: the seed and a counter.

The next block shows the state before and after two draws, and the difference between a generator and an integer seed.

```
import keras
from keras import ops

generator = keras.random.SeedGenerator(7)
print("state at start      :", ops.convert_to_numpy(generator.state).tolist())
first = keras.random.normal((2,), seed=generator)
print("state after one draw:", ops.convert_to_numpy(generator.state).tolist())
second = keras.random.normal((2,), seed=generator)
print("state after two     :", ops.convert_to_numpy(generator.state).tolist())

again = keras.random.SeedGenerator(7)
repeat = keras.random.normal((2,), seed=again)
print("new generator with seed 7 repeats the first draw:",
      bool((ops.convert_to_numpy(repeat) == ops.convert_to_numpy(first)).all()))
print("a draw does not change an integer seed:", keras.random.normal((1,), seed=7).shape ==
(1,))
```

```
state at start      : [7, 0]
state after one draw: [7, 1]
state after two     : [7, 2]
new generator with seed 7 repeats the first draw: True
a draw does not change an integer seed: True
```

The counter goes up by one on each draw, and the seed stays at 7. The new generator starts at counter 0, so it repeats the first draw.

The invariant is: *the numbers are a function of the seed and the counter.* A generator moves the counter, and an integer seed never moves it. This is the cause of the trap in this chapter. The call `seed=7` inside a layer gives the same noise on every call.

Reference

```
import keras
from keras import ops
from examples.signatures import show

show(("ops.convert_to_tensor", ops.convert_to_tensor), ("ops.cast", ops.cast),
     ("ops.convert_to_numpy", ops.convert_to_numpy), ("random.normal", keras.random.normal),
     keras.random.SeedGenerator, ("utils.set_random_seed", keras.utils.set_random_seed))
```

```
ops.convert_to_tensor(x, dtype=None, sparse=None, ragged=None)
ops.cast(x, dtype)
ops.convert_to_numpy(x)
random.normal(shape, mean=0.0, stddev=1.0, dtype=None, seed=None)
SeedGenerator(seed=None, name=None, **kwargs)
utils.set_random_seed(seed)
```

parameter	default	what it does	what goes wrong
`convert_to_tensor(x)`	required	NumPy array or list to a back end tensor	—
`convert_to_tensor(dtype)`	`None`	type of the result	—
`cast(x, dtype)`	required	change the type	`uint8` 200 + 100 gives 44, `float16` 70000 gives `inf` (Chapter 2)
`convert_to_numpy(x)`	required	tensor to NumPy array	print and plot need it
`random.normal(shape)`	required	tensor of random numbers	—
`random.normal(seed)`	`None`	`None` : the global generator that `set_random_seed` sets; integer: stateless; `SeedGenerator` : stateful	an integer seed gives equal draws on every call
`SeedGenerator(seed)`	`None`	start state of a stream of seeds	two draws differ, a new generator repeats the stream
`set_random_seed(seed)`	required	seed Python, NumPy and the back end together	call it once at the start of a script

What you have now

You have a layer-normalization function that runs on any Keras back end and that agrees with NumPy. You also have the four rules above.

Chapter 3 puts such a function inside a layer and shows where the layer keeps its weights. Exercises for this chapter are in exercises/02.md, and the worked solutions are in exercises/solutions/02.md.

Chapter 3 — Layers and weights

> **Build** — a `MyDense` layer that gives the same output as `keras.layers.Dense`, a layer that counts its own calls, and a table of what each kind of weight does in `fit` and `predict`.
>
> **Theory kept** — a layer is a container for weights and a function `call`. Weights come into existence at the first call, not at construction.
>
> **Source** — Keras 3.15.1 on TensorFlow 2.21.0. The code is New.
>
> **Run this first** — `uv run python -m examples.ch03_layers_and_weights`, which is `examples/ch03_layers_and_weights.py`.

Every model in Keras is a tree of layers. A `Dense` layer, a `Conv2D` layer and a whole `Sequential` model are all objects of the class `keras.layers.Layer` or of a subclass. If you know the layer contract, you can read any Keras model and write any custom part.

The contract has four parts: the constructor, `build`, `call` and `get_config`. This chapter writes a layer with all four and then measures what the framework does with it.

A layer in four methods

The first block writes `MyDense`. It is the same computation as `keras.layers.Dense` with no activation: `x @ kernel + bias`.

```
import keras
from keras import ops

class MyDense(keras.layers.Layer):
    def __init__(self, units, **kwargs):
        super().__init__(**kwargs)
        self.units = units

    def build(self, input_shape):
        self.kernel = self.add_weight(
            shape=(input_shape[-1], self.units), initializer="glorot_uniform", name="kernel")
        self.bias = self.add_weight(shape=(self.units,), initializer="zeros", name="bias")

    def call(self, x):
        return ops.matmul(x, self.kernel) + self.bias

    def get_config(self):
        return {**super().get_config(), "units": self.units}
```

The constructor stores the arguments that do not depend on the input. It must call `super().__init__(**kwargs)`, so that `name`, `dtype` and `trainable` work. The method `build` receives the shape of the input and creates the weights with `add_weight`. The method `call` computes the output. The method `get_config` returns the arguments that rebuild the layer.

The `build` method is the part that surprises people. The next block shows that a new layer has no weights.

```
import keras
import numpy as np
from keras import ops

class MyDense(keras.layers.Layer):
    def __init__(self, units, **kwargs):
        super().__init__(**kwargs)
        self.units = units

    def build(self, input_shape):
        self.kernel = self.add_weight(
            shape=(input_shape[-1], self.units), initializer="glorot_uniform", name="kernel")
        self.bias = self.add_weight(shape=(self.units,), initializer="zeros", name="bias")

    def call(self, x):
        return ops.matmul(x, self.kernel) + self.bias

layer = MyDense(3)
print("before the first call: built =", layer.built, "weights =", len(layer.weights))

x = np.ones((2, 4), dtype="float32")
y = layer(x)
print("after the first call:  built =", layer.built, "weights =", len(layer.weights))
print("kernel shape", tuple(layer.kernel.shape), "bias shape", tuple(layer.bias.shape))
print("output shape", tuple(y.shape))
```

```
before the first call: built = False weights = 0
after the first call:  built = True weights = 2
kernel shape (4, 3) bias shape (3,)
output shape (2, 3)
```

The kernel has the shape `(4, 3)` because the first input had 4 features. The layer cannot know this number in advance. This is the reason for lazy weights: the same layer class works on any input width, and Keras fixes the width at the first call.

The same numbers as `Dense`

A custom layer is correct when it matches a reference. The next block copies the weights of `MyDense` into a built-in `Dense` layer and compares the outputs.

```
import keras
import numpy as np
from keras import layers, ops

keras.utils.set_random_seed(0)

class MyDense(keras.layers.Layer):
    def __init__(self, units, **kwargs):
        super().__init__(**kwargs)
        self.units = units

    def build(self, input_shape):
        self.kernel = self.add_weight(
            shape=(input_shape[-1], self.units), initializer="glorot_uniform", name="kernel")
        self.bias = self.add_weight(shape=(self.units,), initializer="random_normal",
name="bias")

    def call(self, x):
        return ops.matmul(x, self.kernel) + self.bias

x = np.random.default_rng(0).normal(size=(5, 4)).astype("float32")
mine = MyDense(3)
reference = layers.Dense(3)
mine(x)
reference(x)
reference.set_weights(mine.get_weights())

difference = np.abs(ops.convert_to_numpy(mine(x)) - ops.convert_to_numpy(reference(x))).max()
print("max difference:", float(difference))
print("weight order in get_weights:", [w.shape for w in mine.get_weights()])
```

```
max difference: 0.0
weight order in get_weights: [(4, 3), (3,)]
```

The difference is zero. The order of `get_weights` is the order in which `add_weight` was called: kernel first, bias second. This order matters because `set_weights` takes a plain list. Two layers that create the same weights in a different order are not interchangeable. Chapter 14 returns to this point.

Trainable and non-trainable weights

A weight is either trainable or not. The optimizer updates trainable weights. It does not update the others. The two groups are lists on every layer: `trainable_weights` and `non_trainable_weights`.

Batch normalization is the standard example. It has two trainable weights, `gamma` and `beta`. It has two non-trainable weights, the moving mean and the moving variance. The optimizer does not touch the last two. The layer updates them itself during training.

```
import keras
import numpy as np
from keras import layers

keras.utils.set_random_seed(0)

x = np.ones((2, 4), dtype="float32")
norm = layers.BatchNormalization()
norm(x)
print("trainable:    ", [w.path for w in norm.trainable_weights])
print("non-trainable:", [w.path for w in norm.non_trainable_weights])

dense = layers.Dense(3)
dense(x)
print("before freeze:", len(dense.trainable_weights), len(dense.non_trainable_weights))
dense.trainable = False
print("after freeze: ", len(dense.trainable_weights), len(dense.non_trainable_weights))
```

```
trainable:     ['batch_normalization/gamma', 'batch_normalization/beta']
non-trainable: ['batch_normalization/moving_mean', 'batch_normalization/moving_variance']
before freeze: 2 0
after freeze:  0 2
```

Setting `trainable = False` moves every weight of the layer to the non-trainable list. This is how you freeze a layer. Chapter 13 uses it.

The list changes at once. The training loop does not change at once. Keras builds the training function at the first `fit` and keeps it. A change of `trainable` after that point has no effect until you call `compile` again. The next block measures four cases. It reports whether the weights of layer `a` changed during one more epoch.

```
import keras
import numpy as np
from keras import layers

x = np.random.default_rng(0).normal(size=(64, 4)).astype("float32")
y = x.sum(1, keepdims=True)

def run(freeze_first, new_state, recompile):
    keras.utils.set_random_seed(0)
    model = keras.Sequential([keras.Input((4,)), layers.Dense(8, name="a"), layers.Dense(1,
name="b")])
    model.get_layer("a").trainable = not freeze_first
    model.compile(optimizer=keras.optimizers.SGD(0.01), loss="mse")
    model.fit(x, y, epochs=1, verbose=0)
    before = model.get_layer("a").get_weights()[0].copy()
    model.get_layer("a").trainable = new_state
    if recompile:
        model.compile(optimizer=keras.optimizers.SGD(0.01), loss="mse")
    model.fit(x, y, epochs=1, verbose=0)
    return not np.array_equal(before, model.get_layer("a").get_weights()[0])

for label, freeze_first, new_state, recompile in [
    ("freeze,   no recompile", False, False, False),
    ("freeze,   recompile   ", False, False, True),
    ("unfreeze, no recompile", True, True, False),
    ("unfreeze, recompile   ", True, True, True),
]:
    print(label, "-> layer a changed:", run(freeze_first, new_state, recompile))
```

```
freeze,   no recompile -> layer a changed: True
freeze,   recompile    -> layer a changed: False
unfreeze, no recompile -> layer a changed: False
unfreeze, recompile    -> layer a changed: True
```

Two of the four cases are wrong in the way that matters. Without `compile`, a frozen layer keeps training and an unfrozen layer stays frozen. Keras raises no error and prints no warning. The rule is simple: call `compile` after every change of `trainable`.

A weight that the optimizer never touches

A non-trainable weight is also a place to keep state. The next layer counts how many times Keras calls it. It does not change its input.

The layer keeps two counters, one for calls in training mode and one for calls in inference mode. The block puts the layer in a model and runs `predict`, `fit` and `evaluate`. After each call it prints both counters.

```
import keras
import numpy as np

keras.utils.set_random_seed(0)

class Counter(keras.layers.Layer):
    def build(self, input_shape):
        self.train_calls = self.add_weight(shape=(), initializer="zeros", trainable=False,
name="train")
        self.infer_calls = self.add_weight(shape=(), initializer="zeros", trainable=False,
name="infer")

    def call(self, x, training=None):
        (self.train_calls if training else self.infer_calls).assign_add(1.0)
        return x

    def read(self):
        return int(self.train_calls.numpy()), int(self.infer_calls.numpy())

counter = Counter()
model = keras.Sequential([keras.Input((4,)), counter, keras.layers.Dense(1)])
model.compile(optimizer="sgd", loss="mse")
x = np.ones((100, 4), dtype="float32")
y = np.ones((100, 1), dtype="float32")

print("(train, infer) at start:", counter.read())
model.predict(x, batch_size=32, verbose=0)
print("after predict, 100 samples in batches of 32:", counter.read())
model.fit(x, y, batch_size=25, epochs=2, verbose=0)
print("after fit, 2 epochs of 4 steps:", counter.read())
model.evaluate(x, y, batch_size=50, verbose=0)
print("after evaluate, 2 batches:", counter.read())
```

```
(train, infer) at start: (0, 0)
after predict, 100 samples in batches of 32: (0, 4)
after fit, 2 epochs of 4 steps: (8, 4)
after evaluate, 2 batches: (8, 6)
```

The counters measure the batch loop. `predict` makes 4 inference calls, because 100 samples in batches of 32 give 4 batches. `fit` makes 8 training calls, because 2 epochs of 4 steps give 8 steps. `evaluate` makes 2 inference calls. The layer sees batches, not samples. Any statistic that the layer keeps is a statistic over batches.

A layer that behaves in two ways

Some layers act differently in training and in inference. `Dropout` is the best known. A custom layer gets this information through the argument `training` of `call`.

```
import keras
import numpy as np

keras.utils.set_random_seed(0)

class AddOne(keras.layers.Layer):
    def call(self, x, training=None):
        return x + 1.0 if training else x

zeros = np.zeros((1, 2), dtype="float32")
layer = AddOne()
print("direct call:         ", layer(zeros).numpy())
print("direct, training=True:", layer(zeros, training=True).numpy())

model = keras.Sequential([keras.Input((2,)), AddOne()])
print("model.predict:        ", model.predict(zeros, verbose=0))
print("model(x, training):   ", model(zeros, training=True).numpy())
```

```
direct call:          [[0. 0.]]
direct, training=True: [[1. 1.]]
model.predict:         [[0. 0.]]
model(x, training):    [[1. 1.]]
```

The default is inference. The previous section measured that `fit` runs in training mode and that `predict` and `evaluate` run in inference mode. When you call a model yourself, you must pass `training=True` if you want the training behavior.

Losses that a layer adds

A layer can add a term to the total loss. A weight regularizer does this. The term appears in `layer.losses` after the layer has weights.

```
import keras
import numpy as np

keras.utils.set_random_seed(0)

layer = keras.layers.Dense(
    2, kernel_regularizer=keras.regularizers.L2(0.01), kernel_initializer="ones")
print("before the first call:", layer.losses)

layer(np.ones((1, 5), dtype="float32"))
print("after the first call: ", [round(float(v), 4) for v in layer.losses])
print("expected: 0.01 * (5 * 2 weights of value 1) =", 0.01 * 5 * 2)
```

```
before the first call: []
after the first call:  [0.1]
expected: 0.01 * (5 * 2 weights of value 1) = 0.1
```

The list is empty until the weights exist. After that the value is the penalty that `fit` adds to the loss at every step. It is 0.01 times the sum of the squared weights. Chapter 8 measures what this does to a model.

How it works

When you call a layer, you call `Layer.__call__`, not your `call`. `__call__` does the bookkeeping first: it converts the inputs, it checks whether the layer is built, and it calls `build` once if it is not. Only then does it call `call`. This is why you never call `build` yourself, and why `call` has no code to create weights.

The next block records the order of events. It also shows what a built layer does with an input of a new width.

```
import keras
import numpy as np
from keras import layers

keras.utils.set_random_seed(0)

events = []

class Traced(keras.layers.Layer):
    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        events.append("__init__")

    def build(self, input_shape):
        events.append(f"build{tuple(input_shape)}")

    def call(self, x):
        events.append("call")
        return x

layer = Traced()
layer(np.ones((2, 3), dtype="float32"))
layer(np.ones((2, 3), dtype="float32"))
layer(np.ones((5, 3), dtype="float32"))
print(events)

dense = layers.Dense(2)
dense(np.ones((1, 3), dtype="float32"))
try:
    dense(np.ones((1, 4), dtype="float32"))
except ValueError as error:
    print("a built Dense with a new width:", str(error).splitlines()[0][:70])
```

```
['__init__', 'build(2, 3)', 'call', 'call', 'call']
a built Dense with a new width: Input 0 with name 'None' of layer 'dense' is incompatible with
the lay
```

`build` runs once, with the shape of the first input. It does not run again for a batch of 5. The invariant is: *the first call fixes the weights.* Every later input must fit them. The `Dense` layer was built for 3 features, so an input of 4 features fails. `Traced` has no weights, so no shape can fail.

The mechanism also explains the counter of this chapter. Keras calls `call` once for each batch, in training and in inference, and the layer sees one batch at a time.

Reference

```
import keras
from keras import layers
from examples.signatures import show

show(("Layer", keras.layers.Layer.__init__), ("Layer.add_weight",
keras.layers.Layer.add_weight),
     ("Layer.build", keras.layers.Layer.build), ("Layer.get_config",
keras.layers.Layer.get_config),
     layers.BatchNormalization)
```

```
Layer(activity_regularizer=None, trainable=True, dtype=None, autocast=True,
    name=None, **kwargs)
Layer.add_weight(*args, shape=None, initializer=None, dtype=None,
    trainable=True, autocast=True, regularizer=None, constraint=None,
    aggregation='none', overwrite_with_gradient=False, name=None)
Layer.build(input_shape)
Layer.get_config()
BatchNormalization(axis=-1, momentum=0.99, epsilon=0.001, center=True,
    scale=True, beta_initializer='zeros', gamma_initializer='ones',
    moving_mean_initializer='zeros', moving_variance_initializer='ones',
    beta_regularizer=None, gamma_regularizer=None, beta_constraint=None,
    gamma_constraint=None, renorm=False, renorm_clipping=None,
    renorm_momentum=0.99, synchronized=False, **kwargs)
```

parameter	default	what it does	what goes wrong
`Layer(name)`	`None`	name of the layer; `None` gives a name from the class	`MyDense` became `my_dense`
`Layer(trainable)`	`True`	the optimizer updates the weights	set to `False` after the first `fit` and Keras ignores it until `compile` (Chapter 3)
`Layer(dtype)`	`None`	type of the weights and the computation	—
`add_weight(shape)`	`None`	shape of the new weight	—
`add_weight(initializer)`	`None`	how the start values are made	`None` leaves the choice to Keras; name the initializer so the choice is visible
`add_weight(trainable)`	`True`	`False` puts the weight in `non_trainable_weights`	a counter layer used this and the optimizer did not touch it
`add_weight(regularizer)`	`None`	adds a penalty term to `layer.losses`	the term appears only after the layer is built
`build(input_shape)`	required	create the weights from the first input shape	runs once
`call(x, training=None)`	—	the computation	`training` is `True` inside `fit` only
`BatchNormalization(momentum)`	`0.99`	speed of the moving mean and variance	the two moving values are non-trainable weights

What you have now

You have a custom layer that matches `Dense` and a counter that exposes the batch loop. You also know how lazy weights, trainable weights and the `training` argument work. The next chapter puts layers together in three different ways and measures that the three give the same model.

Exercises for this chapter are in exercises/03.md, and the worked solutions are in exercises/solutions/03.md. Continue with Chapter 4.

Chapter 4 — Three ways to build a model

> **Build** — the same small network written as a `Sequential` model, as a functional model and as a subclassed model, with a measurement that shows the three are one function. Then a model with two inputs, a shared layer and two outputs.
>
> **Theory kept** — Keras has one model class with three construction styles. The styles differ in what Keras knows about the graph before data arrives.
>
> **Source** — Keras 3.15.1 on TensorFlow 2.21.0. The code is New.
>
> **Run this first** — `uv run python -m examples.ch04_three_ways_to_build`, which is `examples/ch04_three_ways_to_build.py`.

You can build a Keras model in three ways. Each way has a use. This chapter builds one network three times and shows that the three networks are the same function. Then it uses the one style that can express a graph with branches.

The same network, three times

The network has an input of 8 features, a hidden layer of 16 units with a ReLU, and an output of 3 values.

```
import keras
import numpy as np
from keras import layers, ops

keras.utils.set_random_seed(0)

sequential = keras.Sequential([
    keras.Input((8,)),
    layers.Dense(16, activation="relu"),
    layers.Dense(3),
])

inputs = keras.Input((8,))
hidden = layers.Dense(16, activation="relu")(inputs)
functional = keras.Model(inputs, layers.Dense(3)(hidden))

class Subclassed(keras.Model):
    def __init__(self):
        super().__init__()
        self.hidden = layers.Dense(16, activation="relu")
        self.out = layers.Dense(3)

    def call(self, x):
        return self.out(self.hidden(x))

subclassed = Subclassed()
x = np.random.default_rng(0).normal(size=(5, 8)).astype("float32")
subclassed(x)

for model in (sequential, functional, subclassed):
    print(f"{type(model).__name__:11s} params {model.count_params()}  weight arrays
{len(model.weights)}")

functional.set_weights(sequential.get_weights())
subclassed.set_weights(sequential.get_weights())
outputs = [ops.convert_to_numpy(m(x)) for m in (sequential, functional, subclassed)]
print("functional equals sequential:", np.array_equal(outputs[0], outputs[1]))
print("subclassed equals sequential:", np.array_equal(outputs[0], outputs[2]))
```

```
Sequential  params 195  weight arrays 4
Functional  params 195  weight arrays 4
Subclassed  params 195  weight arrays 4
functional equals sequential: True
subclassed equals sequential: True
```

All three have 195 parameters: $8 \times 16 + 16$ for the first layer and $16 \times 3 + 3$ for the second. After the weights are copied, the three models give identical outputs. The styles are three ways to describe one function.

What Keras knows before data arrives

The styles differ in when the weights exist. The next block builds each model and asks for the summary before the model sees data.

```
import keras
from keras import layers

keras.utils.set_random_seed(0)

with_input = keras.Sequential([keras.Input((8,)), layers.Dense(4), layers.Dense(2)])
print("Sequential with Input: weights =", len(with_input.weights))

without_input = keras.Sequential([layers.Dense(4), layers.Dense(2)])
print("Sequential without Input: built =", without_input.built, "weights =",
len(without_input.weights))

class Subclassed(keras.Model):
    def __init__(self):
        super().__init__()
        self.hidden = layers.Dense(4)
        self.out = layers.Dense(2)

    def call(self, x):
        return self.out(self.hidden(x))

subclassed = Subclassed()
print("Subclassed: built =", subclassed.built, "weights =", len(subclassed.weights))
subclassed.summary(line_length=60)
```

```
Sequential with Input: weights = 4
Sequential without Input: built = False weights = 0
Subclassed: built = False weights = 0
Model: "subclassed"
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━┓
┃ Layer (type)              ┃ Output Shape           ┃      Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━┩
│ dense_4 (Dense)           │ ?                      │            0 │
│                           │                        │ (unbuilt)    │
├───────────────────────────┼────────────────────────┼──────────────┤
│ dense_5 (Dense)           │ ?                      │            0 │
│                           │                        │ (unbuilt)    │
└───────────────────────────┴────────────────────────┴──────────────┘
 Total params: 0 (0.00 B)
 Trainable params: 0 (0.00 B)
 Non-trainable params: 0 (0.00 B)
```

A `Sequential` model with an `Input` creates its weights at once. Without the `Input` it waits for data. A subclassed model always waits, and its summary shows `?` for every shape. This is not a defect. A subclassed `call` is ordinary Python code, and Keras cannot know the shapes until it runs the code.

The consequence is practical. For a functional or `Sequential` model with an `Input`, `summary()` is a complete check of the shapes before training. For a subclassed model, call it once on a small batch first.

Branches, shared layers and two outputs

A `Sequential` model is a straight line. Many real models are not. The functional style handles any directed graph of layers. The next model compares two inputs of 4 features with one shared `Dense` layer, joins the results and returns two outputs: a score and a class.

```
import keras
import numpy as np
from keras import layers

keras.utils.set_random_seed(0)

left = keras.Input((4,), name="left")
right = keras.Input((4,), name="right")
shared = layers.Dense(8, activation="relu", name="shared")
joined = layers.Concatenate()([shared(left), shared(right)])
score = layers.Dense(1, name="score")(joined)
label = layers.Dense(3, activation="softmax", name="label")(joined)
model = keras.Model([left, right], [score, label])

print("layers:", [layer.name for layer in model.layers])
print("outputs:", model.output_names)
print("parameters:", model.count_params())

print("a second Dense(8) on 4 inputs would add:", 4 * 8 + 8, "parameters")
```

```
layers: ['left', 'right', 'shared', 'concatenate', 'score', 'label']
outputs: ['score', 'label']
parameters: 108
a second Dense(8) on 4 inputs would add: 40 parameters
```

The shared layer appears once in the layer list and its 40 parameters count once: 108 in total is 40 + 17 + 51. Both inputs go through the same weights. This is the idea of a Siamese network: one function, applied twice.

The model has two outputs. Keras needs a loss for each one. The `compile` call takes a dictionary keyed by the output names.

```
import keras
import numpy as np
from keras import layers

keras.utils.set_random_seed(0)
left = keras.Input((4,), name="left")
right = keras.Input((4,), name="right")
shared = layers.Dense(8, activation="relu", name="shared")
joined = layers.Concatenate()([shared(left), shared(right)])
score = layers.Dense(1, name="score")(joined)
label = layers.Dense(3, activation="softmax", name="label")(joined)
model = keras.Model([left, right], [score, label])

model.compile(
    optimizer="adam",
    loss={"score": "mse", "label": "sparse_categorical_crossentropy"},
    loss_weights={"score": 1.0, "label": 0.5},
)

rng = np.random.default_rng(0)
data = {"left": rng.normal(size=(32, 4)).astype("float32"),
        "right": rng.normal(size=(32, 4)).astype("float32")}
targets = {"score": rng.normal(size=(32, 1)).astype("float32"),
           "label": rng.integers(0, 3, size=(32,))}
history = model.fit(data, targets, epochs=2, batch_size=16, verbose=0)
last = {name: values[-1] for name, values in history.history.items()}
print(sorted(last))
print("loss == score_loss + 0.5 * label_loss:",
      bool(np.isclose(last["loss"], last["score_loss"] + 0.5 * last["label_loss"], rtol=1e-
4)))
```

```
['label_loss', 'loss', 'score_loss']
loss == score_loss + 0.5 * label_loss: True
```

The history has a total `loss` and one loss for each output. The total is `1.0 × score_loss + 0.5 × label_loss`. Pass inputs and targets as dictionaries keyed by name. Then the order of the arguments cannot cause a silent mistake.

Which style to use

Use `Sequential` when the model is a stack. Use the functional style when the graph has branches, shared layers or several inputs. Use subclassing when `call` must contain Python control flow that the graph cannot express, such as a loop of data-dependent length.

Prefer the functional style when you have a choice. It checks the shapes when you build the model. It makes the graph visible in `summary()`. It saves and loads with only built-in parts.

A subclassed model gives up the first two, and in return it gives you full control of `call`. You can save a subclassed model too. Chapter 14 measures what it needs: a registered class, `**kwargs` in `__init__` and a `get_config` for every argument.

You can also mix the styles. A subclassed model can contain a functional model as a layer, and a functional model can contain a subclassed layer. Chapter 12 builds a transformer block as a layer and then stacks the blocks in a functional

model.

How it works

A functional model is a recorded graph. When you call a layer on a `keras.Input`, you do not give it data. You give it a symbolic tensor with a shape and a type. The layer returns another symbolic tensor, and Keras records a node: this layer, applied to these inputs. `keras.Model(inputs, outputs)` collects the nodes between the inputs and the outputs.

This record is the reason a functional model has a complete `summary()` before any data arrive. It is also the reason you can save its configuration and rebuild the model from it. The next block shows the record.

```
import keras
import numpy as np
from keras import layers, ops

keras.utils.set_random_seed(0)

inputs = keras.Input((8,), name="features")
hidden = layers.Dense(16, activation="relu", name="hidden")(inputs)
outputs = layers.Dense(3, name="out")(hidden)
model = keras.Model(inputs, outputs)

print("symbolic tensor:", type(inputs).__name__, inputs.shape)
config = model.get_config()
print("recorded layers:", [(layer["class_name"], layer["name"]) for layer in
config["layers"]])

clone = keras.Model.from_config(config)
clone.set_weights(model.get_weights())
x = np.random.default_rng(0).normal(size=(4, 8)).astype("float32")
same = np.array_equal(ops.convert_to_numpy(model(x)), ops.convert_to_numpy(clone(x)))
print("clone from the config gives the same output:", same)
```

```
symbolic tensor: KerasTensor (None, 8)
recorded layers: [('InputLayer', 'features'), ('Dense', 'hidden'), ('Dense', 'out')]
clone from the config gives the same output: True
```

The configuration lists the three layers in order, and a model built from it is the same function once it has the weights. The invariant is: *a functional model is its graph plus its weights.* A subclassed model has no graph. It has only Python code in `call`, so Keras cannot show its shapes before data arrive. This is the mechanism behind the `?` in the subclassed summary.

Reference

```
import keras
from keras import layers
from examples.signatures import show

show(("Sequential", keras.Sequential.__init__), ("Model", keras.Model.__init__),
     ("Model.summary", keras.Model.summary), ("Input", keras.Input), layers.Concatenate)
```

```
Sequential(layers=None, trainable=True, name=None)
Model(*args, **kwargs)
Model.summary(line_length=None, positions=None, print_fn=None,
    expand_nested=False, show_trainable=False, layer_range=None)
Input(shape=None, batch_size=None, dtype=None, sparse=None, ragged=None,
    batch_shape=None, name=None, tensor=None, optional=False)
Concatenate(axis=-1, **kwargs)
```

parameter	default	what it does	what goes wrong
`Sequential(layers)`	`None`	list of layers in order	without an `Input` the model is built at the first call (Chapter 4)
`Model(inputs, outputs)`	—	the signature shows `*args, **kwargs`: pass `inputs` and `outputs` for a graph, nothing for a subclass	a subclass must call `super().__init__()`
`Model(name)`	—	name of the model, passed as a keyword	—
`Input(name)`	`None`	key for a dictionary of inputs	name inputs when a model has more than one
`Input(shape)`	`None`	shape without the batch axis	—
`Concatenate(axis)`	`-1`	axis along which the inputs join	inputs must agree on all other axes
`summary(line_length)`	`None`	width of the printed table	an unbuilt subclassed model prints `?` and 0 parameters
`summary(expand_nested)`	`False`	show the layers of nested models	—

What you have now

You have three equal models and a measurement of when each style creates its weights. You also have a model with a shared layer and two outputs that trains with a dictionary of losses.

The next part of the book is about training. Chapter 5 starts with the data. Exercises for this chapter are in exercises/04.md, and the worked solutions are in exercises/solutions/04.md.

Chapter 5 — Data pipelines

> **Build** — three input pipelines for one data set, a `Normalization` layer that travels inside the model, and four measured traps: a validation split that sees one class, an optimizer that cannot cope with raw pixels, a constant pixel that explodes, and a pipeline that ignores `shuffle`.
>
> **Theory kept** — the training loop sees batches. How the batches are made decides what the model learns. Scale, order and leakage are properties of the pipeline, not of the model.
>
> **Source** — Keras 3.15.1 on TensorFlow 2.21.0 and the digits data of scikit-learn 1.9.1. The code is New.
>
> **Run this first** — `uv run python -m examples.ch05_data_pipelines`, which is `examples/ch05_data_pipelines.py`.

A model is only as good as the batches that reach it. Most first failures in a Keras project are not model failures. They are pipeline failures: the wrong scale, the wrong order, or a statistic that leaked from the test data.

This chapter uses one data set for the next four chapters. It is the digits data that ships with scikit-learn. It has 1,797 images of 8 × 8 pixels with values from 0 to 16. The data are on your disk, so nothing here needs a network.

The data

The module `examples/digits.py` loads the data and makes one fixed split. The split is stratified: each class has the same share in the training set and in the test set. The first block looks at the raw data.

```
import numpy as np
from sklearn.datasets import load_digits

data = load_digits()
x, y = data.data, data.target
print("shape:", x.shape, "pixel range:", x.min(), "to", x.max())
print("images per class:", np.bincount(y).tolist())
print("first 12 labels:", y[:12].tolist())

from examples.digits import load
x_train, x_test, y_train, y_test = load()
print("train:", x_train.shape, "test:", x_test.shape)
```

```
shape: (1797, 64) pixel range: 0.0 to 16.0
images per class: [178, 182, 177, 183, 181, 182, 181, 179, 174, 180]
first 12 labels: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 0, 1]
train: (1347, 64) test: (450, 64)
```

There are 1,797 images with 64 pixels each. The classes are balanced: each digit has between 174 and 183 images. The labels in the file repeat the pattern 0 to 9, so they are not sorted. This detail matters in the next section.

The validation split takes the last rows

`fit` has an argument `validation_split`. It holds back the last fraction of the arrays as validation data. It does this before it shuffles. If the class order of the rows is sorted, the validation set contains only the last classes.

The next block sorts the same data by label and trains the same model twice. The first run uses the sorted rows. The second run shuffles the rows first.

```
import keras
import numpy as np
from keras import layers
from examples.digits import load

x_train, x_test, y_train, y_test = load()
order = np.argsort(y_train, kind="stable")
shuffled = np.random.default_rng(0).permutation(len(y_train))

def validation_accuracy(x, y):
    keras.utils.set_random_seed(0)
    model = keras.Sequential([keras.Input((64,)), layers.Dense(64, activation="relu"),
layers.Dense(10)])
    model.compile("adam", keras.losses.SparseCategoricalCrossentropy(from_logits=True),
metrics=["accuracy"])
    history = model.fit(x, y, epochs=10, batch_size=32, validation_split=0.2, verbose=0)
    return history.history["accuracy"][-1], history.history["val_accuracy"][-1]

for name, rows in [("sorted by class", order), ("shuffled", shuffled)]:
    train_acc, val_acc = validation_accuracy(x_train[rows], y_train[rows])
    print(f"{name:16s} train accuracy {train_acc:.3f}  validation accuracy {val_acc:.3f}")
```

```
sorted by class  train accuracy 0.974  validation accuracy 0.015
shuffled         train accuracy 0.945  validation accuracy 0.904
```

With sorted rows, the validation set holds the last classes. The model never saw them in training, so it scores close to zero. The training accuracy is high and the validation accuracy is near zero. This looks like extreme overfitting. It is a pipeline fault.

Shuffle the rows before you call `fit` with `validation_split`. Better, make the validation set yourself with a stratified split, as `examples/digits.py` does.

The scale of the input

The next block trains the same model on the same data with three input treatments: raw pixels from 0 to 16, pixels divided by 16, and a `Normalization` layer inside the model. The layer learns the mean and the variance of each pixel with `adapt`. It then subtracts the mean and divides by the standard deviation.

Each treatment runs with two optimizers.

```
import keras
from keras import layers
from examples.digits import load

x_raw, xt_raw, y_train, y_test = load(scale=False)
loss = keras.losses.SparseCategoricalCrossentropy(from_logits=True)

def run(optimizer, treatment):
    keras.utils.set_random_seed(0)
    first = []
    x_train, x_test = x_raw, xt_raw
    if treatment == "divide by 16":
        x_train, x_test = x_raw / 16, xt_raw / 16
    elif treatment == "Normalization":
        norm = layers.Normalization()
        norm.adapt(x_raw)
        first = [norm]
    model = keras.Sequential([keras.Input((64,)), *first,
                              layers.Dense(64, activation="relu"), layers.Dense(10)])
    model.compile(optimizer, loss, metrics=["accuracy"])
    history = model.fit(x_train, y_train, epochs=10, batch_size=32,
                        validation_data=(x_test, y_test), verbose=0)
    return history.history["val_accuracy"][-1]

for optimizer_name, make in [("SGD(0.1)", lambda: keras.optimizers.SGD(0.1)),
                             ("Adam(0.001)", lambda: keras.optimizers.Adam(0.001))]:
    scores = [run(make(), t) for t in ("raw", "divide by 16", "Normalization")]
    print(f"{optimizer_name:12s} raw {scores[0]:.3f}  divide by 16 {scores[1]:.3f}
Normalization {scores[2]:.3f}")
```

```
SGD(0.1)     raw 0.296  divide by 16 0.962  Normalization 0.964
Adam(0.001)  raw 0.962  divide by 16 0.951  Normalization 0.958
```

Read the table by rows. For SGD, raw pixels give an accuracy of 0.296 and both scaled versions give 0.96. For Adam, all three are between 0.951 and 0.962. The test set has 450 images, and one image is 0.0022. The three Adam results differ by 5 images at most, so Adam shows no effect of scale here.

The reason is how the two optimizers use the gradient. SGD steps by the gradient, so a large input gives a large step. Adam divides each step by a running estimate of the size of the gradient, so it removes most of the scale. Adam hides the scale problem. It does not remove it. A model with a different layer, a different loss or a different optimizer can show it again.

A constant pixel can explode

The `Normalization` layer divides by a standard deviation. A pixel that is constant in the training data has a standard deviation of zero. What does the layer do with it?

The next block adapts the layer on the training images and looks for the largest output on the test images.

```
import keras
import numpy as np
from keras import layers, ops
from examples.digits import load

x_train, x_test, y_train, y_test = load(scale=False)
constant = np.where(x_train.std(axis=0) == 0)[0]
print("pixels that are constant in training:", constant.tolist())
print("test images with a non-zero value there:",
      int((x_test[:, constant] != 0).any(axis=1).sum()))

norm = layers.Normalization()
norm.adapt(x_train)
out = ops.convert_to_numpy(norm(x_test))
other = np.delete(out, constant, axis=1)
print("largest output on the other pixels:", round(float(np.abs(other).max()), 1))
print("largest output on the constant pixels:", float(np.abs(out[:, constant]).max()))
```

```
pixels that are constant in training: [0, 24, 32, 39]
test images with a non-zero value there: 2
largest output on the other pixels: 88.6
largest output on the constant pixels: 10000000.0
```

Four pixels are constant in the training set. In the test set, two images have a value of 1 in one of them. The layer divides that difference by a number close to zero, and the output is 10,000,000. The next layer receives a value more than 100,000 times larger than the largest of the other inputs. It does not raise an error, and the value is finite.

The cause is in the data, not in Keras. A pixel that never changes carries no information, so do not give it to the layer. Check the minimum of the standard deviation before you adapt. Remove constant features, or add a floor to the variance.

Three ways to feed `fit`

Until now the data were NumPy arrays. Arrays are the simplest input, and they fit in memory. When the data do not fit, or when each batch needs work, there are two other inputs.

A `keras.utils.PyDataset` is a Python class with `__len__` and `__getitem__`. Keras asks it for batch `i`, and the class returns the arrays. A `tf.data.Dataset` is a pipeline object of TensorFlow. Keras 3 accepts it on every back end.

The next block trains the same model through all three inputs for the same 10 epochs with batch size 32.

```
import warnings

import keras
import numpy as np
import tensorflow as tf
from keras import layers
from examples.digits import load

x_train, x_test, y_train, y_test = load()
loss = keras.losses.SparseCategoricalCrossentropy(from_logits=True)

class Digits(keras.utils.PyDataset):
    def __init__(self, x, y, batch_size, **kwargs):
        super().__init__(**kwargs)
        self.x, self.y, self.batch_size = x, y, batch_size
        self.rng = np.random.default_rng(0)
        self.order = self.rng.permutation(len(x))

    def __len__(self):
        return int(np.ceil(len(self.x) / self.batch_size))

    def __getitem__(self, index):
        rows = self.order[index * self.batch_size:(index + 1) * self.batch_size]
        return self.x[rows], self.y[rows]

    def on_epoch_end(self):
        self.rng.shuffle(self.order)

def fresh_model():
    keras.utils.set_random_seed(0)
    model = keras.Sequential([keras.Input((64,)), layers.Dense(64, activation="relu"),
layers.Dense(10)])
    model.compile("adam", loss, metrics=["accuracy"])
    return model

validation = (x_test, y_test)
pydataset = Digits(x_train, y_train, 32)
print("batches per epoch:", len(pydataset), "last batch shape:", pydataset[len(pydataset) - 1]
[0].shape)

h = fresh_model().fit(x_train, y_train, epochs=10, batch_size=32, validation_data=validation,
verbose=0)
print("NumPy arrays :", round(h.history["val_accuracy"][-1], 3))
h = fresh_model().fit(pydataset, epochs=10, validation_data=validation, verbose=0)
print("PyDataset    :", round(h.history["val_accuracy"][-1], 3))

pipeline = (tf.data.Dataset.from_tensor_slices((x_train, y_train))
            .shuffle(len(x_train), seed=0).batch(32).prefetch(tf.data.AUTOTUNE))
with warnings.catch_warnings(record=True) as caught:
    warnings.simplefilter("always")
    h = fresh_model().fit(pipeline, epochs=10, validation_data=validation, verbose=0)
print("tf.data      :", round(h.history["val_accuracy"][-1], 3))
print("warning:", str(caught[0].message)[:62])
```

```
batches per epoch: 43 last batch shape: (3, 64)
NumPy arrays : 0.951
PyDataset    : 0.949
tf.data      : 0.944
warning: `shuffle=True` was passed, but will be ignored since the data
```

The three accuracies are within 0.007 of each other, which is 3 images of 450. The pipelines are equal for this purpose. The last batch of the `PyDataset` has 3 images, because 1,347 is not a multiple of 32. Keras uses the short batch.

The warning is useful. `fit` shuffles arrays by default, with `shuffle=True`. It cannot shuffle a `tf.data.Dataset`, so it ignores the argument and says so. The pipeline must shuffle for itself, with `.shuffle(buffer_size)`. A pipeline without a shuffle sends the batches in file order in every epoch.

How it works

`Normalization.adapt` is not training. It makes one pass over the data and stores three numbers for each feature: the mean, the variance and the count. They are weights of the layer, but they are not trainable, and no gradient touches them. The layer then computes `(x - mean) / max(sqrt(variance), epsilon)`, where `epsilon` is the Keras back end constant.

The constant is the cause of the 10,000,000 in this chapter. A pixel with zero variance has a standard deviation of 0, so the divisor becomes `epsilon`, which is 1e-7. A value of 1 in that pixel gives 1 / 1e-7 = 10,000,000.

The next block checks the formula against the layer, and it prints the constant.

```
import keras
import numpy as np
from keras import layers, ops
from examples.digits import load

x_train, x_test, _, _ = load(scale=False)
layer = layers.Normalization()
layer.adapt(x_train)

mean, variance = x_train.mean(axis=0), x_train.var(axis=0)
epsilon = keras.backend.epsilon()
by_hand = (x_test - mean) / np.maximum(np.sqrt(variance), epsilon)
result = ops.convert_to_numpy(layer(x_test))

print("epsilon:", epsilon)
print("stored mean equals the NumPy mean:", bool(np.allclose(ops.convert_to_numpy(layer.mean)
[0], mean, atol=1e-5)))
print("formula equals the layer:", bool(np.allclose(result, by_hand, rtol=1e-4, atol=1e-4)))
print("largest value of the formula:", float(np.abs(by_hand).max()))
```

```
epsilon: 1e-07
stored mean equals the NumPy mean: True
formula equals the layer: True
largest value of the formula: 10000000.0
```

The invariant is this: after `adapt`, the layer is a fixed affine map. The statistics come from the data that you give to `adapt`. They stay fixed for every later call.

Two consequences follow. Adapt on the training set only, or the test set leaks into the model. Check for zero variance before you adapt, because the divisor has a floor and not a guard.

Reference

```
import keras
from keras import layers
from examples.signatures import show

show(layers.Normalization, ("Normalization.adapt", layers.Normalization.adapt),
     ("PyDataset", keras.utils.PyDataset.__init__), ("utils.to_categorical",
keras.utils.to_categorical),
     ("Model.fit", keras.Model.fit))
```

```
Normalization(axis=-1, mean=None, variance=None, invert=False, **kwargs)
Normalization.adapt(data)
PyDataset(workers=1, use_multiprocessing=False, max_queue_size=10)
utils.to_categorical(x, num_classes=None)
Model.fit(x=None, y=None, batch_size=None, epochs=1, verbose='auto',
    callbacks=None, validation_split=0.0, validation_data=None, shuffle=True,
    class_weight=None, sample_weight=None, initial_epoch=0,
    steps_per_epoch=None, validation_steps=None, validation_batch_size=None,
    validation_freq=1)
```

parameter	default	what it does	what goes wrong
`Normalization(axis)`	`-1`	the axis that keeps its own statistics	the default gives one mean for each pixel here
`Normalization(mean, variance)`	`None`	fixed statistics, so no `adapt` is needed	—
`adapt(data)`	required	computes mean, variance and count	a constant pixel gives 1e7 on test data
`PyDataset(workers)`	`1`	worker threads or processes for loading	—
`PyDataset(max_queue_size)`	`10`	batches that wait in the queue	—
`fit(validation_split)`	`0.0`	fraction of the last rows held back	sorted rows gave a validation accuracy of 0.015
`fit(shuffle)`	`True`	shuffle each epoch	ignored for `tf.data.Dataset`
`to_categorical(num_classes)`	`None`	width of the one-hot vector	`None` takes the largest label plus 1

A `PyDataset` subclass must define `__len__` and `__getitem__`. It can define `on_epoch_end`. The base class takes only the three loading arguments that the table lists.

What you have now

You have three input pipelines that give the same result. You also have four traps, each with a number. Sorted rows give a validation accuracy of 0.015. SGD on raw pixels gives an accuracy of 0.296. A constant pixel gives an output of 10,000,000. A `shuffle` argument can do nothing and say so only in a warning.

Chapter 6 uses this data to choose a loss, a metric and an optimizer. Exercises for this chapter are in exercises/05.md, and the worked solutions are in exercises/solutions/05.md.

Chapter 6 — Losses, metrics and optimizers

> **Build** — a digit classifier with 97% accuracy on 450 test images, and four measured choices: the pairing of label format and loss, the `from_logits` flag, the optimizer, and gradient clipping.
>
> **Theory kept** — the loss is the number that training lowers. A metric is a number that people read. The two are different objects and they need not agree.
>
> **Source** — Keras 3.15.1 on TensorFlow 2.21.0 and the digits data of scikit-learn 1.9.1. The code is New.
>
> **Run this first** — `uv run python -m examples.ch06_losses_metrics_optimizers`, which is `examples/ch06_losses_metrics_optimizers.py`.

The `compile` call takes three choices: a loss, a list of metrics and an optimizer. Each choice has a default form and a set of ways to be wrong. This chapter measures the ways to be wrong on the digits data of Chapter 5.

A baseline

The model is the same in every block of this chapter: 64 inputs, a hidden layer of 64 units with a ReLU, and 10 outputs. The outputs are raw scores. They are not probabilities. People call raw scores logits.

```
import keras
from keras import layers
from examples.digits import load

x_train, x_test, y_train, y_test = load()
keras.utils.set_random_seed(0)
model = keras.Sequential([keras.Input((64,)), layers.Dense(64, activation="relu"),
layers.Dense(10)])
model.compile(
    optimizer="adam",
    loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    metrics=["accuracy"],
)
history = model.fit(x_train, y_train, epochs=20, batch_size=32,
                    validation_data=(x_test, y_test), verbose=0)
print("train loss      :", round(history.history["loss"][-1], 4))
print("test loss       :", round(history.history["val_loss"][-1], 4))
print("test accuracy   :", round(history.history["val_accuracy"][-1], 4))
print("history keys    :", sorted(history.history))
```

```
train loss      : 0.1283
test loss       : 0.1547
test accuracy   : 0.9711
history keys    : ['accuracy', 'loss', 'val_accuracy', 'val_loss']
```

The model reaches an accuracy of 0.9711 on the test images: 437 of 450 are correct. The history has four keys. Every key that starts with `val_` is the same quantity measured on the validation data.

The label format and the loss

Class labels come in two formats. A label can be an integer, such as 7. It can also be a one-hot vector with 10 values and a 1 at position 7. The two formats need two loss classes: `SparseCategoricalCrossentropy` for integers and `CategoricalCrossentropy` for one-hot vectors.

They compute the same number. The next block checks this claim.

```
import keras
from keras import layers
from examples.digits import load

x_train, x_test, y_train, y_test = load()

def run(loss, y_train, y_test):
    keras.utils.set_random_seed(0)
    model = keras.Sequential([keras.Input((64,)), layers.Dense(64, activation="relu"),
layers.Dense(10)])
    model.compile("adam", loss, metrics=["accuracy"])
    h = model.fit(x_train, y_train, epochs=20, batch_size=32,
                  validation_data=(x_test, y_test), verbose=0)
    return round(h.history["val_loss"][-1], 4), round(h.history["val_accuracy"][-1], 4)

integer = run(keras.losses.SparseCategoricalCrossentropy(from_logits=True), y_train, y_test)
one_hot = run(keras.losses.CategoricalCrossentropy(from_logits=True),
              keras.utils.to_categorical(y_train, 10), keras.utils.to_categorical(y_test, 10))
print("integer labels :", integer)
print("one-hot labels :", one_hot)
print("equal          :", integer == one_hot)
```

```
integer labels : (0.1547, 0.9711)
one-hot labels : (0.1547, 0.9711)
equal          : True
```

The result is the same to four digits. Use integer labels. They are smaller, and there is no step that can build the vectors wrongly. The metric name `"accuracy"` also adapts: Keras picks the sparse or the dense version from the shape of the labels.

The `from_logits` flag

The last layer of the model can end with a softmax, so that the outputs are probabilities. Or it can end with no activation, so that the outputs are logits. The loss must know which one it gets. The flag `from_logits` tells it.

There are three combinations that people write. The next block trains the three and prints the test loss and accuracy.

```
import warnings

import keras
from keras import layers
from examples.digits import load

x_train, x_test, y_train, y_test = load()

def run(last_activation, from_logits):
    keras.utils.set_random_seed(0)
    model = keras.Sequential([keras.Input((64,)), layers.Dense(64, activation="relu"),
                              layers.Dense(10, activation=last_activation)])
    model.compile("adam", keras.losses.SparseCategoricalCrossentropy(from_logits=from_logits),
                  metrics=["accuracy"])
    with warnings.catch_warnings(record=True) as caught:
        warnings.simplefilter("always")
        h = model.fit(x_train, y_train, epochs=20, batch_size=32,
                      validation_data=(x_test, y_test), verbose=0)
    return round(h.history["val_loss"][-1], 4), round(h.history["val_accuracy"][-1], 4),
len(caught) > 0

for label, activation, flag in [
    ("logits,        from_logits=True ", None, True),
    ("softmax,       from_logits=False", "softmax", False),
    ("softmax,       from_logits=True ", "softmax", True),
    ("logits,        from_logits=False", None, False),
]:
    loss, accuracy, warned = run(activation, flag)
    print(f"{label}  val loss {loss:.4f}  val accuracy {accuracy:.4f}  warning: {warned}")
```

```
logits,        from_logits=True   val loss 0.1547  val accuracy 0.9711  warning: False
softmax,       from_logits=False  val loss 0.1547  val accuracy 0.9711  warning: False
softmax,       from_logits=True   val loss 0.1547  val accuracy 0.9711  warning: True
logits,        from_logits=False  val loss 2.1798  val accuracy 0.5711  warning: False
```

The first two rows are the two correct pairings. The third row is the one that older tutorials call a bug: a softmax output with `from_logits=True`. In Keras 3.15.1 the result is the same as the correct pairing to four digits, and Keras gives a warning. The warning says that the output came from a softmax and does not represent logits.

The fourth row is the one that harms. The model outputs raw scores, and the loss treats them as probabilities. The model trains. It gives no error and no warning. The test accuracy is 0.5711 instead of 0.9711.

This number is unstable. In an earlier run of the same setup, without `TF_DETERMINISTIC_OPS`, the accuracy was 0.7089. Do not trust one number from a run that is broken in this way.

The rule is: if the last layer has no activation, set `from_logits=True`. A missing flag is silent.

The optimizer

The optimizer decides how the gradient becomes a step. The next block trains the same model with five optimizers and reports the test accuracy after 1, 5 and 10 epochs.

```
import keras
from keras import layers
from examples.digits import load

x_train, x_test, y_train, y_test = load()
loss = keras.losses.SparseCategoricalCrossentropy(from_logits=True)

optimizers = {
    "SGD 0.1": lambda: keras.optimizers.SGD(0.1),
    "SGD 0.1 momentum 0.9": lambda: keras.optimizers.SGD(0.1, momentum=0.9),
    "RMSprop 0.001": lambda: keras.optimizers.RMSprop(0.001),
    "Adam 0.001": lambda: keras.optimizers.Adam(0.001),
    "AdamW 0.001": lambda: keras.optimizers.AdamW(0.001),
}
for name, make in optimizers.items():
    keras.utils.set_random_seed(0)
    model = keras.Sequential([keras.Input((64,)), layers.Dense(64, activation="relu"),
layers.Dense(10)])
    model.compile(make(), loss, metrics=["accuracy"])
    h = model.fit(x_train, y_train, epochs=10, batch_size=32,
                  validation_data=(x_test, y_test), verbose=0)
    acc = h.history["val_accuracy"]
    print(f"{name:22s} epoch 1 {acc[0]:.3f}   epoch 5 {acc[4]:.3f}   epoch 10 {acc[9]:.3f}")
```

```
SGD 0.1                epoch 1 0.716   epoch 5 0.940   epoch 10 0.962
SGD 0.1 momentum 0.9   epoch 1 0.913   epoch 5 0.969   epoch 10 0.960
RMSprop 0.001          epoch 1 0.693   epoch 5 0.924   epoch 10 0.951
Adam 0.001             epoch 1 0.693   epoch 5 0.927   epoch 10 0.951
AdamW 0.001            epoch 1 0.693   epoch 5 0.924   epoch 10 0.951
```

After the first epoch the optimizers are far apart. SGD with momentum is at 0.913 and the adaptive optimizers are at 0.693. After 10 epochs they are within 0.011 of each other, which is 5 images of 450. Do not rank them by the last column.

The first column is a real difference. Momentum keeps a running sum of past gradients, so it moves faster in a direction that stays the same. The practical statement is limited. On this small problem, momentum SGD reaches 90% sooner. On other problems the order changes.

AdamW is Adam with weight decay that is applied to the weights directly. On this problem the decay is too small to show.

The learning rate schedule

A schedule changes the learning rate during training. Pass a schedule object in place of a number. The next block shows the values of a cosine decay and trains with it.

```
import keras
from keras import layers
from examples.digits import load

x_train, x_test, y_train, y_test = load()
steps = 10 * (len(x_train) // 32)
schedule = keras.optimizers.schedules.CosineDecay(0.01, decay_steps=steps)
print("rate at step 0, middle, end:",
      [round(float(schedule(s)), 5) for s in (0, steps // 2, steps)])

def run(learning_rate):
    keras.utils.set_random_seed(0)
    model = keras.Sequential([keras.Input((64,)), layers.Dense(64, activation="relu"),
layers.Dense(10)])
    model.compile(keras.optimizers.Adam(learning_rate),
                  keras.losses.SparseCategoricalCrossentropy(from_logits=True), metrics=
["accuracy"])
    h = model.fit(x_train, y_train, epochs=10, batch_size=32,
                  validation_data=(x_test, y_test), verbose=0)
    return round(h.history["val_accuracy"][-1], 4)

print("constant 0.01   :", run(0.01))
print("cosine from 0.01:", run(schedule))
```

```
rate at step 0, middle, end: [0.01, 0.005, 0.0]
constant 0.01   : 0.9733
cosine from 0.01: 0.9689
```

The schedule works as described: the rate starts at 0.01 and ends at 0. On this problem it does not help. The constant rate is 0.0044 higher, which is 2 images of 450. A schedule pays back on longer runs and larger models. Do not add one without a measurement.

Gradient clipping

A single bad batch can produce a gradient that is far larger than the rest. Clipping limits the size of the step. Keras has three clipping arguments on every optimizer: `clipnorm`, `clipvalue` and `global_clipnorm`.

The next block uses a learning rate that is too large on purpose. It shows which of the three arguments rescues the run.

```
import keras
from keras import layers
from examples.digits import load

x_train, x_test, y_train, y_test = load()

def run(**clipping):
    keras.utils.set_random_seed(0)
    model = keras.Sequential([keras.Input((64,)), layers.Dense(64, activation="relu"),
layers.Dense(10)])
    model.compile(keras.optimizers.SGD(2.0, **clipping),
                  keras.losses.SparseCategoricalCrossentropy(from_logits=True), metrics=
["accuracy"])
    h = model.fit(x_train, y_train, epochs=5, batch_size=32,
                  validation_data=(x_test, y_test), verbose=0)
    return round(h.history["val_accuracy"][-1], 3)

print("no clipping          :", run())
print("clipnorm=1.0         :", run(clipnorm=1.0))
print("clipvalue=0.1        :", run(clipvalue=0.1))
print("global_clipnorm=1.0  :", run(global_clipnorm=1.0))
```

```
no clipping          : 0.1
clipnorm=1.0         : 0.1
clipvalue=0.1        : 0.942
global_clipnorm=1.0  : 0.96
```

Without clipping the run fails: the accuracy is 0.1, which is chance for 10 classes. `clipnorm` does not help in this run. The argument `clipvalue` helps in part, and `global_clipnorm` rescues the run.

The three arguments act in different ways. `clipnorm` rescales the gradient of each weight array alone. `clipvalue` limits each number of the gradient to the given range. `global_clipnorm` rescales all gradients together by one factor, so the direction of the update stays the same. The measurement shows the effect and not the reason. On this problem only `global_clipnorm` rescued the run fully.

How it works

Cross-entropy is the negative log of the probability that the model gave to the right class. With raw scores, a softmax makes the probabilities. The two steps combine into one formula that is stable in 32-bit numbers: `loss = logsumexp(scores) - scores[label]`. This is what `from_logits=True` computes.

With `from_logits=False`, Keras assumes that the numbers are probabilities already. It clips each value to the range from `epsilon` to `1 - epsilon`, takes the log, and then applies the same stable formula as before. If the numbers are raw scores, the result has no meaning.

A negative score becomes `epsilon`, and the model cannot see how negative it was. The result is still a number, and the optimizer lowers it.

The next block computes both losses by hand and compares them with Keras.

```
import keras
import numpy as np
from keras import ops

rng = np.random.default_rng(0)
scores = rng.normal(size=(8, 10)).astype("float32")
labels = rng.integers(0, 10, size=(8,))
rows = np.arange(8)

shifted = scores - scores.max(axis=1, keepdims=True)
log_sum_exp = np.log(np.exp(shifted).sum(axis=1)) + scores.max(axis=1)
correct = float(np.mean(log_sum_exp - scores[rows, labels]))

epsilon = keras.backend.epsilon()
clipped = np.clip(scores, epsilon, 1 - epsilon)
wrong = float(np.mean(np.log(clipped.sum(axis=1)) - np.log(clipped[rows, labels])))

with_logits = keras.losses.SparseCategoricalCrossentropy(from_logits=True)(labels, scores)
without = keras.losses.SparseCategoricalCrossentropy(from_logits=False)(labels, scores)
print("from_logits=True : by hand %.4f, Keras %.4f" % (correct,
float(ops.convert_to_numpy(with_logits))))
print("from_logits=False: by hand %.4f, Keras %.4f" % (wrong,
float(ops.convert_to_numpy(without))))
```

```
from_logits=True : by hand 3.0124, Keras 3.0124
from_logits=False: by hand 13.2655, Keras 13.2655
```

Both formulas match Keras. The second one is a number, and it is not cross-entropy. The invariant is this: the loss defines the direction of every step. A wrong flag gives a valid loss for the wrong problem. The optimizer follows it without a complaint.

This is the reason that the model in this chapter trained to 0.5711 and not to 0.9711.

Reference

```
import keras
from examples.signatures import show

show(keras.losses.SparseCategoricalCrossentropy, keras.optimizers.SGD, keras.optimizers.Adam,
     keras.optimizers.schedules.CosineDecay, keras.metrics.SparseTopKCategoricalAccuracy)
```

```
SparseCategoricalCrossentropy(from_logits=False, ignore_class=None,
    reduction='sum_over_batch_size', axis=-1,
    name='sparse_categorical_crossentropy', dtype=None)
SGD(learning_rate=0.01, momentum=0.0, nesterov=False, weight_decay=None,
    clipnorm=None, clipvalue=None, global_clipnorm=None, use_ema=False,
    ema_momentum=0.99, ema_overwrite_frequency=None, loss_scale_factor=None,
    gradient_accumulation_steps=None, name=None, **kwargs)
Adam(learning_rate=0.001, beta_1=0.9, beta_2=0.999, epsilon=1e-07,
    amsgrad=False, weight_decay=None, clipnorm=None, clipvalue=None,
    global_clipnorm=None, use_ema=False, ema_momentum=0.99,
    ema_overwrite_frequency=None, loss_scale_factor=None,
    gradient_accumulation_steps=None, name=None, **kwargs)
CosineDecay(initial_learning_rate, decay_steps, alpha=0.0, name='CosineDecay',
    warmup_target=None, warmup_steps=0)
SparseTopKCategoricalAccuracy(k=5, name='sparse_top_k_categorical_accuracy',
    dtype=None, from_sorted_ids=False)
```

parameter	default	what it does	what goes wrong
`SparseCategoricalCrossentropy(from_logits)`	`False`	the output is raw scores, not probabilities	`False` on raw scores gave 0.5711 and no warning
`SparseCategoricalCrossentropy(reduction)`	`'sum_over_batch_size'`	how the per-sample values combine	—
`SGD(learning_rate)`	`0.01`	size of the step	1.5 gave `nan` on the line (Chapter 1)
`SGD(momentum)`	`0.0`	share of the previous step that carries over	0.9 reached 0.913 after one epoch, against 0.716
`Adam(learning_rate)`	`0.001`	base size of the step	too small for the line in 100 epochs
`Adam(epsilon)`	`1e-07`	floor of the divisor in the update	—
`clipnorm`, `clipvalue`, `global_clipnorm`	`None`	three kinds of gradient clipping	at a rate of 2.0 only `global_clipnorm` rescued the run
`CosineDecay(decay_steps)`	required	steps over which the rate falls to `alpha` times the start	—
`SparseTopKCategoricalAccuracy(k)`	`5`	the label counts if it is in the best `k`	the default of 5 is high for 10 classes

What you have now

You have a classifier that reaches 0.9711, and a short list of measured choices. Use integer labels and `from_logits=True`. Do not rank optimizers by a single final number. Add a schedule or clipping only when a run shows the need.

Chapter 7 shows how to stop a run, save the best weights and react to what happens during training. Exercises for this chapter are in exercises/06.md, and the worked solutions are in exercises/solutions/06.md.

Chapter 7 — Callbacks and training control

> **Build** — a training run that stops itself when it should, keeps the best weights, lowers its own learning rate and stops on `nan`, and two custom callbacks of ten lines each.
>
> **Theory kept** — a callback is an object that Keras calls at fixed points of the training loop. It can read the logs, and it can change the model or stop the run.
>
> **Source** — Keras 3.15.1 on TensorFlow 2.21.0 and the digits data of scikit-learn 1.9.1. The code is New.
>
> **Run this first** — `uv run python -m examples.ch07_callbacks_and_training_control`, which is `examples/ch07_callbacks_and_training_control.py`.

Chapter 1 showed two runs that finished without an error and gave a bad result. A fixed number of epochs is a guess. A callback replaces the guess with a rule that reads the run.

All the runs in this chapter use the same overfitting setup. The training set is 200 images from the digits data, and the model has two hidden layers of 256 units. This model memorizes 200 images in a few epochs, so the validation loss falls and then rises. That shape is what the callbacks react to.

Stop when the validation loss stops falling

`EarlyStopping` watches one number in the logs. It stops the run when that number has not improved for `patience` epochs. The argument `restore_best_weights` decides which weights you get back.

```
import keras
import numpy as np
from keras import layers
from examples.digits import load

x_train, x_test, y_train, y_test = load()
rows = np.random.default_rng(0).choice(len(x_train), 200, replace=False)
x_small, y_small = x_train[rows], y_train[rows]
loss = keras.losses.SparseCategoricalCrossentropy(from_logits=True)

def overfitting_model(learning_rate=0.001):
    keras.utils.set_random_seed(0)
    model = keras.Sequential([keras.Input((64,)), layers.Dense(256, activation="relu"),
                              layers.Dense(256, activation="relu"), layers.Dense(10)])
    model.compile(keras.optimizers.Adam(learning_rate), loss, metrics=["accuracy"])
    return model

for restore in (False, True):
    model = overfitting_model()
    stopper = keras.callbacks.EarlyStopping(monitor="val_loss", patience=5,
restore_best_weights=restore)
    history = model.fit(x_small, y_small, epochs=100, batch_size=32,
                        validation_data=(x_test, y_test), callbacks=[stopper], verbose=0)
    val_loss = history.history["val_loss"]
    final = model.evaluate(x_test, y_test, verbose=0)[0]
    print(f"restore_best_weights={restore!s:5s} epochs run {len(val_loss)}  "
          f"best epoch {int(np.argmin(val_loss)) + 1}  best val loss {min(val_loss):.4f}  "
          f"loss of the returned model {final:.4f}")
print("default of restore_best_weights:",
keras.callbacks.EarlyStopping().restore_best_weights)
```

```
restore_best_weights=False epochs run 36  best epoch 31  best val loss 0.2176  loss of the
returned model 0.2186
restore_best_weights=True  epochs run 36  best epoch 31  best val loss 0.2176  loss of the
returned model 0.2176
default of restore_best_weights: False
```

The run stops at epoch 36. The best epoch is 31, five epochs earlier, which is the patience. With `restore_best_weights=False` the returned model is the one from epoch 36, and its loss is a little higher than the best. With `restore_best_weights=True` the returned model has the weights of epoch 31, and its loss equals the best value.

The last line of the block shows the default of `restore_best_weights`. It is `False`. If you do not set it, you get the last model and not the best one.

Save the best model to disk

`ModelCheckpoint` writes the model to a file. With `save_best_only=True` it writes only when the monitored value improves. The file is a complete model: architecture, weights and optimizer state. Chapter 14 opens the format.

```
import os
import tempfile

import keras
import numpy as np
from keras import layers
from examples.digits import load

x_train, x_test, y_train, y_test = load()
rows = np.random.default_rng(0).choice(len(x_train), 200, replace=False)
x_small, y_small = x_train[rows], y_train[rows]
loss = keras.losses.SparseCategoricalCrossentropy(from_logits=True)

keras.utils.set_random_seed(0)
model = keras.Sequential([keras.Input((64,)), layers.Dense(256, activation="relu"),
                          layers.Dense(256, activation="relu"), layers.Dense(10)])
model.compile("adam", loss, metrics=["accuracy"])

path = os.path.join(tempfile.mkdtemp(), "best.keras")
checkpoint = keras.callbacks.ModelCheckpoint(path, monitor="val_loss", save_best_only=True)
history = model.fit(x_small, y_small, epochs=60, batch_size=32,
                    validation_data=(x_test, y_test), callbacks=[checkpoint], verbose=0)

best = keras.saving.load_model(path)
val_loss = history.history["val_loss"]
print("file exists:", os.path.exists(path), " suffix:", os.path.splitext(path)[1])
print("best epoch:", int(np.argmin(val_loss)) + 1, " last epoch:", len(val_loss))
print("val loss, best epoch :", round(min(val_loss), 4))
print("val loss, last epoch :", round(val_loss[-1], 4))
print("val loss, file       :", round(best.evaluate(x_test, y_test, verbose=0)[0], 4))
```

```
file exists: True  suffix: .keras
best epoch: 31  last epoch: 60
val loss, best epoch : 0.2176
val loss, last epoch : 0.2297
val loss, file       : 0.2176
```

The file holds the model of the best epoch. After 60 epochs the last model is worse than the file. This is the setup to use for long runs: the run can go on, and the best model is on disk when it ends.

Lower the learning rate on a plateau

`ReduceLROnPlateau` multiplies the learning rate by `factor` when the monitored value has not improved for `patience` epochs. Keras writes the learning rate into the history under the key `learning_rate`, so no extra code is needed to see it.

```
import keras
import numpy as np
from keras import layers
from examples.digits import load

x_train, x_test, y_train, y_test = load()
loss = keras.losses.SparseCategoricalCrossentropy(from_logits=True)

keras.utils.set_random_seed(0)
model = keras.Sequential([keras.Input((64,)), layers.Dense(64, activation="relu"),
layers.Dense(10)])
model.compile(keras.optimizers.Adam(0.01), loss, metrics=["accuracy"])
reducer = keras.callbacks.ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=1,
min_lr=0.0001)
history = model.fit(x_train, y_train, epochs=15, batch_size=32,
                    validation_data=(x_test, y_test), callbacks=[reducer], verbose=0)

print("keys:", sorted(history.history))
print("val loss     :", [round(v, 3) for v in history.history["val_loss"]])
print("learning rate:", [round(v, 5) for v in history.history["learning_rate"]])
```

```
keys: ['accuracy', 'learning_rate', 'loss', 'val_accuracy', 'val_loss']
val loss     : [0.339, 0.185, 0.145, 0.12, 0.104, 0.097, 0.09, 0.085, 0.08, 0.078, 0.081,
0.084, 0.076, 0.073, 0.073]
learning rate: [0.01, 0.01, 0.01, 0.01, 0.01, 0.01, 0.01, 0.01, 0.01, 0.01, 0.01, 0.005,
0.0025, 0.0025, 0.0025]
```

The validation loss rises at epoch 11, from 0.078 to 0.081. With `patience=1`, the rate halves at once, from 0.01 to 0.005. The loss rises again at epoch 12, and the rate halves again to 0.0025. The `learning_rate` entry of an epoch is the rate that the epoch used. The reduction after epoch 11 shows in the entry for epoch 12.

Stop on `nan`

Chapter 1 trained a line with `SGD(1.5)` and got a loss of `nan` without an error. `TerminateOnNaN` turns that silent result into a stop.

```
import keras
import numpy as np

keras.utils.set_random_seed(0)

x = np.linspace(-1, 1, 64, dtype="float32").reshape(-1, 1)
model = keras.Sequential([keras.Input((1,)), keras.layers.Dense(1)])
model.compile(keras.optimizers.SGD(1.5), "mse")

history = model.fit(x, 2 * x + 1, epochs=100, batch_size=16,
                    callbacks=[keras.callbacks.TerminateOnNaN()], verbose=0)
print("epochs run:", len(history.history["loss"]), "of 100")
print("last loss:", history.history["loss"][-1])
```

```
Batch 3: Invalid loss, terminating training
epochs run: 16 of 100
last loss: inf
```

The run stops in epoch 16 and not in epoch 100. The callback prints the batch where the loss became invalid. This callback has no setting, and it costs nothing. Add it to every run that you do not watch.

Write a callback

A callback is a class with methods named `on_epoch_end`, `on_train_batch_end` and others. Each method gets the logs of that moment. The model is in `self.model`. To stop the run, set `self.model.stop_training = True`.

The first callback stops the run when the validation accuracy reaches a target. The second one records the loss of every batch, which the history does not keep.

```
import keras
import numpy as np
from keras import layers
from examples.digits import load

x_train, x_test, y_train, y_test = load()
loss = keras.losses.SparseCategoricalCrossentropy(from_logits=True)

class StopAtAccuracy(keras.callbacks.Callback):
    def __init__(self, target):
        super().__init__()
        self.target = target

    def on_epoch_end(self, epoch, logs=None):
        if logs["val_accuracy"] >= self.target:
            self.model.stop_training = True

class BatchLosses(keras.callbacks.Callback):
    def on_train_begin(self, logs=None):
        self.values = []

    def on_train_batch_end(self, batch, logs=None):
        self.values.append(float(logs["loss"]))

keras.utils.set_random_seed(0)
model = keras.Sequential([keras.Input((64,)), layers.Dense(64, activation="relu"),
layers.Dense(10)])
model.compile("adam", loss, metrics=["accuracy"])
recorder = BatchLosses()
history = model.fit(x_train, y_train, epochs=50, batch_size=32, validation_data=(x_test,
y_test),
                    callbacks=[StopAtAccuracy(0.95), recorder], verbose=0)

epochs = len(history.history["loss"])
print("stopped after epoch:", epochs, " val accuracy:", round(history.history["val_accuracy"]
[-1], 4))
print("batches recorded:", len(recorder.values), "= epochs x 43:", epochs * 43)
print("loss of the first and last batch:", round(recorder.values[0], 3),
round(recorder.values[-1], 3))

keras.utils.set_random_seed(0)
model = keras.Sequential([keras.Input((64,)), layers.Dense(64, activation="relu"),
layers.Dense(10)])
model.compile("adam", loss, metrics=["accuracy"])
try:
    model.fit(x_train, y_train, epochs=1, batch_size=32, callbacks=[StopAtAccuracy(0.95)],
verbose=0)
except KeyError as error:
    print("without validation data: KeyError", error)
```

```
stopped after epoch: 10  val accuracy: 0.9511
batches recorded: 430 = epochs x 43: 430
loss of the first and last batch: 2.297 0.258
without validation data: KeyError 'val_accuracy'
```

The run stops after epoch 10, when the validation accuracy first reaches 0.95. The recorder holds 430 values, one for each batch. The last value is not the loss of the last batch alone, which has only 3 images. How it works shows what it is.

Two warnings apply to the custom stop. The target is read on the validation data, so it is a decision made with the validation set (Chapter 8 returns to this). Also, `logs` can lack a key. The last lines of the block pass no validation data, and `logs["val_accuracy"]` raises a `KeyError`.

How it works

`fit` calls the methods of every callback at fixed points. The callbacks form a list, and each point calls them in list order. A callback that you write overrides only the methods it needs.

The next block records the points of a run with 2 batches in one epoch and a validation set. It prints them in order.

```
import keras
import numpy as np
from keras import layers

class Trace(keras.callbacks.Callback):
    def __init__(self):
        super().__init__()
        self.events = []

for name in ("on_train_begin", "on_epoch_begin", "on_train_batch_begin", "on_train_batch_end",
             "on_test_begin", "on_test_batch_begin", "on_test_batch_end", "on_test_end",
             "on_epoch_end", "on_train_end"):
    setattr(Trace, name, lambda self, *args, _name=name, **kwargs: self.events.append(_name))

x = np.zeros((64, 4), dtype="float32")
y = np.zeros((64, 1), dtype="float32")
keras.utils.set_random_seed(0)
model = keras.Sequential([keras.Input((4,)), layers.Dense(1)])
model.compile("sgd", "mse")
trace = Trace()
model.fit(x, y, epochs=1, batch_size=32, validation_data=(x[:32], y[:32]), callbacks=[trace],
verbose=0)
for event in trace.events:
    print(event)
```

```
on_train_begin
on_epoch_begin
on_train_batch_begin
on_train_batch_end
on_train_batch_begin
on_train_batch_end
on_test_begin
on_test_batch_begin
on_test_batch_end
on_test_end
on_epoch_end
on_train_end
```

The run has one epoch of two training batches, then a validation pass of one batch. The order is the contract. `on_epoch_end` comes after the validation pass, so its `logs` contain the `val_` keys. This is the reason that `StopAtAccuracy` can read `logs["val_accuracy"]` in `on_epoch_end` and not in `on_train_batch_end`.

The `logs` argument of a batch method does not hold the loss of that batch. The next block records it for two epochs and compares the last batch of each epoch with the history.

```
import keras
import numpy as np
from keras import layers
from examples.digits import load

x_train, x_test, y_train, y_test = load()

class BatchLosses(keras.callbacks.Callback):
    def on_train_begin(self, logs=None):
        self.values = []

    def on_train_batch_end(self, batch, logs=None):
        self.values.append(float(logs["loss"]))

keras.utils.set_random_seed(0)
model = keras.Sequential([keras.Input((64,)), layers.Dense(64, activation="relu"),
layers.Dense(10)])
model.compile("adam", keras.losses.SparseCategoricalCrossentropy(from_logits=True), metrics=
["accuracy"])
recorder = BatchLosses()
history = model.fit(x_train, y_train, epochs=2, batch_size=32, shuffle=False,
                    callbacks=[recorder], verbose=0).history

values = np.array(recorder.values)
print("batches recorded:", len(values))
print("last batch of epoch 1:", round(values[42], 4), " history loss of epoch 1:",
round(history["loss"][0], 4))
print("last batch of epoch 2:", round(values[85], 4), " history loss of epoch 2:",
round(history["loss"][1], 4))
print("first four values:", [round(float(v), 3) for v in values[:4]])
```

```
batches recorded: 86
last batch of epoch 1: 2.0879  history loss of epoch 1: 2.0879
last batch of epoch 2: 1.5542  history loss of epoch 2: 1.5542
first four values: [2.37, 2.318, 2.313, 2.32]
```

The value at the last batch of an epoch equals the loss of that epoch in the history. The callback gets a running average over the batches of the epoch so far, and not the loss of one batch. The invariant is this: the logs of a batch are the epoch average up to that batch. To get the loss of a single batch, compute it yourself in a `train_step` (Chapter 9).

Reference

```
import keras
from examples.signatures import show

show(keras.callbacks.EarlyStopping, keras.callbacks.ModelCheckpoint,
     keras.callbacks.ReduceLROnPlateau, keras.callbacks.TerminateOnNaN,
keras.callbacks.Callback)
```

```
EarlyStopping(monitor='val_loss', min_delta=0, patience=0, verbose=0,
    mode='auto', baseline=None, restore_best_weights=False,
    start_from_epoch=0)
ModelCheckpoint(filepath, monitor='val_loss', verbose=0, save_best_only=False,
    save_weights_only=False, mode='auto', save_freq='epoch',
    initial_value_threshold=None)
ReduceLROnPlateau(monitor='val_loss', factor=0.1, patience=10, verbose=0,
    mode='auto', min_delta=0.0001, cooldown=0, min_lr=0.0, **kwargs)
TerminateOnNaN(raise_error: bool = False)
Callback()
```

parameter	default	what it does	what goes wrong
`EarlyStopping(monitor)`	`'val_loss'`	the log key to watch	—
`EarlyStopping(patience)`	`0`	epochs without improvement before the stop	—
`EarlyStopping(restore_best_weights)`	`False`	return the weights of the best epoch	`False` returns the last model, not the best
`EarlyStopping(mode)`	`'auto'`	direction of "better"	`'min'` on accuracy stopped at epoch 6 with 0.6444
`ModelCheckpoint(filepath)`	required	where to write the model	the chapter wrote a `.keras` file and loaded it back
`ModelCheckpoint(save_best_only)`	`False`	write only on improvement	—
`ReduceLROnPlateau(factor)`	`0.1`	multiplier of the rate	—
`ReduceLROnPlateau(patience)`	`10`	epochs without improvement before the cut	—
`TerminateOnNaN(raise_error)`	`False`	raise an error and not only stop	—

What you have now

You have the four callbacks that most runs need: `EarlyStopping` with `restore_best_weights=True`, `ModelCheckpoint` with `save_best_only=True`, `ReduceLROnPlateau` and `TerminateOnNaN`. You also have the two methods that cover most custom callbacks: `on_epoch_end` and `on_train_batch_end`.

Chapter 8 takes the overfitting setup of this chapter and measures what regularization does to it. Exercises for this chapter are in exercises/07.md, and the worked solutions are in exercises/solutions/07.md.

Chapter 8 — Overfitting and regularization

> **Build** — a table that compares dropout, an L2 penalty, batch normalization and input noise on a model that memorizes 200 images, a learning curve over six training-set sizes, and a measurement of what the validation loss contains when a penalty is present.
>
> **Theory kept** — overfitting is a gap between the score on the training data and the score on new data. A regularizer shrinks the gap. It does not always raise the score on new data.
>
> **Source** — Keras 3.15.1 on TensorFlow 2.21.0 and the digits data of scikit-learn 1.9.1. The code is New.
>
> **Run this first** — `uv run python -m examples.ch08_overfitting_and_regularization`, which is `examples/ch08_overfitting_and_regularization.py`.

A model overfits when it learns the training examples and not the rule behind them. The signal is a gap: the accuracy on the training data is high, and the accuracy on data that the model did not see is lower. Chapter 7 used `EarlyStopping` to stop at the point where the gap starts to hurt. This chapter asks what else you can do.

The setup is the one of Chapter 7. The training set is 200 images, and the model has two hidden layers of 256 units. This model has about 85,000 weights for 200 examples. It can memorize the set.

Four regularizers, one table

The next block trains the same model with each regularizer and prints the training accuracy, the test accuracy and the gap between them. The four regularizers are:

- **Dropout** sets a random half of the activations to zero at each training step.
- **An L2 penalty** adds the sum of the squared weights, times a factor, to the loss.
- **Batch normalization** rescales the activations of each batch.
- **Gaussian noise** adds random noise to the input at each training step.

Every setting uses the same seed and 60 epochs.

```
import keras
import numpy as np
from keras import layers
from examples.digits import load

x_train, x_test, y_train, y_test = load()
rows = np.random.default_rng(0).choice(len(x_train), 200, replace=False)
x_small, y_small = x_train[rows], y_train[rows]
loss = keras.losses.SparseCategoricalCrossentropy(from_logits=True)

def build(dropout=0.0, l2=0.0, batch_norm=False, noise=0.0):
    keras.utils.set_random_seed(0)
    regularizer = keras.regularizers.L2(l2) if l2 else None
    stack = [keras.Input((64,))]
    if noise:
        stack.append(layers.GaussianNoise(noise))
    for _ in range(2):
        stack.append(layers.Dense(256, kernel_regularizer=regularizer))
        if batch_norm:
            stack.append(layers.BatchNormalization())
        stack.append(layers.Activation("relu"))
        if dropout:
            stack.append(layers.Dropout(dropout))
    stack.append(layers.Dense(10))
    model = keras.Sequential(stack)
    model.compile("adam", loss, metrics=["accuracy"])
    return model

def run(**options):
    model = build(**options)
    h = model.fit(x_small, y_small, epochs=60, batch_size=32,
                  validation_data=(x_test, y_test), verbose=0).history
    return h

settings = {
    "baseline": {},
    "dropout 0.5": dict(dropout=0.5),
    "L2 1e-3": dict(l2=1e-3),
    "batch norm": dict(batch_norm=True),
    "noise 0.3": dict(noise=0.3),
}
print(f"{'setting':14s} {'train acc':>9s} {'val acc':>8s} {'gap':>6s}")
for name, options in settings.items():
    h = run(**options)
    train, val = h["accuracy"][-1], h["val_accuracy"][-1]
    print(f"{name:14s} {train:9.3f} {val:8.3f} {train - val:6.3f}")
```

```
setting        train acc  val acc    gap
baseline           1.000    0.933  0.067
dropout 0.5        0.980    0.940  0.040
L2 1e-3            1.000    0.929  0.071
batch norm         1.000    0.927  0.073
noise 0.3          0.935    0.933  0.002
```

Read the table by columns. The baseline reaches a training accuracy of 1.000 and a test accuracy of 0.933, and the gap is 0.067. Dropout 0.5 lowers the gap to 0.040. Input noise lowers it to 0.002. The L2 penalty and batch normalization leave the gap near the baseline or slightly above.

Now read the test accuracy column alone. It runs from 0.927 to 0.940. The test set has 450 images, and one image is 0.0022. The whole spread is 6 images.

All four settings are within 3 images of the baseline. Input noise has the smallest gap and the same test accuracy as the baseline. It lowered the training accuracy to 0.935 and did not raise the test accuracy.

A regularizer shrinks the gap. It does not always move the test score. Judge a regularizer by the test score, not by the gap.

More data beats every regularizer

The lever that always works is more data. The next block trains one small model on growing subsets of the training images and prints both accuracies.

```
import keras
import numpy as np
from keras import layers
from examples.digits import load

x_train, x_test, y_train, y_test = load()
loss = keras.losses.SparseCategoricalCrossentropy(from_logits=True)

print(f"{'training images':>15s} {'train acc':>10s} {'test acc':>9s}")
for n in (50, 100, 200, 400, 800, 1347):
    rows = np.random.default_rng(0).choice(len(x_train), n, replace=False)
    keras.utils.set_random_seed(0)
    model = keras.Sequential([keras.Input((64,)), layers.Dense(64, activation="relu"),
layers.Dense(10)])
    model.compile("adam", loss, metrics=["accuracy"])
    h = model.fit(x_train[rows], y_train[rows], epochs=60, batch_size=32,
                  validation_data=(x_test, y_test), verbose=0).history
    print(f"{n:15d} {h['accuracy'][-1]:10.3f} {h['val_accuracy'][-1]:9.3f}")
```

```
training images  train acc  test acc
             50      0.940     0.720
            100      0.970     0.829
            200      0.975     0.916
            400      0.998     0.938
            800      0.996     0.973
           1347      0.997     0.973
```

The test accuracy rises from 0.720 with 50 images to 0.916 with 200 images and 0.973 with 800 images. No setting of the previous table moved the test accuracy by more than 0.007 at 200 images. Going from 200 to 800 images moved it by 0.057.

The curve also tells you what to do next. Between 800 and 1,347 images the test accuracy does not change, at 0.973. At that size, more of the same data does not help this model. Below 400 images the curve is steep, and collecting data is the

best use of time.

The validation loss contains the penalty

One detail makes it hard to compare runs with and without a penalty. A regularizer adds a term to the loss, and `evaluate` reports the total. The next block trains a model with a strong L2 penalty and splits the reported loss into its two parts.

```
import keras
import numpy as np
from keras import layers
from examples.digits import load

x_train, x_test, y_train, y_test = load()
rows = np.random.default_rng(0).choice(len(x_train), 200, replace=False)
x_small, y_small = x_train[rows], y_train[rows]

keras.utils.set_random_seed(0)
model = keras.Sequential([
    keras.Input((64,)),
    layers.Dense(256, activation="relu", kernel_regularizer=keras.regularizers.L2(1e-2)),
    layers.Dense(10),
])
model.compile("adam", keras.losses.SparseCategoricalCrossentropy(from_logits=True), metrics=
["accuracy"])
model.fit(x_small, y_small, epochs=60, batch_size=32, verbose=0)

reported = model.evaluate(x_test, y_test, verbose=0)[0]
logits = model.predict(x_test, verbose=0)
plain = float(keras.losses.sparse_categorical_crossentropy(y_test, logits,
from_logits=True).numpy().mean())
penalty = float(sum(float(np.asarray(w)) for w in model.losses))
print("reported by evaluate:", round(reported, 4))
print("cross-entropy alone :", round(plain, 4))
print("penalty             :", round(penalty, 4))
print("sum of the last two :", round(plain + penalty, 4))
```

```
reported by evaluate: 0.5433
cross-entropy alone : 0.3435
penalty             : 0.1998
sum of the last two : 0.5433
```

The reported loss is 0.5433. The cross-entropy alone is 0.3435, and the penalty is 0.1998. The sum of the two parts is the reported value. The penalty depends on the weights and not on the data. It adds the same amount to the loss of every batch of test data.

Two runs with different penalties have losses that cannot be compared. Compare the accuracy, or compute the cross-entropy alone as in the block. This also applies to `EarlyStopping(monitor="val_loss")`. The value that it watches contains the penalty. Watch `val_accuracy` or a metric without the penalty when you use a strong regularizer.

What to do in practice

Use the evidence of the tables in this order.

1. Check the pipeline for leakage and for a bad split (Chapter 5). A large gap is often a data fault.
2. Add data if you can. The learning curve says how much it is worth.
3. Use early stopping with `restore_best_weights=True`. It costs no extra weights, and it directly targets the best epoch.
4. Try one regularizer at a time, and judge it by the test score with a stated bound. One image of 450 is 0.0022.

How it works

Dropout in Keras is *inverted* dropout. In training it sets each unit to zero with probability `rate`, and it multiplies the units that stay by `1 / (1 - rate)`. In inference it does nothing. The scale factor keeps the mean of the output the same in both modes. You do not rescale the weights when you switch to inference.

The next block feeds a tensor of ones through dropout with a rate of 0.5.

```
import keras
import numpy as np
from keras import layers, ops

keras.utils.set_random_seed(0)
dropout = layers.Dropout(0.5)
ones = np.ones((1, 10000), dtype="float32")

training = ops.convert_to_numpy(dropout(ones, training=True))
inference = ops.convert_to_numpy(dropout(ones, training=False))
print("values in training mode:", sorted(set(training.ravel().tolist())))
print("share of units set to zero:", round(float((training == 0).mean()), 3))
print("mean in training mode:", round(float(training.mean()), 3))
print("values in inference mode:", sorted(set(inference.ravel().tolist())))
```

```
values in training mode: [0.0, 2.0]
share of units set to zero: 0.503
mean in training mode: 0.994
values in inference mode: [1.0]
```

In training the output has two values: 0 and 2. The factor is 1 / (1 - 0.5) = 2. The share of zeros is near 0.5 and the mean is near 1. In inference the output is the input.

The invariant is: *the expected value of the output does not change with the rate.* The variance does change. A unit that is kept is twice as large, and a unit that is dropped is zero, so the next layer sees a noisier input. This noise is the regularizer. It is also a reason that a model with dropout trains more slowly. The training accuracy in the history of `fit` is measured with the noise on, so it is lower than the accuracy in inference.

Reference

```
import keras
from keras import layers
from examples.signatures import show

show(layers.Dropout, ("regularizers.L2", keras.regularizers.L2), layers.GaussianNoise,
     ("BatchNormalization", layers.BatchNormalization.__init__))
```

```
Dropout(rate, noise_shape=None, seed=None, **kwargs)
regularizers.L2(l2=0.01)
GaussianNoise(stddev, seed=None, **kwargs)
BatchNormalization(axis=-1, momentum=0.99, epsilon=0.001, center=True,
    scale=True, beta_initializer='zeros', gamma_initializer='ones',
    moving_mean_initializer='zeros', moving_variance_initializer='ones',
    beta_regularizer=None, gamma_regularizer=None, beta_constraint=None,
    gamma_constraint=None, renorm=False, renorm_clipping=None,
    renorm_momentum=0.99, synchronized=False, **kwargs)
```

parameter	default	what it does	what goes wrong
`Dropout(rate)`	required	probability that a unit is set to zero	0.5 lowered the gap from 0.067 to 0.040 and did not raise the test score
`Dropout(seed)`	`None`	seed of the layer's generator	—
`L2(l2)`	`0.01`	factor of the squared-weight penalty	the penalty is part of `val_loss` (Chapter 8)
`GaussianNoise(stddev)`	required	standard deviation of the noise, in training only	0.3 lowered the gap to 0.002 and the training accuracy to 0.935
`BatchNormalization(momentum)`	`0.99`	speed of the moving statistics	the moving values are used at inference

What you have now

You have a table of four regularizers with their gaps and their scores. You also have a learning curve and the rule that a gap is not a score. The result that matters most is a negative one. On this problem, no regularizer beat the baseline by more than 3 images.

Chapter 9 opens the training loop itself. Exercises for this chapter are in exercises/08.md, and the worked solutions are in exercises/solutions/08.md.

Chapter 9 — Custom losses, metrics and training steps

> **Build** — a custom loss, a custom metric and a custom `train_step`, each checked against a built-in reference, and a portable way to change the loss of a model on every back end.
>
> **Theory kept** — `fit` is a loop of four things: a forward pass, a loss, a gradient and an update. Keras lets you replace each of them. Replace the smallest part that does the job.
>
> **Source** — Keras 3.15.1 on TensorFlow 2.21.0 and the digits data of scikit-learn 1.9.1. The code is New.
>
> **Run this first** — `uv run python -m examples.ch09_custom_training`, which is `examples/ch09_custom_training.py`.

The built-in pieces cover most work. Sometimes you need a loss that Keras does not have, a metric that the task defines, or a gradient that you must see. Keras gives a hook for each. This chapter uses each hook. It checks each one against the built-in version of the same thing.

Then you know that the hook is right before you change it.

The rule for the whole chapter is: **first make the custom version equal to the built-in version, then change it.** An equal result proves that the loop is right. A changed result is then the change and nothing else.

A custom loss function

A loss is a function of `y_true` and `y_pred` that returns one value for each sample. Keras averages it over the batch. The next block writes cross-entropy by hand with `keras.ops`. It trains one model with the built-in loss and one with the hand-written loss. Both runs use the same seed and the same batch order.

```
import keras
import numpy as np
from keras import layers, ops
from examples.digits import load

x_train, x_test, y_train, y_test = load()

def my_cross_entropy(y_true, y_pred):
    y_true = ops.cast(y_true, "int32")
    log_probability = ops.log_softmax(y_pred, axis=-1)
    picked = ops.take_along_axis(log_probability, ops.expand_dims(y_true, -1), axis=-1)
    return -ops.mean(picked, axis=-1)

def train(loss):
    keras.utils.set_random_seed(0)
    model = keras.Sequential([keras.Input((64,)), layers.Dense(64, activation="relu"),
layers.Dense(10)])
    model.compile("adam", loss, metrics=["accuracy"])
    history = model.fit(x_train, y_train, epochs=3, batch_size=32, shuffle=False, verbose=0)
    return model, history.history["loss"]

built_in, losses_a = train(keras.losses.SparseCategoricalCrossentropy(from_logits=True))
custom, losses_b = train(my_cross_entropy)
print("built-in loss per epoch:", [round(v, 4) for v in losses_a])
print("custom loss per epoch  :", [round(v, 4) for v in losses_b])
difference = max(float(np.abs(a - b).max()) for a, b in zip(built_in.get_weights(),
custom.get_weights()))
print("largest weight difference below 1e-6:", difference < 1e-6)
```

```
built-in loss per epoch: [2.0879, 1.5542, 1.0979]
custom loss per epoch  : [2.0879, 1.5542, 1.0979]
largest weight difference below 1e-6: True
```

The two runs give the same losses and the same weights, to within rounding of 32-bit numbers. The custom function is now trustworthy, and you can change it.

Note two details of the function. It uses only `keras.ops`, so it runs on every back end. It returns one value for each sample, and Keras does the averaging.

A custom loss class

A function has no parameters. For a loss with a parameter, subclass `keras.losses.Loss`. The class below is cross-entropy with label smoothing. The smoothing moves a share `epsilon` of the target probability from the true class to all classes equally.

```
import keras
import numpy as np
from keras import ops

class SmoothedCrossEntropy(keras.losses.Loss):
    def __init__(self, epsilon=0.1, num_classes=10, **kwargs):
        super().__init__(**kwargs)
        self.epsilon = epsilon
        self.num_classes = num_classes

    def call(self, y_true, y_pred):
        target = ops.one_hot(ops.cast(y_true, "int32"), self.num_classes)
        target = target * (1.0 - self.epsilon) + self.epsilon / self.num_classes
        return -ops.sum(target * ops.log_softmax(y_pred, axis=-1), axis=-1)

    def get_config(self):
        return {**super().get_config(), "epsilon": self.epsilon, "num_classes":
self.num_classes}

rng = np.random.default_rng(0)
logits = rng.normal(size=(32, 10)).astype("float32")
labels = rng.integers(0, 10, size=(32,))

mine = SmoothedCrossEntropy(0.1)(labels, logits)
reference = keras.losses.CategoricalCrossentropy(from_logits=True, label_smoothing=0.1)(
    keras.utils.to_categorical(labels, 10), logits)
print("custom class  :", round(float(ops.convert_to_numpy(mine)), 5))
print("built-in class:", round(float(ops.convert_to_numpy(reference)), 5))
print("config:", SmoothedCrossEntropy(0.2).get_config()["epsilon"])
```

```
custom class  : 2.56721
built-in class: 2.56721
config: 0.2
```

The class matches the built-in loss with label smoothing on 32 random samples. The method `get_config` matters for saving. A loss class without `get_config` loads, but its arguments return to their defaults (Chapter 14 measures this).

A custom metric

A metric keeps state across batches. The loss of an epoch is an average of batch values. A metric such as balanced accuracy cannot be built that way, because it is an average over classes of per-class recall. The metric must keep a count for each class and combine the counts at the end.

A `keras.metrics.Metric` has four methods. `update_state` takes one batch. `result` returns the value. `reset_state` clears the counts at the start of an epoch. The state lives in variables made with `add_variable`.

```
import keras
import numpy as np
from keras import layers, ops
from sklearn.metrics import balanced_accuracy_score
from examples.digits import load

class BalancedAccuracy(keras.metrics.Metric):
    def __init__(self, num_classes=10, name="balanced_accuracy", **kwargs):
        super().__init__(name=name, **kwargs)
        self.num_classes = num_classes
        self.hits = self.add_variable(shape=(num_classes,), initializer="zeros", name="hits")
        self.counts = self.add_variable(shape=(num_classes,), initializer="zeros",
name="counts")

    def update_state(self, y_true, y_pred, sample_weight=None):
        y_true = ops.cast(ops.reshape(y_true, (-1,)), "int32")
        predicted = ops.cast(ops.argmax(y_pred, axis=-1), "int32")
        hit = ops.cast(ops.equal(predicted, y_true), "float32")
        onehot = ops.one_hot(y_true, self.num_classes)
        self.hits.assign(self.hits + ops.sum(onehot * ops.expand_dims(hit, -1), axis=0))
        self.counts.assign(self.counts + ops.sum(onehot, axis=0))

    def result(self):
        return ops.mean(self.hits / ops.maximum(self.counts, 1.0))

    def reset_state(self):
        self.hits.assign(ops.zeros_like(self.hits))
        self.counts.assign(ops.zeros_like(self.counts))

x_train, x_test, y_train, y_test = load()
keras.utils.set_random_seed(0)
model = keras.Sequential([keras.Input((64,)), layers.Dense(64, activation="relu"),
layers.Dense(10)])
model.compile("adam", keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=["accuracy", BalancedAccuracy()])
history = model.fit(x_train, y_train, epochs=3, batch_size=32,
                    validation_data=(x_test, y_test), verbose=0).history

predicted = model.predict(x_test, verbose=0).argmax(axis=1)
print("history keys     :", sorted(history))
print("Keras metric     :", round(history["val_balanced_accuracy"][-1], 5))
print("scikit-learn     :", round(float(balanced_accuracy_score(y_test, predicted)), 5))
print("plain accuracy   :", round(history["val_accuracy"][-1], 5))
```

```
history keys     : ['accuracy', 'balanced_accuracy', 'loss', 'val_accuracy',
'val_balanced_accuracy', 'val_loss']
Keras metric     : 0.89342
scikit-learn     : 0.89342
plain accuracy   : 0.89333
```

The custom metric agrees with the scikit-learn function to five digits. It also differs from plain accuracy, because the 450 test images are not perfectly balanced across classes. On a data set with a rare class the two numbers can differ much more. Balanced accuracy is the one that shows a model that ignores the rare class.

A custom `train_step`

The method `train_step` is the loop body of `fit`. It receives one batch and returns a dictionary of metric values. If you override it, you write the forward pass, the gradient and the update.

This is the one place where Keras 3 code depends on the back end. The gradient comes from the back end: `tf.GradientTape` on TensorFlow, `jax.grad` on JAX, and `loss.backward()` on PyTorch. The block below is for the TensorFlow back end.

It adds one thing to the built-in step: the global norm of the gradient, logged as a metric. Before it adds anything, the block checks that the step reproduces the built-in `fit`.

```
import keras
import numpy as np
import tensorflow as tf
from keras import layers
from examples.digits import load

class GradNormModel(keras.Model):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.grad_norm = keras.metrics.Mean(name="grad_norm")

    @property
    def metrics(self):
        return super().metrics + [self.grad_norm]

    def train_step(self, data):
        x, y = data
        with tf.GradientTape() as tape:
            y_pred = self(x, training=True)
            loss = self.compute_loss(x=x, y=y, y_pred=y_pred)
        gradients = tape.gradient(loss, self.trainable_variables)
        self.optimizer.apply(gradients, self.trainable_variables)
        self.grad_norm.update_state(tf.linalg.global_norm(gradients))
        batch = tf.cast(tf.shape(x)[0], "float32")
        for metric in self.metrics:
            if metric.name == "loss":
                metric.update_state(loss, sample_weight=batch)
            elif metric is not self.grad_norm:
                metric.update_state(y, y_pred)
        return {metric.name: metric.result() for metric in self.metrics}

def make(model_class):
    keras.utils.set_random_seed(0)
    inputs = keras.Input((64,))
    hidden = layers.Dense(64, activation="relu")(inputs)
    model = model_class(inputs, layers.Dense(10)(hidden))
    model.compile("adam", keras.losses.SparseCategoricalCrossentropy(from_logits=True),
metrics=["accuracy"])
    return model

x_train, x_test, y_train, y_test = load()
reference, custom = make(keras.Model), make(GradNormModel)
history_a = reference.fit(x_train, y_train, epochs=3, batch_size=32, shuffle=False,
verbose=0).history
history_b = custom.fit(x_train, y_train, epochs=3, batch_size=32, shuffle=False,
verbose=0).history

print("reference loss:", [round(v, 4) for v in history_a["loss"]])
print("custom loss   :", [round(v, 4) for v in history_b["loss"]])
print("weights equal :", all(np.array_equal(a, b) for a, b in zip(reference.get_weights(),
custom.get_weights())))
print("history keys  :", sorted(history_b))
print("gradient norm per epoch:", [round(v, 3) for v in history_b["grad_norm"]])
```

```
reference loss: [2.0879, 1.5542, 1.0979]
custom loss   : [2.0879, 1.5542, 1.0979]
weights equal : True
history keys  : ['accuracy', 'grad_norm', 'loss']
gradient norm per epoch: [0.901, 0.946, 0.915]
```

The custom loop gives the same losses and the same weights as the built-in loop, bit for bit. The extra key `grad_norm` is the new information: the average global norm of the gradient in each epoch.

Two details in the block cost a debugging session each. You must weight the loss metric by the batch size. The last batch has 3 images, and an unweighted mean gives a different epoch loss.

The second detail is the order of layer construction in `make`. Chapter 15 shows how that order decides the initial weights.

A portable change: override `compute_loss`

A `train_step` that uses `tf.GradientTape` runs on one back end only. If the change that you want is a change of the loss, there is a portable hook. Override `compute_loss`. Keras calls it in the training step of every back end.

The block below adds a penalty on the size of the outputs to a `Sequential` model. It then checks the new loss against a hand calculation.

```
import keras
import numpy as np
from keras import layers, ops
from examples.digits import load

class OutputPenalty(keras.Sequential):
    def compute_loss(self, x=None, y=None, y_pred=None, sample_weight=None, training=True):
        base = super().compute_loss(x=x, y=y, y_pred=y_pred, sample_weight=sample_weight,
training=training)
        return base + 0.01 * ops.mean(ops.square(y_pred))

x_train, x_test, y_train, y_test = load()
keras.utils.set_random_seed(0)
model = OutputPenalty([keras.Input((64,)), layers.Dense(64, activation="relu"),
layers.Dense(10)])
loss = keras.losses.SparseCategoricalCrossentropy(from_logits=True)
model.compile("adam", loss, metrics=["accuracy"])
history = model.fit(x_train, y_train, epochs=3, batch_size=32, validation_data=(x_test,
y_test), verbose=0)

logits = model.predict(x_test, verbose=0)
by_hand = float(loss(y_test, logits)) + 0.01 * float(np.mean(logits ** 2))
reported = model.evaluate(x_test, y_test, verbose=0)[0]
print("loss by hand:", round(by_hand, 4))
print("evaluate    :", round(reported, 4))
print("equal       :", abs(by_hand - reported) < 1e-4)
```

```
loss by hand: 0.9281
evaluate    : 0.9281
equal       : True
```

The reported loss is the cross-entropy plus the penalty. The same class runs on JAX and PyTorch, because the hook is part of the Keras `Model` class. This book ran it on TensorFlow only.

Which hook to use

Use the smallest hook that does the job:

1. A different loss: write a function or a `Loss` class.
2. A different number to read: write a `Metric`.
3. An extra term in the loss that uses the model: override `compute_loss`.
4. A change in how the gradient is used, such as a second optimizer or gradient accumulation: override `train_step`, and accept that it ties the code to one back end.

How it works

The optimizer is a small program with its own state. `optimizer.apply(gradients, variables)` reads each gradient, updates its internal numbers and changes the variables. In `train_step`, this one call replaces the update step of `fit`.

Adam keeps two numbers for each weight: a moving average of the gradient `m` and a moving average of its square `v`. At step `t` the update is `lr × m̂ / (√v̂ + ε)`, where `m̂` and `v̂` are corrected for the start. At the first step, `m̂` equals the gradient `g` and `v̂` equals `g²`.

The update is then `lr × g / (|g| + ε)`, which is close to `lr × sign(g)`. The size of the gradient has almost no effect on the first step.

The next block checks this with `optimizer.apply` on one variable.

```
import keras
import numpy as np
from keras import ops

for gradient in (0.5, 50.0):
    weight = keras.Variable(np.array([1.0], dtype="float32"))
    optimizer = keras.optimizers.Adam(learning_rate=0.1)
    optimizer.apply([ops.convert_to_tensor(np.array([gradient], dtype="float32"))], [weight])
    change = 1.0 - float(ops.convert_to_numpy(weight)[0])
    expected = 0.1 * gradient / (abs(gradient) + 1e-7)
    print(f"gradient {gradient:5.1f}: weight moved by {change:.4f}, formula {expected:.4f}")

weight = keras.Variable(np.array([1.0], dtype="float32"))
sgd = keras.optimizers.SGD(learning_rate=0.1)
sgd.apply([ops.convert_to_tensor(np.array([50.0], dtype="float32"))], [weight])
print("SGD with gradient 50.0: weight moved by", round(1.0 -
float(ops.convert_to_numpy(weight)[0]), 4))
```

```
gradient   0.5: weight moved by 0.1000, formula 0.1000
gradient  50.0: weight moved by 0.1000, formula 0.1000
SGD with gradient 50.0: weight moved by 5.0
```

A gradient of 0.5 and a gradient of 50 move the Adam weight by the same amount: 0.1. SGD moves it by 5.

The invariant is this: Adam steps by a fixed size at the start, and SGD steps in proportion to the gradient. This is why Adam hid the scale problem of Chapter 5. It is also why Adam forgives a bad learning rate more than SGD does.

Reference

```
import keras
from examples.signatures import show

show(keras.losses.Loss, keras.metrics.Metric, ("Metric.add_variable",
keras.metrics.Metric.add_variable),
     ("Model.compute_loss", keras.Model.compute_loss), ("Model.compute_metrics",
keras.Model.compute_metrics),
     ("Optimizer.apply", keras.optimizers.Optimizer.apply))
```

```
Loss(name=None, reduction='sum_over_batch_size', dtype=None)
Metric(dtype=None, name=None)
Metric.add_variable(shape, initializer, dtype=None, aggregation='sum',
    name=None)
Model.compute_loss(x=None, y=None, y_pred=None, sample_weight=None,
    training=True)
Model.compute_metrics(x, y, y_pred, sample_weight=None)
Optimizer.apply(grads, trainable_variables=None)
```

parameter	default	what it does	what goes wrong
`Loss.call(y_true, y_pred)`	—	you write it; return one value for each sample	Keras does the average, so do not average inside
`Loss(reduction)`	`'sum_over_batch_size'`	how the per-sample values combine	—
`Metric.update_state(y_true, y_pred, sample_weight)`	—	you write it; add this batch to the state	state must live in variables made with `add_variable`
`Metric.add_variable(shape, initializer)`	required	makes one state variable	—
`Metric.reset_state()`	—	you write it; clear the state	a no-op gave 0.6929 at epoch 3, against 0.8934
`compute_loss(x, y, y_pred, sample_weight)`	all `None`	the loss that `train_step` uses	override it for a change that works on every back end
`compute_metrics(x, y, y_pred)`	required	updates the compiled metrics	returns the loss as 0.0 if nothing updated the loss tracker
`compute_metrics(sample_weight)`	`None`	weights of the samples	—
`Optimizer.apply(grads, trainable_variables)`	—	one update step	—

What you have now

You have a custom loss, a custom metric and a custom step. Each one equals a built-in reference before it changes anything. The habit is the lesson: check the loop against the built-in first.

Chapter 10 starts the part on architectures, with images. Exercises for this chapter are in exercises/09.md, and the worked solutions are in exercises/solutions/09.md.

Chapter 10 — Convolutional networks

> **Build** — a convolutional network with 5,130 weights that classifies the digits with 99.1% accuracy, and a table that shows how five models react when the test images move by one pixel.
>
> **Theory kept** — a convolution applies the same small filter at every position. Its output moves when the input moves. A pooling step over all positions removes most of the dependence on position.
>
> **Source** — Keras 3.15.1 on TensorFlow 2.21.0 and the digits data of scikit-learn 1.9.1. The code is New.
>
> **Run this first** — `uv run python -m examples.ch10_convolutional_networks`, which is `examples/ch10_convolutional_networks.py`.

A dense layer connects every input to every output. For an image, this is a poor fit: it has one weight for each pair of pixel and unit, and it does not know that two pixels are neighbors. A convolutional layer uses a small filter, 3 × 3 for example, and slides it over the image. The same weights are used at every position.

This chapter uses the 8 × 8 digits of Chapter 5, now as images with one channel. The images are small, but the ideas are the same as for large images. Keras uses the layout `(batch, height, width, channels)`.

Shapes and parameters

Three arguments decide the output shape of a `Conv2D` layer: the kernel size, the padding and the stride. With `padding="valid"` the filter stays inside the image, so the output is smaller. With `padding="same"` the image is padded with zeros, so the output has the same size at stride 1. A stride of 2 halves the size.

The parameters do not depend on the size of the image. A layer with `filters` filters of size `k × k` on `c` input channels has `k × k × c × filters + filters` weights.

```
import keras
from keras import layers

keras.utils.set_random_seed(0)

print(f"{'layer':44s} {'output shape':18s} parameters")
for label, layer in [
    ("Conv2D(16, 3, padding='valid')", layers.Conv2D(16, 3, padding="valid")),
    ("Conv2D(16, 3, padding='same')", layers.Conv2D(16, 3, padding="same")),
    ("Conv2D(16, 3, padding='same', strides=2)", layers.Conv2D(16, 3, padding="same",
strides=2)),
    ("Conv2D(16, 5, padding='valid')", layers.Conv2D(16, 5, padding="valid")),
]:
    layer.build((None, 8, 8, 1))
    shape = tuple(layer.compute_output_shape((None, 8, 8, 1)))
    print(f"{label:44s} {str(shape):18s} {layer.count_params()}")
print("formula for the first row: 3 * 3 * 1 * 16 + 16 =", 3 * 3 * 1 * 16 + 16)
```

```
layer                                     output shape       parameters
Conv2D(16, 3, padding='valid')            (None, 6, 6, 16)   160
Conv2D(16, 3, padding='same')             (None, 8, 8, 16)   160
Conv2D(16, 3, padding='same', strides=2)  (None, 4, 4, 16)   160
Conv2D(16, 5, padding='valid')            (None, 4, 4, 16)   416
formula for the first row: 3 * 3 * 1 * 16 + 16 = 160
```

The `valid` filter of size 3 turns 8 pixels into 6, and the `same` filter keeps 8. The weight count does not change with the padding. A filter of size 5 has 25 weights for each filter instead of 9. The same layer has the same 160 weights on a 1,000 × 1,000 image. A dense layer from 64 inputs to 16 units has 1,040 weights. On a 1,000 × 1,000 image the same dense layer has 16 million.

Convolution moves with the input

The property that matters is *equivariance*: if you move the input, the output moves by the same amount. The next block checks it. It builds one convolutional layer with random weights, shifts the images by one pixel to the right, and compares the two outputs.

```
import keras
import numpy as np
from keras import layers, ops
from examples.digits import load

keras.utils.set_random_seed(0)

_, x_test, _, _ = load(images=True)
empty_edge = (x_test[:, :, -1, :].max(axis=(1, 2)) == 0)
images = x_test[empty_edge]
shifted = np.zeros_like(images)
shifted[:, :, 1:, :] = images[:, :, :-1, :]

layer = layers.Conv2D(4, 3, padding="valid")
before = ops.convert_to_numpy(layer(images))
after = ops.convert_to_numpy(layer(shifted))

print("test images with an empty right edge:", int(empty_edge.sum()), "of", len(x_test))
print("output moved by one column:", bool(np.allclose(after[:, :, 1:, :], before[:, :, :-1,
:], atol=1e-6)))
same_max = np.isclose(after.max(axis=(1, 2)), before.max(axis=(1, 2)), atol=1e-6)
print("share of (image, channel) pairs with the same maximum:", round(float(same_max.mean()),
3))
```

```
test images with an empty right edge: 387 of 450
output moved by one column: True
share of (image, channel) pairs with the same maximum: 0.979
```

The output of the layer on the shifted image is the output on the original image, one column to the right. This holds for every column that both outputs contain. The shifted image has one new window at the left edge, and it loses one window at the right edge. So the output moves, but it is not a pure shift at the edges.

The last line measures the effect. The maximum of a channel over all positions is the same for most of the pairs, and not for all of them. A maximum that sat in the window that left, or in the new window, changes. A global pool makes the answer nearly independent of the position. It does not make it exactly independent.

Five models and one pixel

The next block trains five models for 40 epochs on the centered training images. It then scores each model on three sets of test images: the original images, the images shifted one pixel to the right, and the images shifted one pixel to the left. The shifts fill the free column with zero.

- **dense**: flatten, a layer of 64 units, and the output.
- **cnn flatten**: a convolution, a max pool of size 2, flatten, and a dense layer.
- **cnn global**: two convolutions and a global max pool.
- **dense aug**: the dense model with random translations of the training images.
- **cnn aug**: the `cnn flatten` model with the same random translations.

```
import keras
import numpy as np
from keras import layers
from examples.digits import load

x_train, x_test, y_train, y_test = load(images=True)
loss = keras.losses.SparseCategoricalCrossentropy(from_logits=True)

def shift(x, dx):
    out = np.zeros_like(x)
    if dx > 0:
        out[:, :, dx:, :] = x[:, :, :-dx, :]
    elif dx < 0:
        out[:, :, :dx, :] = x[:, :, -dx:, :]
    else:
        out = x.copy()
    return out

def build(kind):
    keras.utils.set_random_seed(0)
    stack = [keras.Input((8, 8, 1))]
    if kind in ("dense aug", "cnn aug"):
        stack.append(layers.RandomTranslation(0.125, 0.125, fill_mode="constant"))
    if kind in ("dense", "dense aug"):
        stack += [layers.Flatten(), layers.Dense(64, activation="relu"), layers.Dense(10)]
    elif kind in ("cnn flatten", "cnn aug"):
        stack += [layers.Conv2D(16, 3, padding="same", activation="relu"),
layers.MaxPooling2D(2),
                  layers.Flatten(), layers.Dense(64, activation="relu"), layers.Dense(10)]
    else:
        stack += [layers.Conv2D(16, 3, padding="same", activation="relu"),
                  layers.Conv2D(32, 3, padding="same", activation="relu"),
                  layers.GlobalMaxPooling2D(), layers.Dense(10)]
    return keras.Sequential(stack)

print(f"{'model':12s} {'weights':>8s} {'centered':>9s} {'right 1px':>10s} {'left 1px':>9s}")
for kind in ("dense", "cnn flatten", "cnn global", "dense aug", "cnn aug"):
    model = build(kind)
    model.compile("adam", loss, metrics=["accuracy"])
    model.fit(x_train, y_train, epochs=40, batch_size=32, verbose=0)
    scores = [model.evaluate(shift(x_test, d), y_test, verbose=0)[1] for d in (0, 1, -1)]
    print(f"{kind:12s} {model.count_params():8d} {scores[0]:9.3f} {scores[1]:10.3f}
{scores[2]:9.3f}")
```

```
model         weights  centered  right 1px  left 1px
dense            4810     0.978      0.462     0.482
cnn flatten     17258     0.978      0.638     0.562
cnn global       5130     0.991      0.991     0.991
dense aug        4810     0.949      0.853     0.847
cnn aug         17258     0.991      0.951     0.936
```

Read the first column: the dense model and the first CNN both score 0.978 on centered images. They do not differ there.

Now read the last two columns. With a shift of one pixel the dense model falls to 0.462 and 0.482. The `cnn flatten` model falls to 0.638 and 0.562, which is better but not good. It has convolutions, but the flatten step gives each position

its own weights again.

The `cnn global` model scores 0.991 in all three columns. It is the best model on the centered images and it does not change under the shift. It also has 5,130 weights, fewer than the 17,258 of `cnn flatten`. Its global max pool removes the position.

Augmentation is the other way to get robustness. It works for both models and it costs training time. The dense model scores 0.949 on centered images and 0.853 on the right shift. The CNN scores 0.991 and 0.951. The augmented CNN is still below the `cnn global` model on the shifted images. A structure that has the right property beats data that teaches it.

What to take from the table

Three results are worth keeping. First, a convolution is not enough. The flatten step puts back what the filter sharing took out.

Second, a global pooling layer at the end gave the invariance at no cost in weights. Third, the choice of the layer is a statement about what does not change in the data. Use the statement that is true for your task. A task in which position carries the answer, such as reading a number from a fixed form, needs the flatten.

The invariance in the table is also limited. It holds for shifts that leave the digit inside the frame. This is a measured property of 8 × 8 digits with an empty border. It will not carry over to other data without a check.

How it works

A `Conv2D` layer is a sliding dot product. For each output position, it takes a window of the input with the size of the kernel. It multiplies the window by the kernel, adds the products, and adds the bias. With several filters, it does this once for each filter. The kernel has the shape `(height, width, input channels, filters)`.

Keras does not flip the kernel. The operation is a cross-correlation, and the books of signal processing call a flipped version a convolution. For a learned kernel, the difference does not matter.

The next block does the sliding dot product with two loops and compares it with the layer.

```
import keras
import numpy as np
from keras import layers, ops
from examples.digits import load

keras.utils.set_random_seed(0)

_, x_test, _, _ = load(images=True)
image = x_test[:1]
layer = layers.Conv2D(2, 3, padding="valid")
output = ops.convert_to_numpy(layer(image))
kernel, bias = (ops.convert_to_numpy(w) for w in layer.weights)
print("kernel shape:", kernel.shape, " output shape:", output.shape)

by_hand = np.zeros_like(output)
for i in range(output.shape[1]):
    for j in range(output.shape[2]):
        window = image[0, i:i + 3, j:j + 3, :]
        for f in range(output.shape[3]):
            by_hand[0, i, j, f] = (window * kernel[:, :, :, f]).sum() + bias[f]
print("two loops equal the layer:", bool(np.allclose(by_hand, output, atol=1e-5)))
print("multiplications for this image:", output.size * 3 * 3 * 1)
```

```
kernel shape: (3, 3, 1, 2)  output shape: (1, 6, 6, 2)
two loops equal the layer: True
multiplications for this image: 648
```

The layer equals the two loops. The invariant is this: every output position uses the same kernel. The 18 numbers of the kernel are the only weights besides the 2 biases, and the layer makes 36 × 18 = 648 multiplications from them. This is the cause of the shift property in the chapter. A feature moves in the input, and the same kernel finds it at the new place.

Reference

```
from keras import layers
from examples.signatures import show

show(layers.Conv2D, layers.MaxPooling2D, layers.GlobalMaxPooling2D, layers.RandomTranslation)
```

```
Conv2D(filters, kernel_size, strides=(1, 1), padding='valid',
    data_format=None, dilation_rate=(1, 1), groups=1, activation=None,
    use_bias=True, kernel_initializer='glorot_uniform',
    bias_initializer='zeros', kernel_regularizer=None, bias_regularizer=None,
    activity_regularizer=None, kernel_constraint=None, bias_constraint=None,
    **kwargs)
MaxPooling2D(pool_size=(2, 2), strides=None, padding='valid',
    data_format=None, name=None, **kwargs)
GlobalMaxPooling2D(data_format=None, keepdims=False, **kwargs)
RandomTranslation(height_factor, width_factor, fill_mode='reflect',
    interpolation='bilinear', seed=None, fill_value=0.0, data_format=None,
    **kwargs)
```

parameter	default	what it does	what goes wrong
`Conv2D(filters)`	required	number of output channels	—
`Conv2D(kernel_size)`	required	height and width of the window	3 gave 6 from 8 pixels at `valid`, 5 gave 4
`Conv2D(padding)`	`'valid'`	`'valid'` shrinks the output, `'same'` keeps the size at stride 1	`valid` loses the border
`Conv2D(strides)`	`(1, 1)`	step of the window	2 halved the output to 4 × 4
`MaxPooling2D(pool_size)`	`(2, 2)`	window of the maximum	—
`GlobalMaxPooling2D(keepdims)`	`False`	maximum over all positions	2% of the image-channel pairs changed their maximum under a 1-pixel shift
`RandomTranslation(height_factor, width_factor)`	required	shift in a share of the size	0.125 of 8 pixels is 1 pixel
`RandomTranslation(fill_mode)`	`'reflect'`	what fills the free border	the book used `'constant'` so that the free column is zero

What you have now

You have a convolutional network with 5,130 weights that scores 0.991 and that does not change when the input moves by one pixel. You also have the arithmetic of shapes and weights, and a test of equivariance.

Chapter 11 moves from images to sequences. Exercises for this chapter are in exercises/10.md, and the worked solutions are in exercises/solutions/10.md.

Chapter 11 — Sequences

> **Build** — an order test that separates models that see order from models that do not, a measurement of the reach of a convolution filter, and a proof that a mask makes padding invisible.
>
> **Theory kept** — a sequence model reads tokens one after another and keeps a state. A bag of tokens has no order. A convolution sees a window of fixed width.
>
> **Source** — Keras 3.15.1 on TensorFlow 2.21.0. The data are generated by the code. The code is New.
>
> **Run this first** — `uv run python -m examples.ch11_sequences`, which is `examples/ch11_sequences.py`.

Text, sensor traces and call logs are sequences. A model for them must answer one question first: does the order of the tokens carry information? If it does not, a bag of tokens is enough. If it does, the model needs a part that reads order.

This chapter builds a task in which only order matters, so that each model type gets a clean score. The data are made by the code. Nothing is downloaded.

From text to numbers

A model reads integers. The `TextVectorization` layer turns strings into integers. You call `adapt` on a list of strings to build the vocabulary, and then the layer maps every word to its position in the vocabulary.

```
from keras import layers

vectorizer = layers.TextVectorization(max_tokens=20, output_sequence_length=6)
vectorizer.adapt(["the cat sat", "the dog sat on the mat", "a cat"])

print("vocabulary:", [str(word) for word in vectorizer.get_vocabulary()])
print("'the cat sat on a sofa' ->", vectorizer(["the cat sat on a sofa"]).numpy()[0].tolist())
print("'a cat' ->", vectorizer(["a cat"]).numpy()[0].tolist())
```

```
vocabulary: ['', '[UNK]', 'the', 'sat', 'cat', 'on', 'mat', 'dog', 'a']
'the cat sat on a sofa' -> [2, 4, 3, 5, 8, 1]
'a cat' -> [8, 4, 0, 0, 0, 0]
```

Id 0 is the padding value, and id 1 is the unknown word. The word `sofa` is not in the vocabulary, so it becomes 1. The layer sorts the vocabulary by frequency, so `the` has id 2. The output has a fixed length of 6: the short sentence is padded with zeros at the end.

An `Embedding` layer then turns each id into a vector of learned numbers. An embedding with a vocabulary of 10 and a width of 16 is a table of 10 × 16 = 160 weights.

```
import keras
import numpy as np
from keras import layers

keras.utils.set_random_seed(0)

embedding = layers.Embedding(input_dim=10, output_dim=16)
tokens = np.array([[3, 1, 4, 1, 5]])
vectors = embedding(tokens)
print("input shape :", tokens.shape)
print("output shape:", tuple(vectors.shape))
print("weights     :", embedding.count_params())
print("same id, same vector:", bool(np.array_equal(vectors.numpy()[0, 1], vectors.numpy()[0,
3])))
```

```
input shape : (1, 5)
output shape: (1, 5, 16)
weights     : 160
same id, same vector: True
```

The output has one more axis than the input: the width of the embedding. The same id always gives the same vector. The embedding does not know the position of a token. Position has to come from a layer that reads the sequence.

A task in which only order matters

Each sequence has 16 tokens from a vocabulary of 10. Token 1 appears once and token 2 appears once. The other tokens are random from 3 to 9. The label is 1 if token 1 comes before token 2.

A model that ignores order cannot do better than chance. The two tokens appear in every sequence, so any count of tokens is the same for both labels.

```
import keras
import numpy as np
from keras import layers

rng = np.random.default_rng(0)

def make(n, length=16, vocabulary=10):
    x = rng.integers(3, vocabulary, size=(n, length))
    positions = np.array([rng.permutation(length)[:2] for _ in range(n)])
    x[np.arange(n), positions[:, 0]] = 1
    x[np.arange(n), positions[:, 1]] = 2
    return x.astype("int32"), (positions[:, 0] < positions[:, 1]).astype("int32"), positions

x_train, y_train, _ = make(6000)
x_test, y_test, test_positions = make(2000)
print("share of label 1 in the test set:", round(float(y_test.mean()), 3))

def build(kind):
    keras.utils.set_random_seed(0)
    stack = [keras.Input((16,), dtype="int32"), layers.Embedding(10, 16)]
    if kind == "bag":
        stack += [layers.GlobalAveragePooling1D(), layers.Dense(32, activation="relu")]
    elif kind == "flatten":
        stack += [layers.Flatten(), layers.Dense(64, activation="relu")]
    elif kind == "conv":
        stack += [layers.Conv1D(32, 3, activation="relu"), layers.GlobalMaxPooling1D(),
                  layers.Dense(32, activation="relu")]
    elif kind == "simple rnn":
        stack += [layers.SimpleRNN(32)]
    elif kind == "gru":
        stack += [layers.GRU(32)]
    elif kind == "lstm":
        stack += [layers.LSTM(32)]
    stack.append(layers.Dense(1))
    model = keras.Sequential(stack)
    model.compile("adam", keras.losses.BinaryCrossentropy(from_logits=True),
                  metrics=[keras.metrics.BinaryAccuracy(threshold=0.0, name="acc")])
    return model

models = {}
print(f"{'model':11s} {'weights':>8s} {'train acc':>10s} {'test acc':>9s}")
for kind in ("bag", "flatten", "conv", "simple rnn", "gru", "lstm"):
    models[kind] = build(kind)
    history = models[kind].fit(x_train, y_train, epochs=15, batch_size=64,
                               validation_data=(x_test, y_test), verbose=0).history
    print(f"{kind:11s} {models[kind].count_params():8d} {history['acc'][-1]:10.3f}
{history['val_acc'][-1]:9.3f}")
```

```
share of label 1 in the test set: 0.501
model        weights  train acc  test acc
bag              737      0.514     0.502
flatten        16673      1.000     1.000
conv            2817      0.826     0.805
simple rnn      1761      1.000     1.000
gru             4993      1.000     1.000
lstm            6465      1.000     1.000
```

The bag model scores 0.502 on the test set, which is chance. Its training accuracy is also near chance. It cannot represent the answer. The `flatten` model scores 1.000, because the length is fixed and it gives each position its own weights. The three recurrent models also score 1.000.

The `conv` model is in between at 0.805. This is the interesting case. It sees windows of 3 tokens. A filter of width 3 has a reach of 3 tokens, and it has no memory of earlier windows. The next block tests this.

The reach of a filter

The next block measures the claim. It splits the test set by the distance between token 1 and token 2 in the sequence. It then scores the `conv` model on each part, and the `gru` model for comparison.

The block trains the two models again, in the same way as before, so that it runs on its own.

```
import keras
import numpy as np
from keras import layers

rng = np.random.default_rng(0)

def make(n, length=16, vocabulary=10):
    x = rng.integers(3, vocabulary, size=(n, length))
    positions = np.array([rng.permutation(length)[:2] for _ in range(n)])
    x[np.arange(n), positions[:, 0]] = 1
    x[np.arange(n), positions[:, 1]] = 2
    return x.astype("int32"), (positions[:, 0] < positions[:, 1]).astype("int32"), positions

x_train, y_train, _ = make(6000)
x_test, y_test, positions = make(2000)
distance = np.abs(positions[:, 0] - positions[:, 1])

def train(kind):
    keras.utils.set_random_seed(0)
    stack = [keras.Input((16,), dtype="int32"), layers.Embedding(10, 16)]
    if kind == "conv":
        stack += [layers.Conv1D(32, 3, activation="relu"), layers.GlobalMaxPooling1D(),
                  layers.Dense(32, activation="relu")]
    else:
        stack += [layers.GRU(32)]
    model = keras.Sequential(stack + [layers.Dense(1)])
    model.compile("adam", keras.losses.BinaryCrossentropy(from_logits=True))
    model.fit(x_train, y_train, epochs=15, batch_size=64, verbose=0)
    return (model.predict(x_test, verbose=0)[:, 0] > 0).astype("int32")

predictions = {kind: train(kind) for kind in ("conv", "gru")}
print(f"{'distance':>9s} {'pairs':>6s} {'conv':>7s} {'gru':>7s}")
for low, high in [(1, 2), (3, 4), (5, 8), (9, 15)]:
    chosen = (distance >= low) & (distance <= high)
    scores = [float((predictions[k][chosen] == y_test[chosen]).mean()) for k in ("conv",
"gru")]
    print(f"{low:3d} to {high:2d} {int(chosen.sum()):6d} {scores[0]:7.3f} {scores[1]:7.3f}")
```

```
 distance  pairs    conv     gru
  1 to  2    511   1.000   1.000
  3 to  4    402   0.652   1.000
  5 to  8    649   0.710   1.000
  9 to 15    438   0.858   1.000
```

Two tokens at a distance of 1 or 2 always fit in one window of 3. The `conv` model is right for all of these pairs: its accuracy is 1.000. For every larger distance it is far from right, between 0.652 and 0.858. The `gru` model is right at every distance: 1.000.

The accuracy of the `conv` model does not fall steadily with the distance. It is 0.858 for the longest distances, which is higher than at distance 3 or 4. This was not what the text predicted before the measurement.

The block does not show the reason.

One candidate is that a token at the edge of the sequence appears in fewer windows. That gives the model a weak cue of position. Exercise 2 tests that candidate. Pairs with a token at the edge score 1.000. Pairs that lie inside score 0.504, which is chance.

The firm result is the first one. A filter has a reach, and a pair inside the reach is easy. A recurrent layer, or an attention layer (Chapter 12), has no such limit. A convolution needs depth to reach as far.

Padding and masks

Real sequences have different lengths. The usual fix is to pad the short ones with zeros to a common length. Then the padding is part of the input, and a recurrent layer reads it like any token.

The `mask_zero=True` argument of `Embedding` creates a mask. Later layers use the mask and skip the padded steps. The next block gives the same sequence to a GRU, with and without padding. It compares the two outputs.

```
import keras
import numpy as np
from keras import layers, ops

plain = np.array([[3, 4, 5]])
padded = np.array([[3, 4, 5, 0, 0]])

for mask_zero in (False, True):
    keras.utils.set_random_seed(0)
    model = keras.Sequential([
        keras.Input((None,), dtype="int32"),
        layers.Embedding(10, 8, mask_zero=mask_zero),
        layers.GRU(4),
    ])
    difference = np.abs(ops.convert_to_numpy(model(plain)) -
ops.convert_to_numpy(model(padded))).max()
    print(f"mask_zero={mask_zero!s:5s} difference between plain and padded:
{float(difference):.4f}")
```

```
mask_zero=False difference between plain and padded: 0.0458
mask_zero=True  difference between plain and padded: 0.0000
```

Without a mask, the two padding zeros change the output by 0.0458. With a mask, the output is exactly equal. The difference is small here, but it is not zero. A model that you train with padding and serve without padding, or the reverse, gives different answers when no mask is in use.

How it works

An `Embedding` layer is a table of vectors with one row for each id. It does not compute anything. Given an id, it returns that row. This is the same result as a one-hot vector times the table, and the lookup is faster because it skips the zeros.

The next block shows both ways, and it shows that the layer has no way to know the position of a token.

```
import keras
import numpy as np
from keras import layers

keras.utils.set_random_seed(0)

embedding = layers.Embedding(10, 4)
tokens = np.array([[3, 1, 4, 1]])
embedding(tokens)
table = embedding.get_weights()[0]

lookup = table[tokens]
one_hot = np.eye(10)[tokens] @ table
print("table shape:", table.shape, " weights:", embedding.count_params())
print("lookup equals one-hot times table:", bool(np.allclose(lookup, one_hot, atol=1e-6)))
print("same id at two positions gives the same row:", bool(np.array_equal(lookup[0, 1],
lookup[0, 3])))
```

```
table shape: (10, 4)  weights: 40
lookup equals one-hot times table: True
same id at two positions gives the same row: True
```

The result confirms the two statements.

The invariant is this: the output at a position is a function of the id and nothing else. Any order information must come from a layer after the embedding. A recurrent layer gets it from the order of its steps. A convolution gets it from the offsets in its window. Attention gets it from a position vector that you add (Chapter 12).

Reference

```
from keras import layers
from examples.signatures import show

show(layers.Embedding, layers.GRU, layers.Conv1D, layers.TextVectorization)
```

```
Embedding(input_dim, output_dim, embeddings_initializer='uniform',
    embeddings_regularizer=None, embeddings_constraint=None, mask_zero=False,
    weights=None, lora_rank=None, lora_alpha=None, quantization_config=None,
    **kwargs)
GRU(units, activation='tanh', recurrent_activation='sigmoid', use_bias=True,
    kernel_initializer='glorot_uniform', recurrent_initializer='orthogonal',
    bias_initializer='zeros', kernel_regularizer=None,
    recurrent_regularizer=None, bias_regularizer=None,
    activity_regularizer=None, kernel_constraint=None,
    recurrent_constraint=None, bias_constraint=None, dropout=0.0,
    recurrent_dropout=0.0, seed=None, return_sequences=False,
    return_state=False, go_backwards=False, stateful=False, unroll=False,
    reset_after=True, use_cudnn='auto', **kwargs)
Conv1D(filters, kernel_size, strides=1, padding='valid', data_format=None,
    dilation_rate=1, groups=1, activation=None, use_bias=True,
    kernel_initializer='glorot_uniform', bias_initializer='zeros',
    kernel_regularizer=None, bias_regularizer=None, activity_regularizer=None,
    kernel_constraint=None, bias_constraint=None, **kwargs)
TextVectorization(max_tokens=None, standardize='lower_and_strip_punctuation',
    split='whitespace', ngrams=None, output_mode='int',
    output_sequence_length=None, pad_to_max_tokens=False, vocabulary=None,
    idf_weights=None, sparse=False, ragged=False, encoding='utf-8', name=None,
    **kwargs)
```

parameter	default	what it does	what goes wrong
`Embedding(input_dim)`	required	number of ids, which is the vocabulary size	the table has `input_dim × output_dim` weights
`Embedding(output_dim)`	required	width of each vector	—
`Embedding(mask_zero)`	`False`	treat id 0 as padding and pass a mask on	`False` changed a GRU output by 0.0458 when two zeros were added
`GRU(units)`	required	size of the state	weights are `3 × (d × units + units² + 2 × units)`: 4,800 for 16 and 32
`GRU(reset_after)`	`True`	two bias vectors for each gate	the count above needs `True`
`GRU(return_sequences)`	`False`	return the state at every step	—
`Conv1D(kernel_size)`	required	tokens in one window	the reach: 1.000 inside 3 tokens, 0.652 to 0.858 outside
`Conv1D(padding)`	`'valid'`	`'valid'` shrinks the output	—
`TextVectorization(max_tokens)`	`None`	size limit of the vocabulary	id 0 is padding and id 1 is unknown
`TextVectorization(output_sequence_length)`	`None`	pad or cut to this length	the short sentence was padded with zeros at the end

What you have now

You have a task with a known answer that tells apart the models that see order from those that do not. The results are a bag at chance, a convolution that is right only inside its reach, and recurrent layers that are right everywhere. You also have a measured mask.

Chapter 12 replaces the recurrent state with attention. Exercises for this chapter are in exercises/11.md, and the worked solutions are in exercises/solutions/11.md.

Chapter 12 — Attention and transformers

> **Build** — attention written by hand and checked against `keras.ops`, a transformer block as a custom layer, and a measurement that shows the block is blind to order until you add positions.
>
> **Theory kept** — attention is a weighted average. The weights come from a softmax over the similarity of a query and a set of keys. Nothing in it knows the order of the tokens.
>
> **Source** — Keras 3.15.1 on TensorFlow 2.21.0. The data are generated by the code. The code is New.
>
> **Run this first** — `uv run python -m examples.ch12_attention_and_transformers`, which is `examples/ch12_attention_and_transformers.py`.

Chapter 11 ended with a limit: a convolution has a reach, and a recurrent layer must carry its memory one step at a time. Attention removes both limits. Every position can read every other position in one step.

This chapter builds attention from its formula, then uses the Keras layer, then builds a transformer block. The order task of Chapter 11 is the test. It has a known answer, and it needs order.

Attention by hand

For each query, attention computes a score against every key. It divides the scores by the square root of the key width, applies a softmax, and uses the result as weights on the values. In symbols, the output is `softmax(Q Kᵀ / √d) V`.

The next block writes this with NumPy and compares it with `keras.ops.dot_product_attention`. The input has the layout `(batch, time, heads, width)`.

```
import numpy as np
from keras import ops

rng = np.random.default_rng(2)
q = rng.normal(size=(1, 5, 2, 8)).astype("float32")

built_in = ops.convert_to_numpy(ops.dot_product_attention(q, q, q))

scores = np.einsum("bthd,bshd->bhts", q, q) / np.sqrt(q.shape[-1])
weights = np.exp(scores - scores.max(-1, keepdims=True))
weights /= weights.sum(-1, keepdims=True)
by_hand = np.einsum("bhts,bshd->bthd", weights, q)

print("weights shape (batch, heads, query, key):", weights.shape)
print("each row of weights sums to 1:", bool(np.allclose(weights.sum(-1), 1.0)))
print("by hand equals keras.ops below 1e-5:", bool(np.abs(by_hand - built_in).max() < 1e-5))
```

```
weights shape (batch, heads, query, key): (1, 2, 5, 5)
each row of weights sums to 1: True
by hand equals keras.ops below 1e-5: True
```

The weight array has one row for each query and one column for each key. Each row sums to 1, so the output is a weighted average of the values. The hand-written version and the built-in version agree.

The Keras layer

`keras.layers.MultiHeadAttention` adds learned projections for the query, the key and the value, runs several attention heads, and projects the result back. With `return_attention_scores=True` it also returns the weights.

The next block checks three properties of the layer: the shape of the weights, equivariance under a permutation of the input, and the effect of a causal mask.

```
import keras
import numpy as np
from keras import layers, ops

keras.utils.set_random_seed(0)

attention = layers.MultiHeadAttention(num_heads=2, key_dim=8)
x = np.random.default_rng(1).normal(size=(1, 6, 16)).astype("float32")

output, scores = attention(x, x, return_attention_scores=True)
print("output shape:", tuple(output.shape), " weights shape:", tuple(scores.shape))
print("rows sum to 1:", bool(np.allclose(ops.convert_to_numpy(scores).sum(-1), 1.0, atol=1e-
5)))

order = np.array([3, 0, 5, 1, 4, 2])
permuted = ops.convert_to_numpy(attention(x[:, order], x[:, order]))
original = ops.convert_to_numpy(attention(x, x))
print("permuted input gives permuted output:", bool(np.abs(permuted - original[:,
order]).max() < 1e-5))

changed = x.copy()
changed[:, 4] += 1.0
open_before = ops.convert_to_numpy(attention(x, x))
open_after = ops.convert_to_numpy(attention(changed, changed))
causal_before = ops.convert_to_numpy(attention(x, x, use_causal_mask=True))
causal_after = ops.convert_to_numpy(attention(changed, changed, use_causal_mask=True))
print("change token 4, outputs at positions 0-3:")
print("  without mask moved by", round(float(np.abs(open_after[:, :4] - open_before[:,
:4]).max()), 3))
print("  with causal mask moved by", float(np.abs(causal_after[:, :4] - causal_before[:,
:4]).max()))
```

```
output shape: (1, 6, 16)  weights shape: (1, 2, 6, 6)
rows sum to 1: True
permuted input gives permuted output: True
change token 4, outputs at positions 0-3:
  without mask moved by 0.513
  with causal mask moved by 0.0
```

The weights have the shape `(batch, heads, query, key)`: 2 heads and a 6 × 6 table. A permutation of the input gives the same permutation of the output. This is the important fact: **attention has no sense of order.** A sequence and

its shuffled copy give the same set of outputs.

The causal mask limits each position to the positions before it and itself. With the mask, a change at token 4 does not move the outputs at positions 0 to 3. Without the mask it moves them. A model that predicts the next token needs the mask. A model that classifies a whole sequence does not.

A transformer block

A transformer block has two parts, each with a residual connection and a layer normalization. The first part is attention. The second is a small dense network that is applied to each position alone. The block below is a `keras.layers.Layer` with these parts.

```
import keras
from keras import layers

keras.utils.set_random_seed(0)

class TransformerBlock(layers.Layer):
    def __init__(self, width, heads, hidden, **kwargs):
        super().__init__(**kwargs)
        self.attention = layers.MultiHeadAttention(num_heads=heads, key_dim=width // heads)
        self.norm_1 = layers.LayerNormalization()
        self.feed_forward = keras.Sequential([layers.Dense(hidden, activation="relu"),
layers.Dense(width)])
        self.norm_2 = layers.LayerNormalization()

    def call(self, x):
        x = self.norm_1(x + self.attention(x, x))
        return self.norm_2(x + self.feed_forward(x))
```

The residual connection `x + attention(x, x)` lets the gradient pass around the attention. The layer normalization keeps the scale of each position stable.

The test: order

The next block trains the same transformer three times on the order task of Chapter 11. The first model has no position information. The second adds a position embedding that is called outside a layer, in the way that many examples write it. The third adds the position embedding inside a layer, so that Keras tracks its weights.

```
import keras
import numpy as np
from keras import layers, ops

rng = np.random.default_rng(0)

def make(n, length=16, vocabulary=10):
    x = rng.integers(3, vocabulary, size=(n, length))
    positions = np.array([rng.permutation(length)[:2] for _ in range(n)])
    x[np.arange(n), positions[:, 0]] = 1
    x[np.arange(n), positions[:, 1]] = 2
    return x.astype("int32"), (positions[:, 0] < positions[:, 1]).astype("int32")

x_train, y_train = make(6000)
x_test, y_test = make(2000)

class TransformerBlock(layers.Layer):
    def __init__(self, width, heads, hidden, **kwargs):
        super().__init__(**kwargs)
        self.attention = layers.MultiHeadAttention(num_heads=heads, key_dim=width // heads)
        self.norm_1 = layers.LayerNormalization()
        self.feed_forward = keras.Sequential([layers.Dense(hidden, activation="relu"),
layers.Dense(width)])
        self.norm_2 = layers.LayerNormalization()

    def call(self, x):
        x = self.norm_1(x + self.attention(x, x))
        return self.norm_2(x + self.feed_forward(x))

class AddPositions(layers.Layer):
    def __init__(self, length, width, **kwargs):
        super().__init__(**kwargs)
        self.table = layers.Embedding(length, width)

    def call(self, x):
        return x + self.table(ops.arange(ops.shape(x)[1]))

def build(positions, width=32):
    keras.utils.set_random_seed(0)
    tokens = keras.Input((16,), dtype="int32")
    x = layers.Embedding(10, width)(tokens)
    if positions == "outside a layer":
        x = x + layers.Embedding(16, width)(ops.arange(16))
    elif positions == "inside a layer":
        x = AddPositions(16, width)(x)
    x = TransformerBlock(width, heads=2, hidden=64)(x)
    x = layers.GlobalAveragePooling1D()(x)
    model = keras.Model(tokens, layers.Dense(1)(x))
    model.compile("adam", keras.losses.BinaryCrossentropy(from_logits=True),
                  metrics=[keras.metrics.BinaryAccuracy(threshold=0.0, name="acc")])
    return model

print(f"{'positions':16s} {'weights':>8s} {'train acc':>10s} {'test acc':>9s}")
for positions in ("none", "outside a layer", "inside a layer"):
    model = build(positions)
    h = model.fit(x_train, y_train, epochs=30, batch_size=64,
```

```
                  validation_data=(x_test, y_test), verbose=0).history
    print(f"{positions:16s} {model.count_params():8d} {h['acc'][-1]:10.3f} {h['val_acc']
[-1]:9.3f}")
```

```
positions         weights  train acc  test acc
none                 8897      0.510     0.502
outside a layer      8897      1.000     1.000
inside a layer       9409      1.000     1.000
```

Without positions the model scores 0.502, which is chance, and its training accuracy is also near chance. Both models with positions score 1.000.

The weights column shows a fault. The model with the embedding outside a layer has 8,897 weights, the same as the model with no positions. Keras did not register the embedding, because it was called on a plain tensor and not on a symbolic input. The 512 weights are a fixed random table that no gradient reaches. The model still scores 1.000, because any fixed and distinct code for each position carries order.

The model with `AddPositions` has 9,409 weights. The 512 extra weights are 16 × 32, one vector of 32 numbers for each of the 16 positions, and they are trained. A model that depends on a trained table must have the table inside a layer. Check `count_params()` after you add a component. The number tells you whether Keras sees it.

The task is the one on which the bag-of-tokens model scored 0.502 in Chapter 11. A transformer without positions is, for this purpose, a bag of tokens with a more complex mixing step.

What to take from this chapter

1. Attention is a weighted average of the values, with weights from a softmax. It is exact to write by hand.
2. It has no sense of order. Add a position signal, or the model sees a set.
3. A causal mask is a separate choice. Use it when each position must not see the future.
4. The block is a layer. Stack it, and the cost of each block is its attention weights and its feed-forward weights.

How it works

`MultiHeadAttention` has four groups of weights. The query, the key and the value each have a projection that maps the input of width `d` to `heads × key_dim` numbers. The output projection maps the heads back to width `d`. The attention step itself, `softmax(Q Kᵀ / √d) V`, has no weights.

This explains the weight count and the behavior of the layer. The next block lists the weights of one layer and counts them by hand.

```
import keras
import numpy as np
from keras import layers

keras.utils.set_random_seed(0)

layer = layers.MultiHeadAttention(num_heads=2, key_dim=8)
layer(np.zeros((1, 6, 16), dtype="float32"), np.zeros((1, 6, 16), dtype="float32"))
for weight in layer.weights:
    print(f"{weight.path:30s} {tuple(weight.shape)}")

d, heads, key = 16, 2, 8
by_hand = 3 * (d * heads * key + heads * key) + (heads * key * d + d)
print("by hand:", by_hand, " layer:", layer.count_params())
layer(np.zeros((1, 10, 16), dtype="float32"), np.zeros((1, 10, 16), dtype="float32"))
print("after a call with 10 tokens instead of 6:", layer.count_params())
```

```
multi_head_attention/query/kernel (16, 2, 8)
multi_head_attention/query/bias (2, 8)
multi_head_attention/key/kernel (16, 2, 8)
multi_head_attention/key/bias  (2, 8)
multi_head_attention/value/kernel (16, 2, 8)
multi_head_attention/value/bias (2, 8)
multi_head_attention/attention_output/kernel (2, 8, 16)
multi_head_attention/attention_output/bias (16,)
by hand: 1088  layer: 1088
after a call with 10 tokens instead of 6: 1088
```

The count matches. The number of weights does not change when the length of the sequence changes from 6 to 10. The invariant is this: the layer is the same function at every position, and only the table of scores $Q\,K^{\intercal}$ grows with the length. That table has $T \times T$ entries for a sequence of T tokens. The cost of attention grows with the square of the length, and the cost of a convolution or a recurrent layer grows with the length.

Reference

```
from keras import layers, ops
from examples.signatures import show

show(layers.MultiHeadAttention, ("MultiHeadAttention.call", layers.MultiHeadAttention.call),
     layers.LayerNormalization, ("ops.dot_product_attention", ops.dot_product_attention))
```

```
MultiHeadAttention(num_heads, key_dim, value_dim=None, dropout=0.0,
    use_bias=True, output_shape=None, attention_axes=None,
    sliding_window=None, flash_attention=None,
    kernel_initializer='glorot_uniform', bias_initializer='zeros',
    kernel_regularizer=None, bias_regularizer=None, activity_regularizer=None,
    kernel_constraint=None, bias_constraint=None, use_gate=False, seed=None,
    **kwargs)
MultiHeadAttention.call(query, value, key=None, query_mask=None,
    value_mask=None, key_mask=None, attention_mask=None,
    return_attention_scores=False, training=None, use_causal_mask=False)
LayerNormalization(axis=-1, epsilon=0.001, center=True, scale=True,
    beta_initializer='zeros', gamma_initializer='ones', beta_regularizer=None,
    gamma_regularizer=None, beta_constraint=None, gamma_constraint=None,
    **kwargs)
ops.dot_product_attention(query, key, value, bias=None, mask=None, scale=None,
    is_causal=False, flash_attention=None, attn_logits_soft_cap=None)
```

parameter	default	what it does	what goes wrong
`MultiHeadAttention(num_heads)`	required	number of attention heads	—
`MultiHeadAttention(key_dim)`	required	width of each head for the query and the key	the block used `width // heads`, so the heads add up to the width
`call(query, value)`	required	self-attention passes the same tensor twice	—
`call(use_causal_mask)`	`False`	each position sees only itself and the past	`False` moved earlier outputs by 0.513; `True` moved them by 0.0
`call(return_attention_scores)`	`False`	also return the weights `(batch, heads, query, key)`	—
`LayerNormalization(axis)`	`-1`	the axis that is normalized	—
`LayerNormalization(epsilon)`	`0.001`	small number added to the variance	—
`ops.dot_product_attention(is_causal)`	`False`	the causal mask for the built-in function	—

The layer has no argument for the position of a token. A model that needs order must add a position vector before the layer, as `AddPositions` does.

What you have now

You have attention from the formula, a `TransformerBlock` layer, and a test that shows what positions add. The measurement is small, but the rule is general.

Chapter 13 reuses trained layers on a new task. Exercises for this chapter are in exercises/12.md, and the worked solutions are in exercises/solutions/12.md.

Chapter 13 — Transfer learning

> **Build** — a base network that was trained on clean digits and reused on noisy digits with 2, 5 and 20 labeled images for each class, in three modes: from scratch, with a frozen base and with a fine-tuned base. Three training budgets show that the budget decides the sign of the comparison.
>
> **Theory kept** — the first layers of a trained network are a feature extractor. A new head can learn on top of them with few labels. Freezing keeps the features. Fine-tuning moves them.
>
> **Source** — Keras 3.15.1 on TensorFlow 2.21.0 and the digits data of scikit-learn 1.9.1. The code is New.
>
> **Run this first** — `uv run python -m examples.ch13_transfer_learning`, which is `examples/ch13_transfer_learning.py`.

Labels are expensive. A network that learned on a large labeled set can pass what it learned to a task with few labels. This chapter measures how much it passes, and how that depends on one choice that is easy to get wrong.

The setup

The base task is the digit classification of the earlier chapters. The new task is the same ten classes, with noise added to every pixel. The noise has a standard deviation of 0.35 on pixels in the range 0 to 1. A model that was trained on clean images scores lower on noisy ones. The target set has 2, 5 or 20 labeled noisy images for each class. The test set is all 450 noisy test images.

```
import keras
import numpy as np
from keras import layers
from examples.digits import load

x_train, x_test, y_train, y_test = load(images=True)
noise = np.random.default_rng(0)

def add_noise(x, deviation=0.35):
    return np.clip(x + noise.normal(0, deviation, x.shape), 0, 1).astype("float32")

x_train_noisy, x_test_noisy = add_noise(x_train), add_noise(x_test)

keras.utils.set_random_seed(0)
base = keras.Sequential([
    keras.Input((8, 8, 1)),
    layers.Conv2D(16, 3, padding="same", activation="relu"),
    layers.Conv2D(32, 3, padding="same", activation="relu"),
    layers.GlobalMaxPooling2D(),
], name="base")
pretrained = keras.Sequential([base, layers.Dense(10)])
pretrained.compile("adam", keras.losses.SparseCategoricalCrossentropy(from_logits=True),
metrics=["accuracy"])
pretrained.fit(x_train, y_train, epochs=30, batch_size=32, verbose=0)

print("base output shape:", base.output_shape, " base weights:", base.count_params())
print("trained on clean, test clean:", round(pretrained.evaluate(x_test, y_test, verbose=0)
[1], 3))
print("trained on clean, test noisy:", round(pretrained.evaluate(x_test_noisy, y_test,
verbose=0)[1], 3))
```

```
base output shape: (None, 32)  base weights: 4800
trained on clean, test clean: 0.991
trained on clean, test noisy: 0.673
```

The base turns each image into 32 numbers. The model reaches 0.991 on clean test images and falls on noisy ones. The question is whether the 32-number features help a new model that has only a few noisy labels.

Three modes

The code below defines one function that trains a new head in each of the three modes. It runs the function for several sizes of the target set.

- **scratch**: a new network with the same layers and random weights.
- **frozen**: a copy of the base with `trainable = False`, and a new `Dense(10)` head.
- **fine-tune**: a copy of the base with all weights trainable, and a new head.

The copy is made with `keras.models.clone_model` and `set_weights`, so that the original base is not changed. The model is compiled after `trainable` is set, as Chapter 3 requires.

```
import keras
import numpy as np
from keras import layers
from examples.digits import load

x_train, x_test, y_train, y_test = load(images=True)
noise = np.random.default_rng(0)

def add_noise(x, deviation=0.35):
    return np.clip(x + noise.normal(0, deviation, x.shape), 0, 1).astype("float32")

x_train_noisy, x_test_noisy = add_noise(x_train), add_noise(x_test)
loss = keras.losses.SparseCategoricalCrossentropy(from_logits=True)

def convolutions():
    return [layers.Conv2D(16, 3, padding="same", activation="relu"),
            layers.Conv2D(32, 3, padding="same", activation="relu"),
            layers.GlobalMaxPooling2D()]

keras.utils.set_random_seed(0)
base = keras.Sequential([keras.Input((8, 8, 1))] + convolutions(), name="base")
pretrained = keras.Sequential([base, layers.Dense(10)])
pretrained.compile("adam", loss, metrics=["accuracy"])
pretrained.fit(x_train, y_train, epochs=30, batch_size=32, verbose=0)

def new_model(mode):
    if mode == "scratch":
        body = keras.Sequential([keras.Input((8, 8, 1))] + convolutions())
    else:
        body = keras.models.clone_model(base)
        body.set_weights(base.get_weights())
        body.trainable = mode == "fine-tune"
    return keras.Sequential([body, layers.Dense(10)])

def accuracy(per_class, mode, learning_rate, epochs, seed):
    rows = np.concatenate([
        np.random.default_rng(seed).choice(np.where(y_train == c)[0], per_class,
replace=False)
        for c in range(10)])
    keras.utils.set_random_seed(seed)
    model = new_model(mode)
    model.compile(keras.optimizers.Adam(learning_rate), loss, metrics=["accuracy"])
    model.fit(x_train_noisy[rows], y_train[rows], epochs=epochs, batch_size=16, verbose=0)
    return model.evaluate(x_test_noisy, y_test, verbose=0)[1]

def table(learning_rate, epochs):
    print(f"learning rate {learning_rate}, {epochs} epochs; mean of 2 seeds")
    print(f"{'images per class':>17s} {'scratch':>8s} {'frozen':>8s} {'fine-tune':>10s}")
    for per_class in (2, 5, 20):
        scores = [np.mean([accuracy(per_class, mode, learning_rate, epochs, seed) for seed in
(1, 2)])
                  for mode in ("scratch", "frozen", "fine-tune")]
        print(f"{per_class:17d} {scores[0]:8.3f} {scores[1]:8.3f} {scores[2]:10.3f}")

table(0.001, 40)
```

```
# output varies
learning rate 0.001, 40 epochs; mean of 2 seeds
 images per class  scratch   frozen  fine-tune
                2    0.131    0.070      0.102
                5    0.157    0.114      0.156
               20    0.592    0.291      0.528
```

The tables of this chapter are marked as output that varies. Between two runs of the same code, a cell moved by one image of 450. In a mean of 2 seeds that is 0.001 to 0.002. Read the third decimal with that in mind.

Read the table by rows. The result is not what transfer learning promises. At this budget, the frozen base does not beat the new network that starts from scratch. At 20 images for each class the frozen head is far below the other two. If you stopped here, you will conclude that transfer does not help. The next two sections show that this conclusion depends on the budget.

A different budget reverses the picture

The first table has a hypothesis behind it that you can test. The frozen head is a single `Dense` layer on 32 inputs. With 200 images and a batch size of 16, one epoch is 13 steps. Then 40 epochs is 520 steps at a rate of 0.001. The hypothesis is that this is too little for the head. The next block gives the same three modes a learning rate of 0.01 and 100 epochs.

```
import keras
import numpy as np
from keras import layers
from examples.digits import load

x_train, x_test, y_train, y_test = load(images=True)
noise = np.random.default_rng(0)

def add_noise(x, deviation=0.35):
    return np.clip(x + noise.normal(0, deviation, x.shape), 0, 1).astype("float32")

x_train_noisy, x_test_noisy = add_noise(x_train), add_noise(x_test)
loss = keras.losses.SparseCategoricalCrossentropy(from_logits=True)

def convolutions():
    return [layers.Conv2D(16, 3, padding="same", activation="relu"),
            layers.Conv2D(32, 3, padding="same", activation="relu"),
            layers.GlobalMaxPooling2D()]

keras.utils.set_random_seed(0)
base = keras.Sequential([keras.Input((8, 8, 1))] + convolutions(), name="base")
pretrained = keras.Sequential([base, layers.Dense(10)])
pretrained.compile("adam", loss, metrics=["accuracy"])
pretrained.fit(x_train, y_train, epochs=30, batch_size=32, verbose=0)

def new_model(mode):
    if mode == "scratch":
        body = keras.Sequential([keras.Input((8, 8, 1))] + convolutions())
    else:
        body = keras.models.clone_model(base)
        body.set_weights(base.get_weights())
        body.trainable = mode == "fine-tune"
    return keras.Sequential([body, layers.Dense(10)])

def accuracy(per_class, mode, learning_rate, epochs, seed):
    rows = np.concatenate([
        np.random.default_rng(seed).choice(np.where(y_train == c)[0], per_class,
replace=False)
        for c in range(10)])
    keras.utils.set_random_seed(seed)
    model = new_model(mode)
    model.compile(keras.optimizers.Adam(learning_rate), loss, metrics=["accuracy"])
    model.fit(x_train_noisy[rows], y_train[rows], epochs=epochs, batch_size=16, verbose=0)
    return model.evaluate(x_test_noisy, y_test, verbose=0)[1]

print("learning rate 0.01, 100 epochs; mean of 2 seeds")
print(f"{'images per class':>17s} {'scratch':>8s} {'frozen':>8s} {'fine-tune':>10s}")
for per_class in (2, 5, 20):
    scores = [np.mean([accuracy(per_class, mode, 0.01, 100, seed) for seed in (1, 2)])
              for mode in ("scratch", "frozen", "fine-tune")]
    print(f"{per_class:17d} {scores[0]:8.3f} {scores[1]:8.3f} {scores[2]:10.3f}")
```

```
# output varies
learning rate 0.01, 100 epochs; mean of 2 seeds
 images per class  scratch   frozen  fine-tune
                2    0.219    0.419      0.324
                5    0.378    0.568      0.469
               20    0.592    0.724      0.653
```

At this budget the picture is the opposite. The frozen base is ahead of the new network in every row: by about 0.20 at 2 images for each class, by about 0.19 at 5 and by about 0.13 at 20. Fine-tuning is in between.

The first table did not measure transfer. It measured a head that had not finished training. The mistake is easy to make, because nothing fails: the loss falls, the run ends, and the number is low. A frozen base needs a budget for the head. The first test of a negative result is to ask whether the new part converged.

Every mode depends on its budget

The second table raises a fair question. If the head needed a larger budget, did the new network need one too? The next block runs a third budget, a learning rate of 0.003 and 60 epochs, for all three modes.

```
import keras
import numpy as np
from keras import layers
from examples.digits import load

x_train, x_test, y_train, y_test = load(images=True)
noise = np.random.default_rng(0)

def add_noise(x, deviation=0.35):
    return np.clip(x + noise.normal(0, deviation, x.shape), 0, 1).astype("float32")

x_train_noisy, x_test_noisy = add_noise(x_train), add_noise(x_test)
loss = keras.losses.SparseCategoricalCrossentropy(from_logits=True)

def convolutions():
    return [layers.Conv2D(16, 3, padding="same", activation="relu"),
            layers.Conv2D(32, 3, padding="same", activation="relu"),
            layers.GlobalMaxPooling2D()]

keras.utils.set_random_seed(0)
base = keras.Sequential([keras.Input((8, 8, 1))] + convolutions(), name="base")
pretrained = keras.Sequential([base, layers.Dense(10)])
pretrained.compile("adam", loss, metrics=["accuracy"])
pretrained.fit(x_train, y_train, epochs=30, batch_size=32, verbose=0)

def new_model(mode):
    if mode == "scratch":
        body = keras.Sequential([keras.Input((8, 8, 1))] + convolutions())
    else:
        body = keras.models.clone_model(base)
        body.set_weights(base.get_weights())
        body.trainable = mode == "fine-tune"
    return keras.Sequential([body, layers.Dense(10)])

def accuracy(per_class, mode, learning_rate, epochs, seed):
    rows = np.concatenate([
        np.random.default_rng(seed).choice(np.where(y_train == c)[0], per_class,
replace=False)
        for c in range(10)])
    keras.utils.set_random_seed(seed)
    model = new_model(mode)
    model.compile(keras.optimizers.Adam(learning_rate), loss, metrics=["accuracy"])
    model.fit(x_train_noisy[rows], y_train[rows], epochs=epochs, batch_size=16, verbose=0)
    return model.evaluate(x_test_noisy, y_test, verbose=0)[1]

print("learning rate 0.003, 60 epochs; mean of 2 seeds")
print(f"{'images per class':>17s} {'scratch':>8s} {'frozen':>8s} {'fine-tune':>10s}")
for per_class in (2, 5, 20):
    scores = [np.mean([accuracy(per_class, mode, 0.003, 60, seed) for seed in (1, 2)])
              for mode in ("scratch", "frozen", "fine-tune")]
    print(f"{per_class:17d} {scores[0]:8.3f} {scores[1]:8.3f} {scores[2]:10.3f}")
```

```
# output varies
learning rate 0.003, 60 epochs; mean of 2 seeds
 images per class  scratch   frozen  fine-tune
                2    0.244    0.166      0.229
                5    0.358    0.294      0.386
               20    0.652    0.653      0.653
```

The third budget gives a third picture. At 20 images for each class the three modes tie, at 0.652 to 0.653. At 2 images the new network is ahead, with 0.244 against 0.166. At 5 images fine-tuning is ahead, with 0.386. The sign of the comparison changes with the budget, inside one table and between tables.

A claim that transfer helps, or that it does not help, needs the budget stated, and it needs a search over budgets for every mode. Take the best entry of each mode at 20 images. Scratch reaches 0.652, the frozen base 0.724 and fine-tuning 0.653. The frozen base is highest. It is ahead of the next best entry, 0.653, by about 0.07, which is about 30 images of 450.

Two limits apply to that number. We chose the budgets by looking at the test accuracy, so each best entry is optimistic. Each row is the mean of 2 seeds. The chapter did not measure the spread over seeds, so it cannot say that 0.07 is outside it. Read it as a lead and not as a result.

How it works

Freezing does two things. It moves the weights of the base to the non-trainable list, so that the optimizer skips them. It also stops the base from changing in any other way: there are no batch statistics in this base, so its output for an input is a fixed function.

The next block measures both. It counts the trainable weights in each mode. Then it checks the base weights after a short training. They must not change in frozen mode, and they must change in fine-tune mode.

```
import keras
import numpy as np
from keras import layers

def convolutions():
    return [layers.Conv2D(16, 3, padding="same", activation="relu"),
            layers.Conv2D(32, 3, padding="same", activation="relu"),
            layers.GlobalMaxPooling2D()]

keras.utils.set_random_seed(0)
base = keras.Sequential([keras.Input((8, 8, 1))] + convolutions())
x = np.random.default_rng(0).random((32, 8, 8, 1)).astype("float32")
y = np.random.default_rng(1).integers(0, 10, size=(32,))

for mode in ("frozen", "fine-tune"):
    body = keras.models.clone_model(base)
    body.set_weights(base.get_weights())
    body.trainable = mode == "fine-tune"
    model = keras.Sequential([body, layers.Dense(10)])
    model.compile(keras.optimizers.Adam(0.01),
keras.losses.SparseCategoricalCrossentropy(from_logits=True))
    before = [w.copy() for w in body.get_weights()]
    trainable = sum(int(np.prod(w.shape)) for w in model.trainable_weights)
    model.fit(x, y, epochs=3, batch_size=16, verbose=0)
    moved = any(not np.array_equal(a, b) for a, b in zip(before, body.get_weights()))
    print(f"{mode:9s} trainable weights {trainable:5d} of {model.count_params()}, base weights
changed: {moved}")
```

```
frozen    trainable weights   330 of 5130, base weights changed: False
fine-tune trainable weights  5130 of 5130, base weights changed: True
```

In frozen mode, only the head trains: 330 weights of 5,130. The base is a fixed function. In fine-tune mode, all of them train.

The invariant is this: a frozen base gives the same 32 numbers for the same image in every epoch. The head then solves a small problem on 32 inputs. A small problem needs fewer examples.

This is the reason that transfer can work with few labels. It is also the reason that its result depends on how well the head is trained.

Reference

```
import keras
from examples.signatures import show

show(("models.clone_model", keras.models.clone_model),
     ("Model.get_weights", keras.Model.get_weights),
     ("Model.set_weights", keras.Model.set_weights),
     ("optimizers.Adam", keras.optimizers.Adam.__init__))
```

```
models.clone_model(model, input_tensors=None, clone_function=None,
    call_function=None, recursive=False, **kwargs)
Model.get_weights()
Model.set_weights(weights)
optimizers.Adam(learning_rate=0.001, beta_1=0.9, beta_2=0.999, epsilon=1e-07,
    amsgrad=False, weight_decay=None, clipnorm=None, clipvalue=None,
    global_clipnorm=None, use_ema=False, ema_momentum=0.99,
    ema_overwrite_frequency=None, loss_scale_factor=None,
    gradient_accumulation_steps=None, name=None, **kwargs)
```

parameter	default	what it does	what goes wrong
`clone_model(model)`	required	new model with the same architecture and new random weights	the clone has new weights; copy them with `set_weights`
`model.trainable`	`True`	all weights of the model train, or none	change it before `compile`, or the change is ignored (Chapter 3)
`set_weights(weights)`	required	assign a list of arrays, in layer order	a list with the wrong shapes raises an error
`Adam(learning_rate)`	`0.001`	step size	a frozen head with 0.001 and 40 epochs had not converged

What you have now

You have a measured case in which transfer learning wins, a measured case in which it seems to lose, and a test for the difference. Each mode needs a budget that suits it, and a result that is negative is a reason to check the budget before you conclude.

Chapter 14 saves what you built. Exercises for this chapter are in exercises/13.md, and the worked solutions are in exercises/solutions/13.md.

Chapter 14 — Saving, loading and export

> **Build** — a round trip that gives back the same predictions bit for bit and the same training, a list of four ways that a load fails or silently changes a model, and an exported artifact that a service can load without Keras.
>
> **Theory kept** — a saved model is a configuration plus a list of arrays. The configuration says how to rebuild the objects. The arrays fill them.
>
> **Source** — Keras 3.15.1 on TensorFlow 2.21.0. The code is New.
>
> **Run this first** — `uv run python -m examples.ch14_saving_loading_export`, which is `examples/ch14_saving_loading_export.py`.

A trained model that you cannot load again is lost work. Saving looks like one call, and it is one call when the model uses only built-in parts. It becomes a source of silent faults when the model has custom parts. This chapter measures each case.

What a `.keras` file holds

`model.save("name.keras")` writes a zip archive. The next block opens it and lists what is inside. It also prints how the weights are keyed.

```
import io
import json
import os
import tempfile
import zipfile

import h5py
import keras
import numpy as np
from keras import layers

keras.utils.set_random_seed(0)
model = keras.Sequential([
    keras.Input((8,)),
    layers.Dense(4, activation="relu", name="hidden"),
    layers.Dense(2, name="out"),
])
model.compile("adam", "mse")
model.fit(np.zeros((8, 8), dtype="float32"), np.zeros((8, 2), dtype="float32"), verbose=0)

path = os.path.join(tempfile.mkdtemp(), "model.keras")
model.save(path)
archive = zipfile.ZipFile(path)
print("files:", archive.namelist())

config = json.loads(archive.read("config.json"))
print("config keys:", sorted(config))
print("layer names in the config:", [layer["config"]["name"] for layer in config["config"]
["layers"]])

weights = h5py.File(io.BytesIO(archive.read("model.weights.h5")))
print("weight groups:", sorted(weights["layers"]))
print("optimizer arrays:", len(weights["optimizer"]["vars"]))
```

```
files: ['metadata.json', 'config.json', 'model.weights.h5']
config keys: ['build_config', 'class_name', 'compile_config', 'config', 'module',
'registered_name']
layer names in the config: ['input_layer', 'hidden', 'out']
weight groups: ['dense', 'dense_1']
optimizer arrays: 10
```

The archive has three files: a small `metadata.json`, a `config.json` that rebuilds the objects, and a `model.weights.h5` with the arrays. The optimizer state is in the weights file too. This is why a loaded model can continue training from where it stopped.

One detail is easy to miss. In the config, the layers are named `hidden` and `out`. In the weights file, the groups are named `dense` and `dense_1`. The file keys the arrays by the class of the layer and its order, and not by the name that you gave.

How it works

Loading is two steps. First Keras reads `config.json` and builds each object from its `class_name` and `config`. A class that Keras does not know must be found in a registry or in the `custom_objects` that you pass. Then Keras builds the weights and assigns the arrays, in order, to the objects that it made.

The invariant is this: the config must rebuild an object that has the same weights as the saved one, in the same order. Every fault in this chapter breaks one half of this sentence. A class that cannot be found breaks the first step. An argument that is not in the config gives an object with a different shape. Then the second step fails, or it fills the object with the wrong arrays.

A round trip is exact

The next block saves a trained model, loads it, and compares. Then it trains both the original and the loaded model for one more epoch on the same batches.

```
import os
import tempfile

import keras
import numpy as np
from keras import layers
from examples.digits import load

x_train, x_test, y_train, y_test = load()
keras.utils.set_random_seed(0)
model = keras.Sequential([keras.Input((64,)), layers.Dense(32, activation="relu"),
layers.Dense(10)])
model.compile("adam", keras.losses.SparseCategoricalCrossentropy(from_logits=True), metrics=
["accuracy"])
model.fit(x_train, y_train, epochs=2, batch_size=32, verbose=0)

path = os.path.join(tempfile.mkdtemp(), "model.keras")
model.save(path)
loaded = keras.saving.load_model(path)

print("predictions equal:", np.array_equal(model.predict(x_test, verbose=0),
loaded.predict(x_test, verbose=0)))
print("loaded model is compiled:", loaded.compiled)
print("optimizer iterations:", int(model.optimizer.iterations.numpy()),
int(loaded.optimizer.iterations.numpy()))

model.fit(x_train, y_train, epochs=1, batch_size=32, shuffle=False, verbose=0)
loaded.fit(x_train, y_train, epochs=1, batch_size=32, shuffle=False, verbose=0)
same = all(np.array_equal(a, b) for a, b in zip(model.get_weights(), loaded.get_weights()))
print("weights equal after one more epoch:", same)
```

```
predictions equal: True
loaded model is compiled: True
optimizer iterations: 86 86
weights equal after one more epoch: True
```

The predictions are bit for bit equal. The loaded model is compiled, and its optimizer has the same step count. After one more epoch of the same batches, the weights are still equal. The file holds everything that training needs.

A custom layer must be findable

A custom class is not known to Keras when it loads a file. The next block saves a model with a custom layer and loads it three ways.

```
import os
import tempfile

import keras
from keras import layers

class Scale(keras.layers.Layer):
    def __init__(self, factor=2.0, **kwargs):
        super().__init__(**kwargs)
        self.factor = factor

    def call(self, x):
        return x * self.factor

    def get_config(self):
        return {**super().get_config(), "factor": self.factor}

@keras.saving.register_keras_serializable(package="book")
class RegisteredScale(Scale):
    pass

folder = tempfile.mkdtemp()
plain = keras.Sequential([keras.Input((4,)), Scale(3.0), layers.Dense(1)])
plain.save(os.path.join(folder, "plain.keras"))
registered = keras.Sequential([keras.Input((4,)), RegisteredScale(3.0), layers.Dense(1)])
registered.save(os.path.join(folder, "registered.keras"))

try:
    keras.saving.load_model(os.path.join(folder, "plain.keras"))
except TypeError as error:
    print("not registered:", str(error).split(" could not")[0])

with_objects = keras.saving.load_model(os.path.join(folder, "plain.keras"), custom_objects=
{"Scale": Scale})
print("with custom_objects, factor:", with_objects.layers[0].factor)

by_registry = keras.saving.load_model(os.path.join(folder, "registered.keras"))
print("registered class, factor:", by_registry.layers[0].factor)
```

```
not registered: <class 'keras.src.models.sequential.Sequential'>
with custom_objects, factor: 3.0
registered class, factor: 3.0
```

Without registration the load fails with a `TypeError` that names the model class and not the missing layer. The message is not helpful, so know this cause. There are two fixes: pass the class in `custom_objects`, or decorate it with

`register_keras_serializable`. The decorator is the better one, because the file then loads anywhere that the class is imported.

A subclassed model needs `get_config` for its arguments

A model with a constructor argument has a second trap. The next block saves four subclassed models and tries to load each one. Every class is registered.

```
import os
import tempfile

import keras
import numpy as np
from keras import layers, ops

keras.utils.set_random_seed(0)
x = np.zeros((2, 3), dtype="float32")
folder = tempfile.mkdtemp()

@keras.saving.register_keras_serializable(package="book")
class WithKwargs(keras.Model):
    def __init__(self, width=4, **kwargs):
        super().__init__(**kwargs)
        self.hidden = layers.Dense(width)
        self.out = layers.Dense(2)

    def call(self, x):
        return self.out(self.hidden(x))

@keras.saving.register_keras_serializable(package="book")
class WithConfig(WithKwargs):
    def __init__(self, width=4, **kwargs):
        super().__init__(width=width, **kwargs)
        self.width = width

    def get_config(self):
        return {**super().get_config(), "width": self.width}

@keras.saving.register_keras_serializable(package="book")
class NoKwargs(keras.Model):
    def __init__(self):
        super().__init__()
        self.hidden = layers.Dense(4)
        self.out = layers.Dense(2)

    def call(self, x):
        return self.out(self.hidden(x))

def round_trip(label, model):
    model(x)
    path = os.path.join(folder, label + ".keras")
    model.save(path)
    try:
        loaded = keras.saving.load_model(path)
        equal = np.array_equal(ops.convert_to_numpy(model(x)),
ops.convert_to_numpy(loaded(x)))
        print(f"{label:28s} loads, outputs equal: {equal}")
    except Exception as error:
        print(f"{label:28s} fails: {type(error).__name__}")

round_trip("default argument", WithKwargs())
round_trip("width=7, no get_config", WithKwargs(width=7))
round_trip("width=7, with get_config", WithConfig(width=7))
round_trip("__init__ without **kwargs", NoKwargs())
```

```
default argument             loads, outputs equal: True
width=7, no get_config       fails: ValueError
width=7, with get_config     loads, outputs equal: True
__init__ without **kwargs    fails: TypeError
```

The first and the third cases load, and their outputs are equal. The second case fails: the file holds no `width`, so Keras rebuilds the model with the default of 4, and the weights of the file do not fit. The error message talks about missing variables, and it does not mention the argument. The fourth case fails because Keras passes `name`, `trainable` and `dtype` to `__init__`, and the class accepts none of them.

The rule has two lines. A subclassed model must pass `**kwargs` to its parent. It must return every non-default argument from `get_config`.

A loss without `get_config` forgets its arguments

A compiled model saves its loss too. A loss class that has no `get_config` loads with its default arguments. Nothing fails.

```
import os
import tempfile

import keras
import numpy as np
from keras import layers, ops

keras.utils.set_random_seed(0)

@keras.saving.register_keras_serializable(package="book")
class ScaledError(keras.losses.Loss):
    def __init__(self, factor=0.1, **kwargs):
        super().__init__(**kwargs)
        self.factor = factor

    def call(self, y_true, y_pred):
        return ops.mean(ops.square(y_true - y_pred), axis=-1) * (1.0 + self.factor)

@keras.saving.register_keras_serializable(package="book")
class ScaledErrorWithConfig(ScaledError):
    def get_config(self):
        return {**super().get_config(), "factor": self.factor}

folder = tempfile.mkdtemp()
for loss_class in (ScaledError, ScaledErrorWithConfig):
    model = keras.Sequential([keras.Input((3,)), layers.Dense(1)])
    model.compile("sgd", loss_class(0.5))
    model.fit(np.zeros((4, 3), dtype="float32"), np.zeros((4, 1), dtype="float32"), verbose=0)
    path = os.path.join(folder, loss_class.__name__ + ".keras")
    model.save(path)
    print(f"{loss_class.__name__:22s} factor saved 0.5, factor after load:",
          keras.saving.load_model(path).loss.factor)
```

```
ScaledError            factor saved 0.5, factor after load: 0.1
ScaledErrorWithConfig  factor saved 0.5, factor after load: 0.5
```

The class without `get_config` comes back with a factor of 0.1. The class with `get_config` comes back with 0.5. A model that you load to continue training now trains with a different loss, and it reports no error. Give every custom class a `get_config`, and test the round trip.

Export for serving

A `.keras` file needs Keras and your classes. A service often has neither. `model.export` writes a TensorFlow SavedModel. It holds the computation and the weights and no Keras code.

```
import os
import tempfile

import keras
import numpy as np
import tensorflow as tf
from keras import layers
from examples.digits import load

x_train, x_test, y_train, y_test = load()
keras.utils.set_random_seed(0)
model = keras.Sequential([keras.Input((64,), name="pixels"), layers.Dense(32,
activation="relu"),
                          layers.Dense(10)])
model.compile("adam", keras.losses.SparseCategoricalCrossentropy(from_logits=True))
model.fit(x_train, y_train, epochs=2, batch_size=32, verbose=0)

folder = os.path.join(tempfile.mkdtemp(), "export")
model.export(folder, verbose=False)
print("files:", sorted(os.listdir(folder)))

served = tf.saved_model.load(folder)
print("endpoints:", sorted(served.signatures))
endpoint = served.signatures["serving_default"]
result = endpoint(pixels=tf.constant(x_test[:5]))
print("output names:", list(result))
expected = model.predict(x_test[:5], verbose=0)
print("equal to model.predict:", bool(np.allclose(result["output_0"].numpy(), expected,
atol=1e-5)))
```

```
files: ['assets', 'fingerprint.pb', 'saved_model.pb', 'variables']
endpoints: ['serve', 'serving_default']
output names: ['output_0']
equal to model.predict: True
```

The exported model has a `serving_default` endpoint, and it returns the same numbers as `predict`. The key of the input in the signature is the name of the input of the model, `pixels`. Name your inputs when you build the model. An unnamed input gets a name such as `keras_tensor_37`, and the number depends on how many tensors the process

made before it. In the first version of this block, the call with `keras_tensor` worked when the block ran alone. It failed when it ran after the other blocks of the chapter.

Reference

```
import keras
from examples.signatures import show

show(("Model.save", keras.Model.save), ("saving.load_model", keras.saving.load_model),
     ("Model.save_weights", keras.Model.save_weights), ("Model.load_weights",
keras.Model.load_weights),
     ("Model.export", keras.Model.export),
     ("saving.register_keras_serializable", keras.saving.register_keras_serializable))
```

```
Model.save(filepath, overwrite=True, zipped=None, **kwargs)
saving.load_model(filepath, custom_objects=None, compile=True, safe_mode=True)
Model.save_weights(filepath, overwrite=True, max_shard_size=None)
Model.load_weights(filepath, skip_mismatch=False, **kwargs)
Model.export(filepath, format='tf_saved_model', verbose=None,
    input_signature=None, **kwargs)
saving.register_keras_serializable(package='Custom', name=None)
```

parameter	default	what it does	what goes wrong
`save(filepath)`	required	path that ends in `.keras`	—
`save(overwrite)`	`True`	replace an existing file	—
`load_model(custom_objects)`	`None`	map from name to class	needed for an unregistered class
`load_model(compile)`	`True`	rebuild the optimizer and the loss from the file	the loss class must be registered, or pass `custom_objects`
`load_model(safe_mode)`	`True`	refuse to run unsafe code from a file	—
`save_weights(filepath)`	required	write the arrays only; the name ends in `.weights.h5`	the architecture must be rebuilt by you
`export(filepath)`	required	write a SavedModel for serving	an unnamed input became `keras_tensor_37` in one run, so a service cannot use it as a key
`export(format)`	`'tf_saved_model'`	kind of artifact	—
`register_keras_serializable(package)`	`'Custom'`	put the class in the registry under `package>Name`	an unregistered class failed to load (this chapter)

What you have now

You have a round trip that you measured, and four faults that you can now find: a class that is not registered, a subclassed model that does not keep its arguments, a loss that forgets its arguments, and a missing `**kwargs`. You also have an exported artifact that a service can load.

Chapter 15 is about speed and repeatability. Exercises for this chapter are in exercises/14.md, and the worked solutions are in exercises/solutions/14.md.

Chapter 15 — Performance and reproducibility

> **Build** — a test that counts how many different results the same training gives, a measurement of how much one setting narrows them, a rule for layer order that decides the start weights, and four measured facts about `steps_per_execution`, `jit_compile`, mixed precision and the back end.
>
> **Theory kept** — a result is reproducible when the same inputs give the same bits. Seeds are necessary and not sufficient. The order of floating-point operations must also be fixed.
>
> **Source** — Keras 3.15.1 on TensorFlow 2.21.0, CPU only. The code is New.
>
> **Run this first** — `uv run python -m examples.ch15_performance_and_reproducibility`, which is `examples/ch15_performance_and_reproducibility.py`.

This book printed numbers to four digits and compared them exactly. That is only fair if the same code gives the same numbers on every run. This chapter shows when it does not. It measures one setting that makes runs closer, and it says what that setting did not fix.

Speed is the second topic. This book ran on a CPU with no accelerator, so the chapter does not report timings. It reports what each speed option does to the result, and what it did not do on this machine.

The same code, many results

Setting a seed is not enough. The fingerprint of a run in this section is a hash of the bytes of all weights, and the last training loss. The program is a small CNN that trains for 40 epochs. It is the module `examples/determinism_probe.py`.

The next block runs the probe in 8 separate processes at the same time. It does this once with the TensorFlow setting `TF_DETERMINISTIC_OPS` on and once with it off. It prints the number of distinct weight hashes and the spread of the final loss. The output depends on the machine and on the load, so the checker does not compare it. The text block holds what the run that wrote this chapter printed.

```
import os
import subprocess
import sys
from concurrent.futures import ThreadPoolExecutor

def run_probe(flag):
    env = {**os.environ, "TF_DETERMINISTIC_OPS": flag}
    result = subprocess.run([sys.executable, "-m", "examples.determinism_probe"],
                            capture_output=True, text=True, env=env)
    digest, loss = result.stdout.strip().splitlines()[-1].split()
    return digest, float(loss)

for flag, label in (("1", "on "), ("0", "off")):
    with ThreadPoolExecutor(max_workers=8) as pool:
        runs = list(pool.map(run_probe, [flag] * 8))
    losses = [loss for _, loss in runs]
    print(f"flag {label}: 8 runs, distinct weight hashes {len({d for d, _ in runs})}, "
          f"loss spread {max(losses) - min(losses):.2e}")
```

```
# output varies
flag on : 8 runs, distinct weight hashes 2, loss spread 1.00e-08
flag off: 8 runs, distinct weight hashes 2, loss spread 1.16e-05
```

Read the result with care. The setting did not make the run bit-exact on this machine: 8 runs gave 2 different sets of weights with it on, and 2 with it off. The setting did make the difference small.

The spread of the final loss fell from 1.16e-05 to 1.00e-08, which is about 1,000 times. The chapter did not find the source of the remaining difference. A separate test with one TensorFlow thread also gave 2 hashes in 8 runs.

What this means for you: compare two runs with a tolerance and not with `==`. The checker of this book compares printed values that are rounded to 3 or 4 decimals, and a spread of 1e-8 is far below that.

The next block checks the part that is firm. It runs the probe 4 times with the setting on and tests that the final losses agree to 1e-6.

```
import os
import subprocess
import sys

env = {**os.environ, "TF_DETERMINISTIC_OPS": "1"}
losses = []
for _ in range(4):
    result = subprocess.run([sys.executable, "-m", "examples.determinism_probe"],
                            capture_output=True, text=True, env=env)
    losses.append(float(result.stdout.strip().splitlines()[-1].split()[1]))
print("runs:", len(losses))
print("losses agree to 1e-6:", max(losses) - min(losses) < 1e-6)
```

```
runs: 4
losses agree to 1e-6: True
```

How it works

A floating-point sum depends on the order of the additions. In 32-bit numbers, `(a + b) + c` can differ from `a + (b + c)`. A multi-threaded reduction adds the parts in the order in which the threads finish, and that order can change from run to run. The `TF_DETERMINISTIC_OPS` setting asks TensorFlow to use implementations with a fixed order where they exist. This book measured only the effect of the setting, shown in the previous section.

The next block shows the first fact. It does not show the cause of the differences in training, which this book did not observe directly.

```
import numpy as np

a, b, c = np.float32(1e8), np.float32(-1e8), np.float32(1.0)
print("(a + b) + c =", (a + b) + c)
print("a + (b + c) =", a + (b + c))
```

```
(a + b) + c = 1.0
a + (b + c) = 0.0
```

The two orders give 1.0 and 0.0. The invariant is this: a result is bit-reproducible only if every random number and every order of addition is fixed. The seed fixes the random numbers. The setting fixes the order of the operations that it covers. The previous section shows that this was not enough for exact equality on this machine.

The setting has a second effect that surprised the author. With determinism on, a `fit` call without a seed failed in several blocks of this book. The message was: `Random ops require a seed to be set when determinism is enabled`. The shuffle of the training data is the random operation.

A script that worked before can fail with this message. The fix is a call to `keras.utils.set_random_seed` at the top. Exercise 2 reproduces the error.

The order of construction decides the start weights

The start weights of a layer come from a random generator that Keras advances each time it makes an initializer. So the order in which you create layers decides which numbers each layer gets. The two lines below build the same network. They do not give the same weights.

```
import keras
import numpy as np
from keras import layers

def nested():
    keras.utils.set_random_seed(0)
    inputs = keras.Input((8,))
    outputs = layers.Dense(3)(layers.Dense(5)(inputs))
    return keras.Model(inputs, outputs)

def in_order():
    keras.utils.set_random_seed(0)
    inputs = keras.Input((8,))
    hidden = layers.Dense(5)(inputs)
    return keras.Model(inputs, layers.Dense(3)(hidden))

first, second = nested(), in_order()
print("first layer kernels equal:", np.array_equal(first.get_weights()[0],
second.get_weights()[0]))
```

```
first layer kernels equal: False
```

In the nested line, Python creates the outer `Dense(3)` before it calls the inner expression, so the outer layer takes the first random numbers. The two models have the same shapes and different weights. When you compare two runs, build the layers in the same order, and write the order down.

Four speed options, measured for their effect

The next block trains the same model with the default, with `steps_per_execution=4`, with `jit_compile=True`, and under a mixed-precision policy. It prints the final loss of 3 epochs and, for the first two, the largest difference of the weights.

```
import warnings

import keras
import numpy as np
from keras import layers
from examples.digits import load

x_train, x_test, y_train, y_test = load()
loss = keras.losses.SparseCategoricalCrossentropy(from_logits=True)

def train(**compile_options):
    keras.utils.set_random_seed(0)
    model = keras.Sequential([keras.Input((64,)), layers.Dense(64, activation="relu"),
layers.Dense(10)])
    with warnings.catch_warnings(record=True) as caught:
        warnings.simplefilter("always")
        model.compile("adam", loss, **compile_options)
        history = model.fit(x_train, y_train, epochs=3, batch_size=32, shuffle=False,
verbose=0).history
    flat = np.concatenate([w.ravel() for w in model.get_weights()])
    return flat, history["loss"][-1], [str(c.message)[:58] for c in caught]

base, base_loss, _ = train()
fused, fused_loss, _ = train(steps_per_execution=4)
jitted, jitted_loss, notes = train(jit_compile=True)
print("default             loss", round(base_loss, 4))
print("steps_per_execution loss", round(fused_loss, 4), " weight difference",
float(np.abs(base - fused).max()))
print("jit_compile=True    loss", round(jitted_loss, 4), " weight difference",
float(np.abs(base - jitted).max()))
print("jit message:", notes[0] if notes else None)

keras.mixed_precision.set_global_policy("mixed_float16")
model = keras.Sequential([keras.Input((64,)), layers.Dense(64, activation="relu"),
layers.Dense(10)])
first = model.layers[0]
print("mixed policy: compute", first.compute_dtype, " variables", first.variable_dtype,
      " output", model(x_test[:2]).dtype)
keras.utils.set_random_seed(0)
model.compile("adam", loss)
mixed_loss = model.fit(x_train, y_train, epochs=3, batch_size=32, shuffle=False,
verbose=0).history["loss"][-1]
print("mixed precision loss", round(mixed_loss, 4), " optimizer:",
type(model.optimizer).__name__)
keras.mixed_precision.set_global_policy("float32")
```

```
default             loss 1.0979
steps_per_execution loss 1.0979  weight difference 0.0
jit_compile=True    loss 1.0979  weight difference 0.0
jit message: Model doesn't support `jit_compile=True`. Proceeding with
mixed policy: compute float16  variables float32  output <dtype: 'float16'>
mixed precision loss 1.1083  optimizer: LossScaleOptimizer
```

Read the lines in order. `steps_per_execution=4` runs four steps in one call, and it gives the same weights: the difference is 0. The option changes the cost of the loop and not the result. `jit_compile=True` printed a message that

the model does not support it. Keras ran without it, so the weights are equal for that reason, and this chapter did not measure any speed gain.

Mixed precision changed the result. The compute type is `float16`, the variables stay `float32`, and the optimizer is wrapped in a loss-scaling optimizer. The loss after 3 epochs is different from the default. The difference is the price of the 16-bit numbers of Chapter 2. Whether the speed pays for it depends on hardware that this book did not have.

The other back ends

Keras 3 runs on JAX and on PyTorch. This book installed neither, so no result in it comes from them. The next block shows what Keras says when you ask.

```
import os
import subprocess
import sys

for backend in ("jax", "torch"):
    result = subprocess.run([sys.executable, "-c", "import keras"],
                            capture_output=True, text=True,
                            env={**os.environ, "KERAS_BACKEND": backend})
    print(backend, "->", result.stderr.strip().splitlines()[-1])
```

```
jax -> ModuleNotFoundError: No module named 'jax'
torch -> ModuleNotFoundError: No module named 'torch'
```

The statement "this code runs on every back end" is a property of the functions that the book used, and not a measurement. Run the blocks on your back end before you rely on it. A custom `train_step` with `tf.GradientTape` (Chapter 9) will not run on another back end.

A checklist

1. Call `keras.utils.set_random_seed` once, at the top.
2. Set `TF_DETERMINISTIC_OPS=1` to make runs closer, and compare them with a tolerance.
3. Build layers in a fixed order, and do not nest constructors that you compare.
4. Change one option at a time: the chapter showed an option that changes nothing and an option that changes the result.
5. State the hardware and the back end next to every number.

Reference

```
import keras
from examples.signatures import show

show(("Model.compile", keras.Model.compile),
     ("utils.set_random_seed", keras.utils.set_random_seed),
     ("mixed_precision.set_global_policy", keras.mixed_precision.set_global_policy))
```

```
Model.compile(optimizer='rmsprop', loss=None, loss_weights=None, metrics=None,
    weighted_metrics=None, run_eagerly=False, steps_per_execution=1,
    jit_compile='auto', auto_scale_loss=True)
utils.set_random_seed(seed)
mixed_precision.set_global_policy(policy)
```

parameter	default	what it does	what goes wrong
`compile(steps_per_execution)`	`1`	steps that run in one call	4 gave the same weights as 1
`compile(jit_compile)`	`'auto'`	compile the step with XLA	`True` printed that the model does not support it, on this machine
`compile(run_eagerly)`	`False`	run the step in Python	—
`set_random_seed(seed)`	required	seeds Python, NumPy and the back end	with determinism on, a missing seed is an error
`set_global_policy(policy)`	required	default compute and variable types	`'mixed_float16'` changed the loss

What you have now

You have a count of distinct results and a setting that narrows the differences. You also have two sources of difference that you can control: the order of layer construction and the seed. The cause of the remaining difference is open. The list of what this book did not measure is in Chapter 16: speed, and any back end except TensorFlow.

Chapter 16 collects the book into a table from symptom to cause. Exercises for this chapter are in exercises/15.md, and the worked solutions are in exercises/solutions/15.md.

Chapter 16 — A field guide

> **Build** — a table from symptom to cause to chapter, a check that every row of the table points at a real chapter, the list of errors that this book made and corrected while it was written, and the list of things it did not measure.
>
> **Theory kept** — most failures in a Keras project are not exceptions. They are results that look plausible. A measurement with a number is the only reliable way to tell.
>
> **Source** — the earlier chapters of this book. The code is New.
>
> **Run this first** — `uv run python -m examples.ch16_field_guide`, which is `examples/ch16_field_guide.py`.

From symptom to cause

The table lists what you see, the most likely cause, the measurement of the book that shows it, and the chapter. Find your symptom in the first column.

You see	Likely cause	Measured in this book	Chapter
the loss is `nan` after some epochs	the learning rate is too large for the scale of the input	`SGD(1.5)` on the line gave `nan`; raw pixels with SGD gave 0.296	1, 5
a run finishes with no error and a bad score	a wrong flag gives a valid loss for the wrong problem	logits with `from_logits=False`: 0.5711 against 0.9711	6
training accuracy is high and validation accuracy is near zero	`validation_split` takes the last rows of sorted data	0.015 validation accuracy	5
a frozen layer keeps training, or an unfrozen layer stays frozen	`trainable` changed after `fit` without a new `compile`	all four cases printed	3
a value of 10,000,000 at test time	a feature that is constant in the training data	`Normalization` divided by 1e-7	5
the returned model is not the best one	`restore_best_weights` is `False` by default	loss 0.2186 against 0.2176	7
the loss in a callback is not the loss of the batch	the logs hold the running average of the epoch	last batch equals the epoch loss	7
a regularizer lowers the gap and not the error	the gap is not the score	all four within 3 images of 450	8
a custom metric gives a plausible wrong value	`reset_state` does not clear the state	0.6929 against 0.8934	9
a CNN is not robust to a one-pixel shift	flatten gives each position its own weights	0.638 against 0.991 with a global max pool	10
a sequence model is at chance	the model has no sense of order	bag of tokens 0.502	11
a transformer is at chance	no position signal	0.502 without, 1.000 with	12
a model with positions has the same weight count as one without	the embedding was called outside a layer and is not tracked	8,897 against 9,409	12
a model fails to load with "expected 2 variables"	a constructor argument is not in `get_config`	`width=7` fails, with `get_config` loads	14
a loaded model trains with a different loss	the loss class has no `get_config`	factor 0.5 came back as 0.1	14
the same seed gives different results	order of floating-point operations, or layer order	2 weight hashes in 8 runs; layer kernels differ	15

The next block checks the table against the files of the book. It reads this chapter, takes every chapter number in the last column and tests that the chapter file exists.

```
import pathlib
import re

root = pathlib.Path(".")
text = (root / "chapters" / "16-field-guide.md").read_text(encoding="utf-8")
links = re.findall(r"\[(\d+)\]\((\d\d-[a-z-]+\.md)\)", text)
missing = [name for _, name in links if not (root / "chapters" / name).exists()]
numbers_match = all(name.startswith(f"{int(number):02d}-") for number, name in links)
print("links in the table:", len(links))
print("missing chapter files:", missing)
print("the number in each link matches its file name:", numbers_match)
```

```
links in the table: 17
missing chapter files: []
the number in each link matches its file name: True
```

Errors that this book made and fixed

A book that shows its corrections is easier to trust. Each item below was found by a measurement and not by a reader. The text of the chapter now says the measured version.

Chapter	The first draft said	The measurement said
3	exercise 1 expected 237 trainable parameters	241; the by-hand count agrees
6	exercise 2 expected a top-3 accuracy above 0.95	0.8822
6	`from_logits=False` divides by the row sum, then clips	it clips, then takes the log; the first formula gave 8.5090, Keras gave 13.2655
7	the batch log is the loss of that batch	it is the running average of the epoch
10	a global max pool makes the output exactly shift-invariant	the maximum was the same for 97.9% of the image-channel pairs
11	the accuracy of the `conv` model falls steadily with the distance	it did not; token position at the edge leaks
12	a position embedding called outside a layer is a trained table	it is a fixed table with 0 trained weights; the count stayed at 8,897
4	a subclassed model gives up saving without extra code	it saves and loads when the class is registered, `__init__` passes `**kwargs`, and `get_config` returns every non-default argument
9	a loss without `get_config` cannot be rebuilt	it loads, and its arguments return to the defaults
6	the accuracy of the broken `from_logits=False` setup is 0.7089	0.5711 with the determinism setting on; the setup is unstable, so no single number is firm
13	the frozen base loses to training from scratch	the head had not converged; at a larger budget the frozen base was ahead, and at a third budget the three modes tied at 20 images for each class
14	the export signature takes the key `keras_tensor`	the name depends on how many tensors the process made before; the key failed in the chapter module and works with a named input
15	a `Dense` layer without a seed fails under determinism	the shuffle in `fit` is the random operation; building the layer did not fail
all	outputs of CNN training are repeatable with a seed	2 weight hashes in 8 runs; `TF_DETERMINISTIC_OPS=1` narrowed the loss spread from 1.16e-05 to 1.00e-08 and did not remove it

What this book did not measure

- **Speed.** All runs were on one CPU. `jit_compile` was refused for the model of Chapter 15, and no timing is reported.
- **Other back ends.** JAX and PyTorch were not installed. "Portable" means "uses only `keras.ops`", and not "was run on three back ends".
- **Accelerators.** No GPU or TPU was used. Mixed precision was run for its numbers and not for its speed.
- **Large data.** The largest training set has 1,347 images of 8 × 8 pixels. The traps are the same at scale. The sizes of the effects are not.
- **Other versions.** Every signature and every message comes from Keras 3.15.1 with TensorFlow 2.21.0. A later version can differ.

What you have now

You have a table that takes you from a symptom to a measurement. You also have a list of the mistakes that this book made and a list of what it did not test. Use the table before you debug.

Exercises for this chapter are in exercises/16.md, and the worked solutions are in exercises/solutions/16.md.

Appendix A — Described, not built

This book measures what it states. A few statements are not measured, and each one is written as a statement about the world and not as a result. They are here, so that you can see where the book's evidence stops.

Statement	Where	Why it was not measured
Code that uses only `keras.ops` runs on JAX and PyTorch.	2, 9, 15	JAX and PyTorch were not installed. The book says what Keras prints when you ask for them.
A `train_step` with `tf.GradientTape` runs on TensorFlow only; `compute_loss` overrides run on every back end.	9	Same reason. The first half follows from the use of `tf`. The second half is the documented contract.
Mixed precision is faster on a GPU.	15	No GPU. The book measured the effect on the loss and the dtypes only.
`jit_compile=True` speeds up a model.	15	Keras refused it for the model of the chapter on this machine. No timing exists.
The cost of attention grows with the square of the length.	12	The book measured the shape of the score table, `T × T`. It did not time it.
The cause of the remaining run-to-run difference with `TF_DETERMINISTIC_OPS=1`.	15	The book measured the size of the difference and one setting that did not remove it. The cause is open.
Why a `conv1d` model scores 1.000 on pairs with a token at the edge.	11	The edge test confirmed that edge pairs score 1.000 and interior pairs 0.504. The mechanism (fewer windows at the border) is the candidate and was not isolated further.
Behavior of Keras versions other than 3.15.1, and of TensorFlow other than 2.21.0.	all	Not installed. Every signature and message comes from the installed versions.
Behavior at scale: millions of images, long sequences, many GPUs.	all	The largest training set has 1,347 images of 8 × 8 pixels.

Exercises — Chapter 1

Chapter: The first model · module: `examples/ch01_the_first_model.py` · solutions: 01

1. [measure] Train the line model of the chapter for 100 epochs with 64 samples. Read `model.optimizer.iterations` after the run for batch size 16 and for batch size 64. Expected: 400 and 100, exactly.

2. [break] Keep `SGD(0.1)`, the model and the 100 epochs. Change only the data: multiply `x` by 20, so that `x` runs from -20 to 20, and keep `y = 2x + 1`. Report the final loss. Expected: `nan`. Then say which property of the loss explains why the same learning rate now fails.

3. [judge] A colleague proposes this rule for the team: "Always train for 100 epochs and ship the result. If the loss is small, the model is good." What does a good answer weigh?

Exercises — Chapter 2

Chapter: Tensors and ops · module: `examples/ch02_tensors_and_ops.py` · solutions: 02

1. [measure] Cast the integer 2049 to `float16` and back to a NumPy value. Report the number you get. Then find the smallest positive integer that `float16` cannot represent exactly. Expected: 2048 for the first part, and 2049 for the second.

2. [break] Write a function `noisy(x)` that returns `x + keras.random.normal(x.shape, seed=3)`. Call it twice on the same input and show that the noise is identical. Then change one line so that two calls give different noise and the whole run is still repeatable. Expected: `True` before the change, `False` after.

3. [judge] The team runs on TensorFlow only, and has done so for four years. A reviewer asks you to write a new custom layer with `tf.*` functions because "they are better documented and we will never switch". What does a good answer weigh?

Exercises — Chapter 3

Chapter: Layers and weights · module: `examples/ch03_layers_and_weights.py` · solutions: 03

1. [measure] A model has `Dense(10)` on an input of 20 features, then `BatchNormalization`, then `Dense(1)`. Use the layer attributes to count the trainable and the non-trainable parameters. Expected: 241 trainable parameters and 20 non-trainable parameters.

2. [break] Write a layer whose `build` method creates its weight with `trainable=False`, put it in a model with a `Dense(1)` and train with SGD. Show that the layer weight does not change. Then change one argument so that the weight does change. Expected: `False` and then `True`.

3. [judge] A colleague wants a custom layer that keeps a running maximum of its input in a non-trainable weight, to use at inference. Is this a good use of the mechanism of this chapter? What does a good answer weigh?

Exercises — Chapter 4

Chapter: Three ways to build a model · module: `examples/ch04_three_ways_to_build.py` · solutions: 04

1. [measure] Build the siamese model of the chapter again, but give the two inputs 6 features each and make the shared layer `Dense(10)`. Report `count_params()`. Expected: 70 + 21 + 63 = 154.

2. [break] In the functional model of the chapter, replace the shared layer by two separate `Dense(8)` layers, one for each input. Report the parameter count and the number of `Dense` layers in `model.layers`. Say what property of the model you lost. Expected: 148 parameters.

3. [judge] A teammate writes every model as a subclass of `keras.Model`, because "it is just Python and I can debug it". Another teammate says to use the functional style. What does a good answer weigh?

Exercises — Chapter 5

Chapter: Data pipelines · module: `examples/ch05_data_pipelines.py` · solutions: 05

1. [measure] Count the batches in one epoch of the `PyDataset` of the chapter for batch sizes 32, 64 and 100. Report the size of the last batch for each. Expected: 43, 22 and 14 batches, with last batches of 3, 3 and 47 images.

2. [break] Adapt a `Normalization` layer on the training images, then apply it to the test images in which you have set pixel 24 to zero. Show that the huge output disappears. Then explain why that fix would hide a real problem if you used it in production.

3. [judge] A colleague reports 99% validation accuracy on a new data set. The code uses `validation_split=0.2` on arrays that a database query produced. What does a good answer ask and weigh before anyone celebrates?

Exercises — Chapter 6

Chapter: Losses, metrics and optimizers · module: `examples/ch06_losses_metrics_optimizers.py` · solutions: 06

1. [measure] Train the baseline classifier of the chapter for 20 epochs and compute the confusion between classes 4 and 9 on the test images: how many images of class 4 are predicted as 9, and how many of class 9 as 4? Report the two counts. Expected: both counts are 0, and the model makes 13 errors in 450 images.

2. [break] Use `SparseCategoricalCrossentropy(from_logits=False)` with a model that has no final activation, as in the chapter, but add the metric `SparseTopKCategoricalAccuracy(k=3)`. Report the top-3 accuracy next to the top-1 accuracy. Say what this tells you about the broken model. Expected: top-1 near 0.57 and top-3 near 0.88, with the determinism setting on.

3. [judge] A teammate wants to ship the optimizer that has the best accuracy in the last column of the optimizer table. What does a good answer weigh?

Exercises — Chapter 7

Chapter: Callbacks and training control · module: `examples/ch07_callbacks_and_training_control.py` · solutions: 07

1. [measure] Run the early-stopping setup of the chapter with `patience` 1, 5 and 15. For each value report the epoch where the run stops and the best epoch. Expected: the run stops exactly `patience` epochs after the best epoch, whenever the run stops before epoch 100.

2. [break] Give `EarlyStopping` the arguments `monitor="val_accuracy"` and `mode="min"`, so that a lower accuracy counts as an improvement. Report the epoch where the run stops, and the test accuracy of the returned model, for `mode="min"` and for `mode="max"`. Say which result is the intended one. Expected: the two runs stop at different epochs, and the `"min"` run returns a worse model.

3. [judge] `ModelCheckpoint` with `save_best_only=True` monitors the validation set, and so does `EarlyStopping`. After 60 epochs of selection, can you report the validation accuracy of the saved model as the accuracy of the model? What does a good answer weigh?

Exercises — Chapter 8

Chapter: Overfitting and regularization · module: `examples/ch08_overfitting_and_regularization.py` · solutions: 08

1. [measure] Repeat the regularizer table with three different seeds for the baseline and for dropout 0.5. Report the test accuracy for each seed and the spread of each setting. Expected: the spread within a setting is 0.004, and the sign of the difference between the settings is not the same as in the single-seed table of the chapter.

2. [break] Train the dropout 0.5 model and compare `model(x_test, training=True)` with `model(x_test, training=False)` on the same input. Show that the two outputs differ, and show that two calls with `training=False` are equal. Say what this means if you forget to pass `training=False` when you serve the model.

3. [judge] The learning curve is flat from 800 to 1,347 images. A manager asks the team to stop collecting data. What does a good answer weigh?

Exercises — Chapter 9

Chapter: Custom losses, metrics and training steps · module: `examples/ch09_custom_training.py` · solutions: 09

1. [measure] Train the model of the metric section for 3 epochs. Then call `evaluate` on the test images with `batch_size=1` and with `batch_size=450`. Report `balanced_accuracy` for both. Expected: the two values are equal to 5 digits, because the metric keeps counts and not a mean of batches.

2. [break] Change `reset_state` of `BalancedAccuracy` so that it does nothing. Train for 3 epochs with validation data. Compare the `val_balanced_accuracy` of the third epoch with the correct value from scikit-learn. Expected: the broken value is not equal to the correct one, and the metric is a mean over all epochs.

3. [judge] A teammate wants label smoothing on a TensorFlow-only project and plans to write a custom `train_step` with `tf.GradientTape` to get it. What does a good answer weigh?

Exercises — Chapter 10

Chapter: Convolutional networks · module: `examples/ch10_convolutional_networks.py` · solutions: 10

1. [measure] A network has `Conv2D(8, 3, padding="valid")`, `MaxPooling2D(2)`, `Conv2D(16, 3, padding="valid")` and `GlobalMaxPooling2D()` on an input of 8 × 8 × 1. Report the shape after each layer and the total number of convolution weights. Expected: shapes (6, 6, 8), (3, 3, 8), (1, 1, 16), (16,), and 80 + 1,168 = 1,248 weights.

2. [break] Take the `cnn global` model of the chapter and shift the test images by two pixels to the right. Report the accuracy. Then say why the result is different from the one-pixel result. Expected: the accuracy falls below 0.99.

3. [judge] A colleague says "use a CNN for everything with a grid, because convolution is translation invariant". What does a good answer weigh?

Exercises — Chapter 11

Chapter: Sequences · module: `examples/ch11_sequences.py` · solutions: 11

1. [measure] Compute by hand the number of weights of the `gru` model: an `Embedding(10, 16)`, a `GRU(32)` and a `Dense(1)`. Then check the number with `count_params()`. Use this fact: a GRU with input width `d` and `u` units has `3 × (d × u + u × u + 2 × u)` weights. Expected: 160 + 4,800 + 33 = 4,993.

2. [break] The chapter offers one candidate for the shape of the `conv` curve: tokens at the edge of the sequence appear in fewer windows. Test it. Train the `conv` model, then score it only on test pairs with a distance of 3 or more. Split the pairs into those where both tokens are at positions 4 to 11, and those where at least one token is at position 0, 1, 14 or 15. Report both accuracies. Expected: the interior pairs score near 0.5, and the edge pairs score near 1.0.

3. [judge] A team classifies call transcripts into 12 topics. A bag-of-words model scores 0.88 and trains in a minute. A recurrent model scores 0.89 and trains in an hour. Which one does the team ship? What does a good answer weigh?

Exercises — Chapter 12

Chapter: Attention and transformers · module: `examples/ch12_attention_and_transformers.py` · solutions: 12

1. [measure] Count the weights of `MultiHeadAttention(num_heads=4, key_dim=16)` on an input of width 64. Do it by hand from the structure in the chapter, then with `count_params()`. Expected: 3 × (64 × 4 × 16 + 4 × 16) + (4 × 16 × 64 + 64) = 16,640.

2. [break] Build the `TransformerBlock` of the chapter with a global average pool, with no position embedding. Give it a batch and the same batch with the tokens of every sequence in a different order. Show that the outputs are equal. Then add `AddPositions` and show that they differ. Expected: the first difference is below 1e-5, and the second is not.

3. [judge] A team must classify call logs of 4,000 tokens. A colleague says that attention costs `T²`, so a GRU is the only choice. What does a good answer weigh?

Exercises — Chapter 13

Chapter: Transfer learning · module: `examples/ch13_transfer_learning.py` · solutions: 13

1. [measure] Build the frozen model of the chapter and count its trainable weights and all of its weights. Expected: 330 trainable weights of 5,130.

2. [break] Build a frozen model, compile it, train one epoch, then set `trainable = True` on the base **without** compiling again, and train one more epoch. Report whether the base weights changed. Then repeat with a new `compile`. Expected: `False`, then `True`.

3. [judge] A team has a model that a vendor trained on 10 million images from another domain. They have 200 labeled images of their own. Should they freeze the vendor model, fine-tune it, or train from scratch? What does a good answer weigh?

Exercises — Chapter 14

Chapter: Saving, loading and export · module: `examples/ch14_saving_loading_export.py` · solutions: 14

1. [measure] Train a model of three `Dense` layers with Adam and save it. Count the arrays in the `optimizer` group of the weights file. Predict the number first. Expected: 2 + 2 × 6 = 14, because there are 6 weight arrays, and Adam keeps two numbers for each plus the step count and the learning rate.

2. [break] Save the weights of a model with two `Dense(4)` layers and a `relu` after the first. Load them into a model with the same shapes and the `relu` after the second. Report whether the load raises an error and whether the predictions are equal. Expected: no error, and the predictions differ.

3. [judge] The serving team asks for "the model file". Do you give them the `.keras` file or the exported SavedModel? What does a good answer weigh?

Exercises — Chapter 15

Chapter: Performance and reproducibility · module: `examples/ch15_performance_and_reproducibility.py` · solutions: 15

1. [measure] Run the determinism probe 3 times with `TF_DETERMINISTIC_OPS=1` and compare the final losses. Report the largest difference. Expected: below 1e-6. Do not expect equal weights.

2. [break] Turn determinism on, build a small model without calling `set_random_seed`, and call `fit` for one epoch. Report the first words of the error. Then add the seed and show that the error is gone. Expected: the message starts with "Random ops require a seed".

3. [judge] A colleague shows that mixed precision made training 1.8 times faster on a GPU, and wants it for every project. What does a good answer weigh?

Exercises — Chapter 16

Chapter: A field guide · module: `examples/ch16_field_guide.py` · solutions: 16

1. [measure] Count the rows of the symptom table of the chapter with code. Expected: 16.

2. [break] Take the text of the chapter, change one link target to a file that does not exist, and run the table check again. Show that it reports the file. Expected: the list of missing files has one entry.

3. [judge] Choose one symptom from your own project that the table does not list. What measurement would you add to the book to support it, and what number would it have to show?

Solutions — Chapter 1

Exercises: 01 · chapter: The first model

1. [measure]

```
import keras
import numpy as np

x = np.linspace(-1, 1, 64, dtype="float32").reshape(-1, 1)
for batch_size in (16, 64):
    keras.utils.set_random_seed(0)
    model = keras.Sequential([keras.Input((1,)), keras.layers.Dense(1)])
    model.compile(optimizer=keras.optimizers.Adam(0.1), loss="mse")
    model.fit(x, 2 * x + 1, epochs=100, batch_size=batch_size, verbose=0)
    print(batch_size, int(model.optimizer.iterations.numpy()))
```

```
16 400
64 100
```

One epoch has `ceil(64 / batch_size)` steps. With batch size 16 that is 4 steps, so 100 epochs give 400 updates. With batch size 64 an epoch is one step, so 100 epochs give 100 updates. The number of updates changes with the batch size even when the number of epochs does not.

2. [break]

```
import keras
import numpy as np

x = 20 * np.linspace(-1, 1, 64, dtype="float32").reshape(-1, 1)
y = 2 * x + 1
for scale, data in (("x in [-1, 1]", x / 20), ("x in [-20, 20]", x)):
    keras.utils.set_random_seed(0)
    model = keras.Sequential([keras.Input((1,)), keras.layers.Dense(1)])
    model.compile(optimizer=keras.optimizers.SGD(0.1), loss="mse")
    history = model.fit(data, 2 * data + 1, epochs=100, batch_size=16, verbose=0)
    print(f"{scale:16s} final loss {history.history['loss'][-1]:.3g}")
```

```
x in [-1, 1]     final loss 1.31e-13
x in [-20, 20]   final loss nan
```

The gradient of the mean squared error with respect to the weight contains the factor `x * x`. When `x` is 20 times larger, the gradient is about 400 times larger. A step that was stable at a gradient of size 1 now overshoots by a factor that grows with each update. The stable learning rate depends on the scale of the input. This is the reason that the data must be normalized before training (Chapter 5).

3. [judge]

Position: reject the rule. A small loss on the training data does not make a model good, and 100 epochs is not a property of the problem.

What a good answer weighs. The loss on the training data says that the model fits the data it saw. It says nothing about data it did not see (Chapter 8). The chapter also shows two runs that finish without an error and give a bad result: a loss of `nan` and a loss of 1.27. A fixed epoch count would ship both. Training is finished when a measurement on held-out data stops improving, so the rule must name that measurement and not a number of epochs.

Strongest objection. A fixed recipe is easy to audit, and a team that uses one recipe on every problem makes fewer mistakes than a team that tunes each run. This is true when the recipe also contains a check. The rule fails because it has no check, not because it is fixed.

What would overturn it. A set of past projects in which a fixed-epoch recipe, with a held-out check added, gave the same result as early stopping at lower cost.

Solutions — Chapter 2

Exercises: 02 · chapter: Tensors and ops

1. [measure]

```
import numpy as np
from keras import ops

stored = float(ops.convert_to_numpy(ops.cast(2049.0, "float16")))
print("2049 as float16:", stored)

smallest = next(n for n in range(1, 5000) if float(np.float16(n)) != n)
print("smallest integer float16 cannot hold:", smallest)
```

```
2049 as float16: 2048.0
smallest integer float16 cannot hold: 2049
```

A `float16` number has 11 significant bits, so it holds every integer up to 2048. Above 2048 the spacing between numbers is 2, and an odd integer falls between two neighbors. A count that grows past 2048 in `float16` stops growing when you add 1.

2. [break]

```
import keras
import numpy as np
from keras import ops

def noisy_fixed(shape):
    return ops.convert_to_numpy(keras.random.normal(shape, seed=3))

print("integer seed, calls equal:", np.array_equal(noisy_fixed((4,)), noisy_fixed((4,))))

generator = keras.random.SeedGenerator(3)

def noisy_generator(shape):
    return ops.convert_to_numpy(keras.random.normal(shape, seed=generator))

print("SeedGenerator, calls equal:", np.array_equal(noisy_generator((4,)),
noisy_generator((4,))))
```

```
integer seed, calls equal: True
SeedGenerator, calls equal: False
```

An integer seed has no state, so the same integer always gives the same numbers. A `SeedGenerator` holds a counter that advances on each draw. A new generator with the same starting seed gives the same sequence, so the run stays repeatable.

3. [judge]

Position: write the layer with `keras.ops`.

What a good answer weighs. The cost of `keras.ops` is small: the names match NumPy and the documentation is one page. The benefit is that the layer works on every back end and with every Keras feature that traces a layer symbolically. The cost of `tf.*` is a permanent link to one back end. The reviewer's claim "we will never switch" is a prediction, and the chapter's measurements do not support or refute it. They show only that the portable choice is not harder to write.

Strongest objection. Some operations have no `keras.ops` twin, for example a TensorFlow-specific sparse operation. In that case `tf.*` is the right tool, and the layer should say that it is TensorFlow-only.

What would overturn it. A profile that shows a `keras.ops` function that is slower than its `tf.*` twin on the team's hardware, by an amount that matters for a real workload.

Solutions — Chapter 3

Exercises: 03 · chapter: Layers and weights

1. [measure]

```
import keras
import numpy as np
from keras import layers

keras.utils.set_random_seed(0)

model = keras.Sequential([
    keras.Input((20,)),
    layers.Dense(10),
    layers.BatchNormalization(),
    layers.Dense(1),
])
trainable = sum(int(np.prod(w.shape)) for w in model.trainable_weights)
frozen = sum(int(np.prod(w.shape)) for w in model.non_trainable_weights)
print("trainable:", trainable, "non-trainable:", frozen)
```

```
trainable: 241 non-trainable: 20
```

The count by hand: the first `Dense` has 20 × 10 + 10 = 210. `BatchNormalization` has 10 values of `gamma` and 10 of `beta`, which is 20 trainable. It has 10 moving means and 10 moving variances, which is 20 non-trainable. The last `Dense` has 10 + 1 = 11. The trainable total is 210 + 20 + 11 = 241.

2. [break]

```
import keras
import numpy as np

class Fixed(keras.layers.Layer):
    def __init__(self, trainable_weight, **kwargs):
        super().__init__(**kwargs)
        self.trainable_weight = trainable_weight

    def build(self, input_shape):
        self.scale = self.add_weight(
            shape=(1,), initializer="ones", trainable=self.trainable_weight, name="scale")

    def call(self, x):
        return x * self.scale

x = np.random.default_rng(0).normal(size=(64, 3)).astype("float32")
y = x.sum(1, keepdims=True)
for flag in (False, True):
    keras.utils.set_random_seed(0)
    layer = Fixed(flag)
    model = keras.Sequential([keras.Input((3,)), layer, keras.layers.Dense(1)])
    model.compile(optimizer=keras.optimizers.SGD(0.05), loss="mse")
    model.fit(x, y, epochs=3, verbose=0)
    changed = float(layer.scale.numpy()[0]) != 1.0
    print("trainable =", flag, "-> scale changed:", changed)
```

```
trainable = False -> scale changed: False
trainable = True -> scale changed: True
```

The optimizer updates only the trainable list. A weight with `trainable=False` is state that the layer owns, and the training loop leaves it alone.

3. [judge]

Position: yes, with one condition. The mechanism fits, but the update must happen in the right mode.

What a good answer weighs. A non-trainable weight is the correct place for state that the optimizer must not touch, and the `Counter` layer in the chapter shows that `assign_add` works inside `fit`. The chapter also shows that the layer sees batches, so the running maximum is a maximum over batches seen so far. The colleague must decide the mode. The `Counter` layer updates in both modes. A maximum that updates at inference changes the output of the model between two predictions, which makes the model a function of its history. Update in training mode only, with the `training` argument.

Strongest objection. Computing the maximum in a preprocessing step, once, over the whole training set, is simpler and cannot depend on the order of the batches.

What would overturn it. A requirement that the statistic must adapt in production, with a test that shows the adaptive version is stable under reordering of requests.

Solutions — Chapter 4

Exercises: 04 · chapter: Three ways to build a model

1. [measure]

```
import keras
from keras import layers

keras.utils.set_random_seed(0)

def siamese(features, width, shared_layer=True):
    left = keras.Input((features,), name="left")
    right = keras.Input((features,), name="right")
    if shared_layer:
        shared = layers.Dense(width, activation="relu", name="shared")
        left_out, right_out = shared(left), shared(right)
    else:
        left_out = layers.Dense(width, activation="relu", name="left_dense")(left)
        right_out = layers.Dense(width, activation="relu", name="right_dense")(right)
    joined = layers.Concatenate()([left_out, right_out])
    score = layers.Dense(1, name="score")(joined)
    label = layers.Dense(3, activation="softmax", name="label")(joined)
    return keras.Model([left, right], [score, label])

print("6 features, Dense(10):", siamese(6, 10).count_params())
```

```
6 features, Dense(10): 154
```

The shared layer has 6 × 10 + 10 = 70 parameters. The concatenation has 20 values. The `score` head has 20 + 1 = 21 parameters, and the `label` head has 20 × 3 + 3 = 63. The total is 70 + 21 + 63 = 154.

2. [break]

```
import keras
from keras import layers

keras.utils.set_random_seed(0)

def count(shared_layer):
    left = keras.Input((4,))
    right = keras.Input((4,))
    if shared_layer:
        shared = layers.Dense(8, activation="relu")
        a, b = shared(left), shared(right)
    else:
        a = layers.Dense(8, activation="relu")(left)
        b = layers.Dense(8, activation="relu")(right)
    joined = layers.Concatenate()([a, b])
    model = keras.Model([left, right], [layers.Dense(1)(joined), layers.Dense(3)(joined)])
    dense = [layer for layer in model.layers if isinstance(layer, layers.Dense)]
    return model.count_params(), len(dense)

print("shared  :", count(True))
print("separate:", count(False))
```

```
shared  : (108, 3)
separate: (148, 4)
```

The separate model has 148 parameters, which is 40 more, and one more `Dense` layer. The lost property is the constraint that both inputs go through the same function. The shared model must give the same output for `(a, b)` and `(b, a)` in the first stage, apart from the order of the concatenation. The separate model can learn a different function for each side.

3. [judge]

Position: use the functional style by default and subclass only for a reason.

What a good answer weighs. The chapter measures one fact on each side. The functional model reports every shape in `summary()` before data arrives. The subclassed model reports `?` and zero parameters until it is called. The debugging argument is real: a subclass is plain Python and you can put a breakpoint in `call`. The functional style also lets you debug, but you debug the construction and not a loop of calls.

Strongest objection. A graph that has data-dependent control flow cannot be a functional model. A team that has such graphs will subclass anyway, and a single style across the code base has a value of its own.

What would overturn it. A count of shape bugs found at build time in the functional models of the team, against the time lost to indirection. If the count is zero over a year, the argument for the functional style is weak.

Solutions — Chapter 5

Exercises: 05 · chapter: Data pipelines

1. [measure]

```
import math

from examples.digits import load

x_train, _, y_train, _ = load()
n = len(x_train)
for batch_size in (32, 64, 100):
    batches = math.ceil(n / batch_size)
    last = n - (batches - 1) * batch_size
    print(f"batch size {batch_size:3d}: {batches} batches, last batch {last} images")
```

```
batch size  32: 43 batches, last batch 3 images
batch size  64: 22 batches, last batch 3 images
batch size 100: 14 batches, last batch 47 images
```

The number of batches is `ceil(1347 / batch_size)`. The last batch holds the remainder. A batch of 3 images gives a noisy gradient. If this matters, pass `drop_remainder=True` to `tf.data` or make `__len__` of the `PyDataset` use the floor.

2. [break]

```
import numpy as np
from keras import layers, ops
from examples.digits import load

x_train, x_test, _, _ = load(scale=False)
norm = layers.Normalization()
norm.adapt(x_train)

patched = x_test.copy()
patched[:, 24] = 0.0
print("original test images, largest output:",
float(np.abs(ops.convert_to_numpy(norm(x_test))).max()))
print("pixel 24 set to zero, largest output: ",
round(float(np.abs(ops.convert_to_numpy(norm(patched))).max()), 1))
```

```
original test images, largest output: 10000000.0
pixel 24 set to zero, largest output:  88.6
```

The huge value disappears because the data no longer contain it. The fix removes the evidence and keeps the cause. In production a pixel that was constant in training and is not constant now says that the input has changed: a new sensor, a new scanner, a new crop. A silent patch hides that message. The better fix is a check at the input that raises or logs when a feature leaves the range seen in training.

3. [judge]

Position: do not celebrate yet. A validation accuracy of 99% on a new data set is a reason to look for a fault first.

What a good answer weighs. The chapter shows that `validation_split` takes the last 20% of the rows. If the database query returns rows sorted by time, by customer or by label, the validation set is a different population from the training set, and the score can be too low or too high. A score that is too high has other causes, and the questions are the same ones that the chapter measures. Do rows of one customer appear in both parts? Did a statistic such as a mean come from all the rows? Does a feature encode the answer, such as an identifier or a time stamp? Ask for a stratified or grouped split with a seed and for the accuracy of a trivial baseline, such as the most common class.

Strongest objection. Sometimes the task is easy and 99% is correct. This is true. The remedy is to check, not to assume either way.

What would overturn it. The same score on a second split that groups the rows by the unit that must generalize, and a baseline that scores far lower.

Solutions — Chapter 6

Exercises: 06 · chapter: Losses, metrics and optimizers

1. [measure]

```
import keras
import numpy as np
from keras import layers
from examples.digits import load

x_train, x_test, y_train, y_test = load()
keras.utils.set_random_seed(0)
model = keras.Sequential([keras.Input((64,)), layers.Dense(64, activation="relu"),
layers.Dense(10)])
model.compile("adam", keras.losses.SparseCategoricalCrossentropy(from_logits=True), metrics=
["accuracy"])
model.fit(x_train, y_train, epochs=20, batch_size=32, verbose=0)

predicted = model.predict(x_test, verbose=0).argmax(axis=1)
print("class 4 predicted as 9:", int(((y_test == 4) & (predicted == 9)).sum()))
print("class 9 predicted as 4:", int(((y_test == 9) & (predicted == 4)).sum()))
print("all errors:", int((predicted != y_test).sum()), "of", len(y_test))
```

```
class 4 predicted as 9: 0
class 9 predicted as 4: 0
all errors: 13 of 450
```

The model makes 13 errors in 450 images. None of them is a 4 read as a 9 or a 9 read as a 4. The errors are in other pairs. A single accuracy number hides this, and a confusion matrix shows it.

2. [break]

```
import keras
from keras import layers
from examples.digits import load

x_train, x_test, y_train, y_test = load()
keras.utils.set_random_seed(0)
model = keras.Sequential([keras.Input((64,)), layers.Dense(64, activation="relu"),
layers.Dense(10)])
model.compile(
    "adam",
    keras.losses.SparseCategoricalCrossentropy(from_logits=False),
    metrics=["accuracy", keras.metrics.SparseTopKCategoricalAccuracy(k=3, name="top3")],
)
h = model.fit(x_train, y_train, epochs=20, batch_size=32,
              validation_data=(x_test, y_test), verbose=0).history
print("top-1:", round(h["val_accuracy"][-1], 4), " top-3:", round(h["val_top3"][-1], 4))
```

```
top-1: 0.5711  top-3: 0.8822
```

The top-1 accuracy is 0.5711, and the top-3 accuracy is 0.8822. The right class is in the first three for 88% of the images and first for 57%. The model has learned something, but less than the correct pairing of the chapter, which reaches 0.9711 at top-1.

This is the same number as the fourth row of the table in the chapter, 0.5711, and the extra metric did not change it. The chapter also records that an earlier run without the determinism setting gave 0.7089. A broken setup like this one is unstable, so do not quote one number from it.

Top-3 accuracy does not show the fault. A model with a bad top-1 and a fair top-3 is a reason to check that the loss and the output match.

3. [judge]

Position: do not ship on the last column. Choose by the whole curve and by a stated bound.

What a good answer weighs. The table of the chapter has a spread of 0.011 in the last column, which is 5 images of 450. The first epoch is where the optimizers differ by a wide margin, and that matters if the budget for training is small. The right question is what the team can afford: if training runs for 10 epochs, momentum SGD reaches the plateau first. If it runs for hundreds, the choice is almost free. Run each candidate with several seeds, and compare the spread within a candidate to the difference between candidates.

Strongest objection. The team has one test set and one budget. A study with several seeds costs time that the project does not have. This is true, and it is a reason to pick the default that is cheapest to tune, not a reason to read a 5-image gap as a result.

What would overturn it. A repeated run in which one optimizer wins by more than the spread over seeds on a held-out set.

Solutions — Chapter 7

Exercises: 07 · chapter: Callbacks and training control

1. [measure]

```
import keras
import numpy as np
from keras import layers
from examples.digits import load

x_train, x_test, y_train, y_test = load()
rows = np.random.default_rng(0).choice(len(x_train), 200, replace=False)
x_small, y_small = x_train[rows], y_train[rows]
loss = keras.losses.SparseCategoricalCrossentropy(from_logits=True)

for patience in (1, 5, 15):
    keras.utils.set_random_seed(0)
    model = keras.Sequential([keras.Input((64,)), layers.Dense(256, activation="relu"),
                              layers.Dense(256, activation="relu"), layers.Dense(10)])
    model.compile("adam", loss, metrics=["accuracy"])
    stopper = keras.callbacks.EarlyStopping(monitor="val_loss", patience=patience,
restore_best_weights=True)
    h = model.fit(x_small, y_small, epochs=100, batch_size=32,
                  validation_data=(x_test, y_test), callbacks=[stopper], verbose=0).history
    stop, best = len(h["val_loss"]), int(np.argmin(h["val_loss"])) + 1
    print(f"patience {patience:2d}: stops at epoch {stop}, best epoch {best}, difference {stop
- best}")
```

```
patience  1: stops at epoch 29, best epoch 28, difference 1
patience  5: stops at epoch 36, best epoch 31, difference 5
patience 15: stops at epoch 46, best epoch 31, difference 15
```

The run stops `patience` epochs after the best epoch. A small patience can stop at a local minimum of a noisy curve. A large patience wastes epochs. Look at the curve of a first run to choose the value.

2. [break]

```
import keras
import numpy as np
from keras import layers
from examples.digits import load

x_train, x_test, y_train, y_test = load()
rows = np.random.default_rng(0).choice(len(x_train), 200, replace=False)
x_small, y_small = x_train[rows], y_train[rows]
loss = keras.losses.SparseCategoricalCrossentropy(from_logits=True)

for mode in ("max", "min"):
    keras.utils.set_random_seed(0)
    model = keras.Sequential([keras.Input((64,)), layers.Dense(256, activation="relu"),
                              layers.Dense(256, activation="relu"), layers.Dense(10)])
    model.compile("adam", loss, metrics=["accuracy"])
    stopper = keras.callbacks.EarlyStopping(monitor="val_accuracy", mode=mode, patience=5,
                                            restore_best_weights=True)
    h = model.fit(x_small, y_small, epochs=100, batch_size=32,
                  validation_data=(x_test, y_test), callbacks=[stopper], verbose=0).history
    accuracy = model.evaluate(x_test, y_test, verbose=0)[1]
    print(f"mode={mode}: stops at epoch {len(h['val_accuracy'])}, accuracy of the returned
model {accuracy:.4f}")
```

```
mode=max: stops at epoch 17, accuracy of the returned model 0.9378
mode=min: stops at epoch 6, accuracy of the returned model 0.6444
```

With `mode="min"` the callback treats a lower accuracy as better. It stops the run at epoch 6 and returns a model with an accuracy of 0.6444, against 0.9378 for the intended setting. The code is valid and the run finishes without a message. `mode` is a direction, and a wrong value gives a worse model with no warning.

3. [judge]

Position: no. Report the accuracy on a third set that did not take part in any selection.

What a good answer weighs. The saved model is the best of 60 candidates, and "best" was judged on the validation set. The maximum of 60 noisy numbers is higher than the mean of the same numbers, even if every candidate is equal. The validation accuracy of the saved model is therefore an optimistic estimate. The size of the bias depends on the number of candidates and on the noise of one evaluation. With 450 validation images, one image is 0.0022, so 60 comparisons can add several images of bias.

Strongest objection. The bias may be small compared with other uncertainties, and a third set costs data that the team does not have. If data are scarce, cross-validation gives a less biased estimate at the cost of compute.

What would overturn it. A comparison, on a data set that is large enough, of the selection score and the score on a held-out third set. If the two agree within the noise, the bias is small for this setup.

Solutions — Chapter 8

Exercises: 08 · chapter: Overfitting and regularization

1. [measure]

```
import keras
import numpy as np
from keras import layers
from examples.digits import load

x_train, x_test, y_train, y_test = load()
rows = np.random.default_rng(0).choice(len(x_train), 200, replace=False)
x_small, y_small = x_train[rows], y_train[rows]
loss = keras.losses.SparseCategoricalCrossentropy(from_logits=True)

def test_accuracy(seed, dropout):
    keras.utils.set_random_seed(seed)
    stack = [keras.Input((64,))]
    for _ in range(2):
        stack.append(layers.Dense(256, activation="relu"))
        if dropout:
            stack.append(layers.Dropout(dropout))
    stack.append(layers.Dense(10))
    model = keras.Sequential(stack)
    model.compile("adam", loss, metrics=["accuracy"])
    model.fit(x_small, y_small, epochs=60, batch_size=32, verbose=0)
    return model.evaluate(x_test, y_test, verbose=0)[1]

for name, dropout in (("baseline", 0.0), ("dropout 0.5", 0.5)):
    scores = [test_accuracy(seed, dropout) for seed in (1, 2, 3)]
    print(f"{name:12s}", [round(s, 3) for s in scores], "spread", round(max(scores) -
min(scores), 3))
```

```
baseline     [0.933, 0.929, 0.933] spread 0.004
dropout 0.5  [0.927, 0.924, 0.929] spread 0.004
```

The spread inside a setting is 0.004, which is 2 images. The difference between the two settings is about 0.005. In the chapter, at seed 0, dropout 0.5 was 0.007 above the baseline. At seeds 1, 2 and 3 it is 0.004 to 0.006 below. The sign of the difference changes with the seed, so one seed cannot rank the two. A claim that is smaller than the spread is not a result.

2. [break]

```
import keras
import numpy as np
from keras import layers
from examples.digits import load

x_train, x_test, y_train, y_test = load()
keras.utils.set_random_seed(0)
model = keras.Sequential([keras.Input((64,)), layers.Dense(256, activation="relu"),
                          layers.Dropout(0.5), layers.Dense(10)])
batch = x_test[:100]
train_a = model(batch, training=True).numpy()
train_b = model(batch, training=True).numpy()
infer_a = model(batch, training=False).numpy()
infer_b = model(batch, training=False).numpy()
print("two training=True calls equal :", bool(np.array_equal(train_a, train_b)))
print("two training=False calls equal:", bool(np.array_equal(infer_a, infer_b)))
print("training vs inference equal   :", bool(np.array_equal(train_a, infer_a)))
```

```
two training=True calls equal : False
two training=False calls equal: True
training vs inference equal   : False
```

With `training=True`, dropout draws a new mask in each call, so the outputs differ from call to call. With `training=False` the layer does nothing, and the calls are equal. If a service calls the model with `training=True` by mistake, the same request gives different answers, and each answer is the output of a smaller random network. The mistake is easy to make in a custom loop and hard to see in a single request.

3. [judge]

Position: the manager is right for this model, and wrong as a general rule.

What a good answer weighs. The curve is flat from 800 to 1,347 images at 0.973 for this model on this task. More images of the same kind will not move it. The curve does not say that no data would help. A different model, with more capacity, can use more data, and new data of a kind that the test set lacks can show new errors. The curve is one model on one test set.

Strongest objection. A flat curve of two points is thin evidence. Each point comes from one subset and one seed. A curve with several subsets for each size would tell more.

What would overturn it. Repeat each size with five subsets. If the mean at 1,347 is still within the spread of the mean at 800, stop. If it is not, continue.

Solutions — Chapter 9

Exercises: 09 · chapter: Custom losses, metrics and training steps

1. [measure]

```
import keras
from keras import layers, ops
from examples.digits import load

class BalancedAccuracy(keras.metrics.Metric):
    def __init__(self, num_classes=10, name="balanced_accuracy", **kwargs):
        super().__init__(name=name, **kwargs)
        self.num_classes = num_classes
        self.hits = self.add_variable(shape=(num_classes,), initializer="zeros", name="hits")
        self.counts = self.add_variable(shape=(num_classes,), initializer="zeros",
name="counts")

    def update_state(self, y_true, y_pred, sample_weight=None):
        y_true = ops.cast(ops.reshape(y_true, (-1,)), "int32")
        predicted = ops.cast(ops.argmax(y_pred, axis=-1), "int32")
        hit = ops.cast(ops.equal(predicted, y_true), "float32")
        onehot = ops.one_hot(y_true, self.num_classes)
        self.hits.assign(self.hits + ops.sum(onehot * ops.expand_dims(hit, -1), axis=0))
        self.counts.assign(self.counts + ops.sum(onehot, axis=0))

    def result(self):
        return ops.mean(self.hits / ops.maximum(self.counts, 1.0))

    def reset_state(self):
        self.hits.assign(ops.zeros_like(self.hits))
        self.counts.assign(ops.zeros_like(self.counts))

x_train, x_test, y_train, y_test = load()
keras.utils.set_random_seed(0)
model = keras.Sequential([keras.Input((64,)), layers.Dense(64, activation="relu"),
layers.Dense(10)])
model.compile("adam", keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=[BalancedAccuracy()])
model.fit(x_train, y_train, epochs=3, batch_size=32, verbose=0)
for batch_size in (1, 450):
    result = model.evaluate(x_test, y_test, batch_size=batch_size, verbose=0,
return_dict=True)
    print(f"batch size {batch_size:3d}: balanced accuracy {result['balanced_accuracy']:.5f}")
```

```
batch size   1: balanced accuracy 0.89342
batch size 450: balanced accuracy 0.89342
```

The metric keeps the number of hits and the number of images of each class. These counts do not depend on how the images are grouped into batches. A metric that averaged a per-batch value would change with the batch size, and a batch of one image would give a meaningless per-class mean.

2. [break]

```
import keras
from keras import layers, ops
from sklearn.metrics import balanced_accuracy_score
from examples.digits import load

class Broken(keras.metrics.Metric):
    def __init__(self, num_classes=10, name="balanced_accuracy", **kwargs):
        super().__init__(name=name, **kwargs)
        self.num_classes = num_classes
        self.hits = self.add_variable(shape=(num_classes,), initializer="zeros", name="hits")
        self.counts = self.add_variable(shape=(num_classes,), initializer="zeros",
name="counts")

    def update_state(self, y_true, y_pred, sample_weight=None):
        y_true = ops.cast(ops.reshape(y_true, (-1,)), "int32")
        predicted = ops.cast(ops.argmax(y_pred, axis=-1), "int32")
        hit = ops.cast(ops.equal(predicted, y_true), "float32")
        onehot = ops.one_hot(y_true, self.num_classes)
        self.hits.assign(self.hits + ops.sum(onehot * ops.expand_dims(hit, -1), axis=0))
        self.counts.assign(self.counts + ops.sum(onehot, axis=0))

    def result(self):
        return ops.mean(self.hits / ops.maximum(self.counts, 1.0))

    def reset_state(self):
        pass

x_train, x_test, y_train, y_test = load()
keras.utils.set_random_seed(0)
model = keras.Sequential([keras.Input((64,)), layers.Dense(64, activation="relu"),
layers.Dense(10)])
model.compile("adam", keras.losses.SparseCategoricalCrossentropy(from_logits=True), metrics=
[Broken()])
history = model.fit(x_train, y_train, epochs=3, batch_size=32,
                    validation_data=(x_test, y_test), verbose=0).history
correct = balanced_accuracy_score(y_test, model.predict(x_test, verbose=0).argmax(axis=1))
print("broken, validation, per epoch:", [round(v, 4) for v in
history["val_balanced_accuracy"]])
print("correct value after epoch 3  :", round(float(correct), 4))
```

```
broken, validation, per epoch: [0.4308, 0.6082, 0.6929]
correct value after epoch 3  : 0.8934
```

Keras calls `reset_state` at the start of each epoch and of each evaluation. Without it, the counts of epoch 1 stay in the variables, and the value of epoch 3 is the recall over all three evaluations, which includes the weak model of epoch 1. The result is a number that looks plausible and is wrong. A custom metric needs a test against a reference, as the chapter does.

3. [judge]

Position: do not write a `train_step`. Use the built-in loss with `label_smoothing`.

What a good answer weighs. The chapter's rule is to use the smallest hook that does the job. Label smoothing is a change of the loss. The class `CategoricalCrossentropy` has the argument `label_smoothing`, and the chapter shows that a hand-written smoothed loss equals it. A custom `train_step` adds code that must be right in the forward pass, the gradient, the update and the metrics. The chapter shows four such details in one block. It also ties the project to one back end, and the team said that it is TensorFlow-only today.

Strongest objection. The sparse loss class does not have `label_smoothing`, so the team must convert the labels to one-hot first, or write the loss class of the chapter. This is a small cost compared with a custom step.

What would overturn it. A requirement that the smoothing must depend on something that a loss function cannot see, such as the epoch number. A callback that changes a variable in the loss would cover even that case, so the bar for a `train_step` is high.

Solutions — Chapter 10

Exercises: 10 · chapter: Convolutional networks

1. [measure]

```
import keras
from keras import layers

keras.utils.set_random_seed(0)

model = keras.Sequential([
    keras.Input((8, 8, 1)),
    layers.Conv2D(8, 3, padding="valid"),
    layers.MaxPooling2D(2),
    layers.Conv2D(16, 3, padding="valid"),
    layers.GlobalMaxPooling2D(),
])
for layer in model.layers:
    print(f"{layer.__class__.__name__:20s} {tuple(layer.output.shape)[1:]} weights
{layer.count_params()}")
print("convolution weights:", sum(l.count_params() for l in model.layers))
```

```
Conv2D               (6, 6, 8) weights 80
MaxPooling2D         (3, 3, 8) weights 0
Conv2D               (1, 1, 16) weights 1168
GlobalMaxPooling2D   (16,) weights 0
convolution weights: 1248
```

The first convolution has $3 \times 3 \times 1 \times 8 + 8 = 80$ weights. The second has $3 \times 3 \times 8 \times 16 + 16 = 1{,}168$. The pooling layers have none.

2. [break]

```
import keras
import numpy as np
from keras import layers
from examples.digits import load

x_train, x_test, y_train, y_test = load(images=True)
keras.utils.set_random_seed(0)
model = keras.Sequential([
    keras.Input((8, 8, 1)),
    layers.Conv2D(16, 3, padding="same", activation="relu"),
    layers.Conv2D(32, 3, padding="same", activation="relu"),
    layers.GlobalMaxPooling2D(),
    layers.Dense(10),
])
model.compile("adam", keras.losses.SparseCategoricalCrossentropy(from_logits=True), metrics=
["accuracy"])
model.fit(x_train, y_train, epochs=40, batch_size=32, verbose=0)

def shift_right(x, d):
    out = np.zeros_like(x)
    out[:, :, d:, :] = x[:, :, :-d, :] if d else x
    return out

for d in (1, 2, 3):
    cut = float((x_test[:, :, -d:, :].max(axis=(1, 2, 3)) > 0).mean())
    accuracy = model.evaluate(shift_right(x_test, d), y_test, verbose=0)[1]
    print(f"shift {d}: images that lose ink {cut:.3f}, accuracy {accuracy:.3f}")
```

```
shift 1: images that lose ink 0.140, accuracy 0.991
shift 2: images that lose ink 0.911, accuracy 0.973
shift 3: images that lose ink 0.996, accuracy 0.713
```

A shift moves the columns at the right edge out of the frame. The last column of each row of the table counts the images that lose ink in this way. The network cannot recover what is gone. The invariance of the chapter holds while the digit stays inside the 8 × 8 frame. The frame is small, so the limit is reached quickly.

3. [judge]

Position: the statement is too strong. Convolution is translation equivariant, and invariance needs a pooling step and has limits.

What a good answer weighs. The chapter measures three things. A convolution followed by flatten fell to 0.638 under a one-pixel shift. A convolution followed by global max pooling stayed at 0.991. The invariance broke when the digit left the frame. A grid also does not mean that position is irrelevant: a form, a chessboard and a spectrogram have meaning in the position. For those, a flatten or a position code is correct.

Strongest objection. Large images have much more room, and the pooling makes the shift limit unimportant in practice. This is true for natural images, and it is the reason CNNs work there. The chapter's data are small and the border effect is large.

What would overturn it. A measurement on the colleague's own images with several shift sizes, as in the solution to exercise 2.

Solutions — Chapter 11

Exercises: 11 · chapter: Sequences

1. [measure]

```
import keras
from keras import layers

keras.utils.set_random_seed(0)

d, u = 16, 32
by_hand = {"Embedding": 10 * 16, "GRU": 3 * (d * u + u * u + 2 * u), "Dense": 32 * 1 + 1}
print("by hand:", by_hand, "total", sum(by_hand.values()))

model = keras.Sequential([keras.Input((16,), dtype="int32"), layers.Embedding(10, 16),
                          layers.GRU(32), layers.Dense(1)])
print("count_params:", model.count_params())
```

```
by hand: {'Embedding': 160, 'GRU': 4800, 'Dense': 33} total 4993
count_params: 4993
```

The factor 3 is the three gates of a GRU: the reset gate, the update gate and the candidate. The `2 × u` is two bias vectors for each gate, which the Keras default `reset_after=True` uses. The two numbers agree.

2. [break]

```
import keras
import numpy as np
from keras import layers

rng = np.random.default_rng(0)

def make(n, length=16, vocabulary=10):
    x = rng.integers(3, vocabulary, size=(n, length))
    positions = np.array([rng.permutation(length)[:2] for _ in range(n)])
    x[np.arange(n), positions[:, 0]] = 1
    x[np.arange(n), positions[:, 1]] = 2
    return x.astype("int32"), (positions[:, 0] < positions[:, 1]).astype("int32"), positions

x_train, y_train, _ = make(6000)
x_test, y_test, positions = make(2000)

keras.utils.set_random_seed(0)
model = keras.Sequential([keras.Input((16,), dtype="int32"), layers.Embedding(10, 16),
                          layers.Conv1D(32, 3, activation="relu"),
layers.GlobalMaxPooling1D(),
                          layers.Dense(32, activation="relu"), layers.Dense(1)])
model.compile("adam", keras.losses.BinaryCrossentropy(from_logits=True))
model.fit(x_train, y_train, epochs=15, batch_size=64, verbose=0)
correct = (model.predict(x_test, verbose=0)[:, 0] > 0).astype("int32") == y_test

far = np.abs(positions[:, 0] - positions[:, 1]) >= 3
interior = ((positions >= 4) & (positions <= 11)).all(axis=1)
edge = ((positions <= 1) | (positions >= 14)).any(axis=1)
for name, chosen in (("interior pairs", far & interior), ("edge pairs", far & edge)):
    print(f"{name:15s} {int(chosen.sum()):4d} pairs, accuracy {correct[chosen].mean():.3f}")
```

```
interior pairs   250 pairs, accuracy 0.504
edge pairs       742 pairs, accuracy 1.000
```

The candidate is confirmed. Pairs with both tokens inside the sequence score 0.504, which is chance. Pairs with a token at the edge score 1.000. The `conv` model has no position input. It reads the edge from the way a token at the border appears in fewer windows. This is a leak of position that works only at the border, and it explains why the accuracy of the chapter's table did not fall steadily with the distance.

3. [judge]

Position: ship the bag-of-words model, and keep the recurrent model as a challenger.

What a good answer weighs. The gap is 0.01. On a test set of a few hundred transcripts, one transcript is a few thousandths, and a gap of 0.01 is within the noise of a single split. The chapter's task is built so that order is the only signal, and there the bag scores at chance. For topic classification, the words carry most of the signal, and the chapter gives no reason to expect that order matters. The bag is 60 times cheaper to train and simpler to run. The first step is to find out whether order carries signal in this data, for example with the bag-versus-order test of the chapter on the team's own labels.

Strongest objection. The recurrent model may help on the rare topics that are decided by phrases such as "not interested". A mean accuracy hides this. Look at the per-topic scores.

What would overturn it. A per-topic table in which the recurrent model wins on the topics that matter by more than the spread over seeds, or a business rule that errors on one topic cost much more than errors on the others.

Solutions — Chapter 12

Exercises: 12 · chapter: Attention and transformers

1. [measure]

```
import keras
import numpy as np
from keras import layers

keras.utils.set_random_seed(0)
heads, key_dim, width = 4, 16, 64
by_hand = 3 * (width * heads * key_dim + heads * key_dim) + (heads * key_dim * width + width)

layer = layers.MultiHeadAttention(num_heads=heads, key_dim=key_dim)
x = np.zeros((1, 5, width), dtype="float32")
layer(x, x)
print("by hand:", by_hand, " count_params:", layer.count_params())
```

```
by hand: 16640  count_params: 16640
```

Three projections for the query, the key and the value each have a kernel of `64 × 4 × 16` and a bias of `4 × 16`. The output projection has a kernel of `4 × 16 × 64` and a bias of 64.

2. [break]

```
import keras
import numpy as np
from keras import layers, ops

keras.utils.set_random_seed(0)

class TransformerBlock(layers.Layer):
    def __init__(self, width, heads, hidden, **kwargs):
        super().__init__(**kwargs)
        self.attention = layers.MultiHeadAttention(num_heads=heads, key_dim=width // heads)
        self.norm_1 = layers.LayerNormalization()
        self.feed_forward = keras.Sequential([layers.Dense(hidden, activation="relu"),
layers.Dense(width)])
        self.norm_2 = layers.LayerNormalization()

    def call(self, x):
        x = self.norm_1(x + self.attention(x, x))
        return self.norm_2(x + self.feed_forward(x))

class AddPositions(layers.Layer):
    def __init__(self, length, width, **kwargs):
        super().__init__(**kwargs)
        self.table = layers.Embedding(length, width)

    def call(self, x):
        return x + self.table(ops.arange(ops.shape(x)[1]))

def build(with_positions):
    keras.utils.set_random_seed(0)
    tokens = keras.Input((16,), dtype="int32")
    x = layers.Embedding(10, 32)(tokens)
    if with_positions:
        x = AddPositions(16, 32)(x)
    x = TransformerBlock(32, 2, 64)(x)
    return keras.Model(tokens, layers.GlobalAveragePooling1D()(x))

rng = np.random.default_rng(0)
batch = rng.integers(0, 10, size=(8, 16)).astype("int32")
shuffled = np.stack([row[rng.permutation(16)] for row in batch])
for with_positions in (False, True):
    model = build(with_positions)
    difference = np.abs(ops.convert_to_numpy(model(batch)) -
ops.convert_to_numpy(model(shuffled))).max()
    print(f"positions={with_positions!s:5s} largest difference after shuffling:
{float(difference):.6f}")
```

```
positions=False largest difference after shuffling: 0.000000
positions=True  largest difference after shuffling: 0.172792
```

Without positions the model computes a function of the set of tokens, so the order of the tokens cannot change the output. This is the reason that it stayed at chance on the order task. With positions, the order is part of the input and the outputs differ.

3. [judge]

Position: do not choose by the T^2 alone. Measure memory and time at 4,000 tokens, and consider a model that does not read all pairs.

What a good answer weighs. The chapter shows that the table of scores has $T \times T$ entries, so 4,000 tokens give 16 million entries for each head and each example. That is a real memory cost. A GRU reads one step at a time and has a cost that grows with T, but it also carries its memory through 4,000 steps, and the chapter shows that a long dependency is the case where attention is exact and a window is not. Logs often have structure: most of the signal is in a few lines. A cheap model that finds those lines first can cut T before attention.

Strongest objection. On the hardware of the team, a library kernel for attention can avoid the full table. This book did not measure that, and the objection is fair.

What would overturn it. A measurement on the team's data and hardware of memory and accuracy for both models at the real length.

Solutions — Chapter 13

Exercises: 13 · chapter: Transfer learning

1. [measure]

```
import keras
import numpy as np
from keras import layers

keras.utils.set_random_seed(0)
base = keras.Sequential([
    keras.Input((8, 8, 1)),
    layers.Conv2D(16, 3, padding="same", activation="relu"),
    layers.Conv2D(32, 3, padding="same", activation="relu"),
    layers.GlobalMaxPooling2D(),
])
base.trainable = False
model = keras.Sequential([base, layers.Dense(10)])
trainable = sum(int(np.prod(w.shape)) for w in model.trainable_weights)
print("trainable:", trainable, " all:", model.count_params())
```

```
trainable: 330  all: 5130
```

The head has 32 × 10 + 10 = 330 weights. The two convolutions have 160 and 4,640.

2. [break]

```
import keras
import numpy as np
from keras import layers

x = np.random.default_rng(0).random((32, 8, 8, 1)).astype("float32")
y = np.random.default_rng(1).integers(0, 10, size=(32,))
loss = keras.losses.SparseCategoricalCrossentropy(from_logits=True)

def run(recompile):
    keras.utils.set_random_seed(0)
    body = keras.Sequential([keras.Input((8, 8, 1)), layers.Conv2D(8, 3, activation="relu"),
                             layers.GlobalMaxPooling2D()])
    body.trainable = False
    model = keras.Sequential([body, layers.Dense(10)])
    model.compile("adam", loss)
    model.fit(x, y, epochs=1, batch_size=16, verbose=0)
    before = [w.copy() for w in body.get_weights()]
    body.trainable = True
    if recompile:
        model.compile("adam", loss)
    model.fit(x, y, epochs=1, batch_size=16, verbose=0)
    return any(not np.array_equal(a, b) for a, b in zip(before, body.get_weights()))

print("unfrozen without compile, base changed:", run(False))
print("unfrozen with compile,    base changed:", run(True))
```

```
unfrozen without compile, base changed: False
unfrozen with compile,    base changed: True
```

Keras keeps the training function from the first `fit`. The change of `trainable` has no effect until `compile`. This is the rule of Chapter 3, seen in a transfer setting. The mistake is easy: the model trains, the loss falls, and the base is still frozen.

3. [judge]

Position: freeze first and train the head. Fine-tune with a small learning rate only if the frozen result is not enough and a held-out set shows that fine-tuning helps.

What a good answer weighs. With 200 images, a model with millions of weights can memorize the set, and fine-tuning every layer is the case where that happens. The chapter's table gives the shape of the evidence: with few labels, the frozen base was ahead of the new network when the head was trained enough, and fine-tuning was in between. The first question is how far the vendor domain is from the team's domain. The chapter did not test a large domain gap, so it cannot say. The second question is the training budget of the head. The chapter's first table shows that an under-trained head can reverse the conclusion.

Strongest objection. A frozen base cannot adapt to a domain gap, so fine-tuning might be the only way to reach a useful score. This is true, and the way to learn it is to try both on a held-out set with several seeds.

What would overturn it. A held-out result in which fine-tuning beats the frozen base by more than the spread over seeds, with the same budget for the head.

Solutions — Chapter 14

Exercises: 14 · chapter: Saving, loading and export

1. [measure]

```
import io
import os
import tempfile
import zipfile

import h5py
import keras
import numpy as np
from keras import layers

keras.utils.set_random_seed(0)
model = keras.Sequential([keras.Input((8,)), layers.Dense(6, activation="relu"),
                          layers.Dense(5, activation="relu"), layers.Dense(2)])
model.compile("adam", "mse")
model.fit(np.zeros((8, 8), dtype="float32"), np.zeros((8, 2), dtype="float32"), verbose=0)
path = os.path.join(tempfile.mkdtemp(), "m.keras")
model.save(path)

weights = h5py.File(io.BytesIO(zipfile.ZipFile(path).read("model.weights.h5")))
print("weight arrays:", len(model.weights), " optimizer arrays:", len(weights["optimizer"]
["vars"]))
```

```
weight arrays: 6  optimizer arrays: 14
```

Adam keeps the step count and the learning rate, and two moving averages for each weight array. For 6 arrays that is 2 + 12 = 14. This is why an Adam checkpoint is three times larger than the weights alone.

2. [break]

```
import os
import tempfile

import keras
import numpy as np
from keras import layers, ops

x = np.random.default_rng(0).normal(size=(3, 5)).astype("float32")

def build(relu_after):
    keras.utils.set_random_seed(0)
    first = layers.Dense(4, activation="relu" if relu_after == 1 else None)
    second = layers.Dense(4, activation="relu" if relu_after == 2 else None)
    return keras.Sequential([keras.Input((5,)), first, second])

source, target = build(1), build(2)
path = os.path.join(tempfile.mkdtemp(), "m.weights.h5")
source.save_weights(path)
target.load_weights(path)
print("loaded without an error")
print("predictions equal:", bool(np.allclose(ops.convert_to_numpy(source(x)),
ops.convert_to_numpy(target(x)))))
```

```
loaded without an error
predictions equal: False
```

The weights file holds arrays, and the arrays fit by shape. Keras checks the shapes and not the function. A model with the same shapes and a different activation loads without a message and computes something else. Save the whole model, with its configuration, when you can.

3. [judge]

Position: give both, and say what each is for. The `.keras` file is for people who continue the work. The SavedModel is for the service.

What a good answer weighs. The chapter shows that the `.keras` file holds the configuration, the weights and the optimizer state, and that it needs Keras and every custom class to load. The exported model holds the computation and the weights, runs without Keras, and returned the same numbers as `predict`. It does not hold the optimizer, so it cannot continue training. The serving team usually needs the second. The research team needs the first.

Strongest objection. Two artifacts can drift apart. Make the export a step of the same script that saves the `.keras` file, and name both with the same version.

What would overturn it. A serving platform that loads Keras files directly and has your custom classes, which removes the reason for the export.

Solutions — Chapter 15

Exercises: 15 · chapter: Performance and reproducibility

1. [measure]

```
import os
import subprocess
import sys

env = {**os.environ, "TF_DETERMINISTIC_OPS": "1"}
losses = []
for _ in range(3):
    result = subprocess.run([sys.executable, "-m", "examples.determinism_probe"],
                            capture_output=True, text=True, env=env)
    losses.append(float(result.stdout.strip().splitlines()[-1].split()[1]))
print("runs:", len(losses), " largest difference below 1e-6:", max(losses) - min(losses) < 1e-
6)
```

```
runs: 3  largest difference below 1e-6: True
```

The losses agree to far better than 1e-6. The chapter's own measurement shows that the weights of two runs can still differ in their last bits, so the check is on the loss and with a tolerance.

2. [break]

```
import os

os.environ["TF_DETERMINISTIC_OPS"] = "1"

import keras
import numpy as np

x = np.ones((16, 3), dtype="float32")
y = np.ones((16, 1), dtype="float32")

def train():
    model = keras.Sequential([keras.Input((3,)), keras.layers.Dense(1)])
    model.compile("sgd", "mse")
    model.fit(x, y, epochs=1, batch_size=4, verbose=0)

try:
    train()
    print("without a seed: no error")
except RuntimeError as error:
    print("without a seed:", str(error)[:40])

keras.utils.set_random_seed(0)
train()
print("with a seed: no error")
```

```
without a seed: Random ops require a seed to be set when
with a seed: no error
```

With determinism on, a random operation without a seed is an error. Here the shuffle of the data in `fit` is the random operation. The line that fails is `fit`, and the message does not name the shuffle.

3. [judge]

Position: measure it on the project, and compare accuracy as well as speed.

What a good answer weighs. The chapter measured a change of the loss under mixed precision and no speed, because the machine had no GPU. A 1.8-times gain on one model and one GPU is a fact about that pair. It can be smaller for a small model, which spends its time in the input pipeline and not in the layers. The chapter also shows that the optimizer is wrapped in a loss-scaling optimizer, which is an extra moving part. Loss, accuracy and the variation over seeds must be compared at the same budget.

Strongest objection. On modern GPUs mixed precision is the default for a reason, and the loss differences are usually small. This is likely true, and the only evidence that counts here is the team's own.

What would overturn it. A run on the project's model and data with several seeds, in which the accuracy difference is smaller than the spread over seeds and the speed gain is at least as large as the colleague reports.

Solutions — Chapter 16

Exercises: 16 · chapter: A field guide

1. [measure]

```
import pathlib
import re

text = pathlib.Path("chapters/16-field-guide.md").read_text(encoding="utf-8")
table = text.split("## From symptom to cause")[1].split("The next block checks the table")[0]
rows = [line for line in table.splitlines() if line.startswith("| ") and "---" not in line]
print("rows including the header:", len(rows), " symptom rows:", len(rows) - 1)
```

```
rows including the header: 17  symptom rows: 16
```

The header row is not a symptom, so the table has one row fewer than the lines that start with a bar.

2. [break]

```
import pathlib
import re

root = pathlib.Path(".")
text = pathlib.Path("chapters/16-field-guide.md").read_text(encoding="utf-8")
broken = text.replace("(07-callbacks-and-training-control.md)", "(07-callbacks-and-training-
control-v2.md)", 1)
links = re.findall(r"\[(\d+)\]\((\d\d-[a-z0-9-]+\.md)\)", broken)
missing = sorted({name for _, name in links if not (root / "chapters" / name).exists()})
print("missing chapter files:", missing)
```

```
missing chapter files: ['07-callbacks-and-training-control-v2.md']
```

The check finds the one link that no longer points at a file. A check that passes on a correct table and fails on this one is a check that can be trusted to notice a rename.

3. [judge]

Position: a good answer names a symptom, a cause that one run can measure, and a number with a bound.

What a good answer weighs. The rows of the chapter's table share a shape: a plain description of what is seen, a cause, and a figure that comes from a run. A row without a figure is a story. Take a symptom from a real project, such as "the loaded model predicts differently on the server". Ask which single change of the setup moves it. The answer is a block of ten lines that prints two numbers that must be equal and are not.

Strongest objection. A table of symptoms cannot cover a field. This is true. The table is a way to organize measurements, and its value comes from the measurements and not from its size.

What would overturn it. A symptom that no single measurement can isolate, which needs a different method, such as a trace of the whole system.